CLOUD

Lecture 12

presentation

DAD – Distributed & Cloud App Development

Cristian Toma

D.I.C.E/D.E.I.C – Department of Economic Informatics & Cybernetics
www.dice.ase.ro

Cristian Toma – Business Card

Agenda for Lecture 12

Distributed Systems and Distributed Applications Development | Cloud

Distributed Systems

It’s not just about the programming, but providing smart solutions

DAD Sections & References

1.3 Distributed Systems

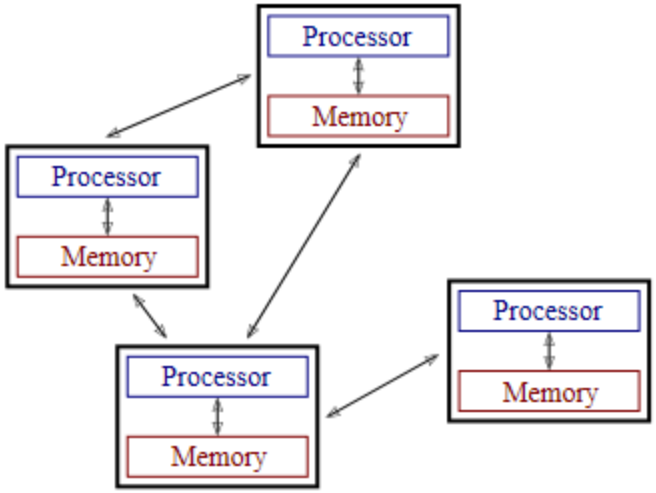
What is a Distributed System?

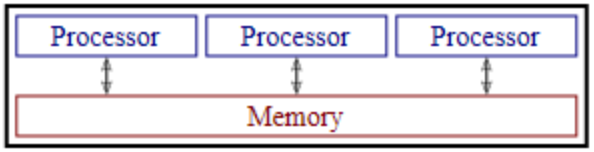
A distributed system is one in which components located into computers connected in network, communicate and coordinate their actions only by passing messages (sometimes, in addition, by migrating processes), in order to resolve a set of problems.

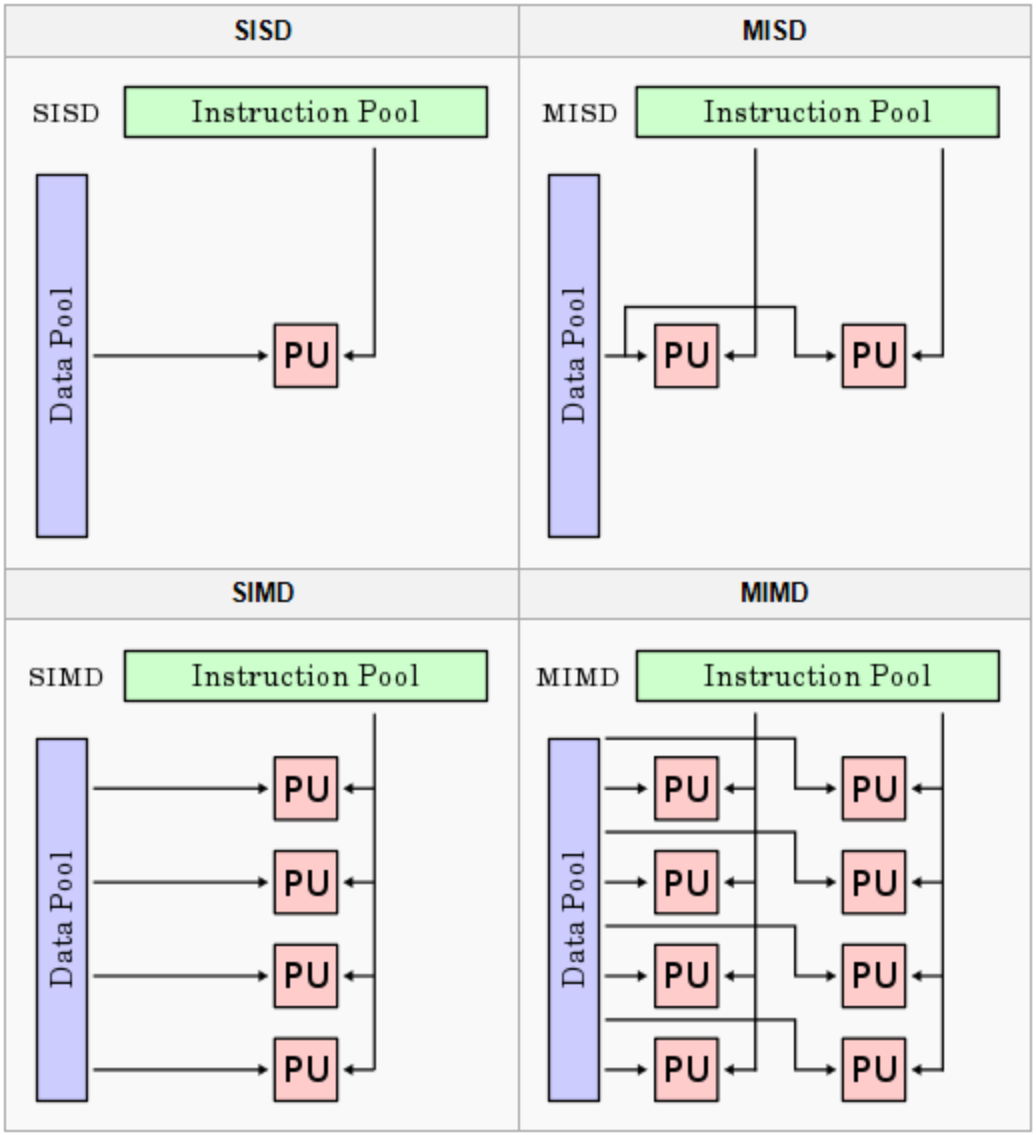
A distributed system consists of a collection of autonomous computers linked by a computer network and equipped with distributed system software. This software enables computers to coordinate their activities and to share the re- sources of the system hardware, software, and data.

Flynn Taxonomy
Parallel vs. Distributed Systems

Parallel vs. Distributed Computing / Algorithms







Where is the picture for:
Distributed System and ***Parallel System***?

http://en.wikipedia.org/wiki/Distributed_computing

http://en.wikipedia.org/wiki/Flynn's_taxonomy

1.3 Distributed Systems

How to characterize a distributed system?

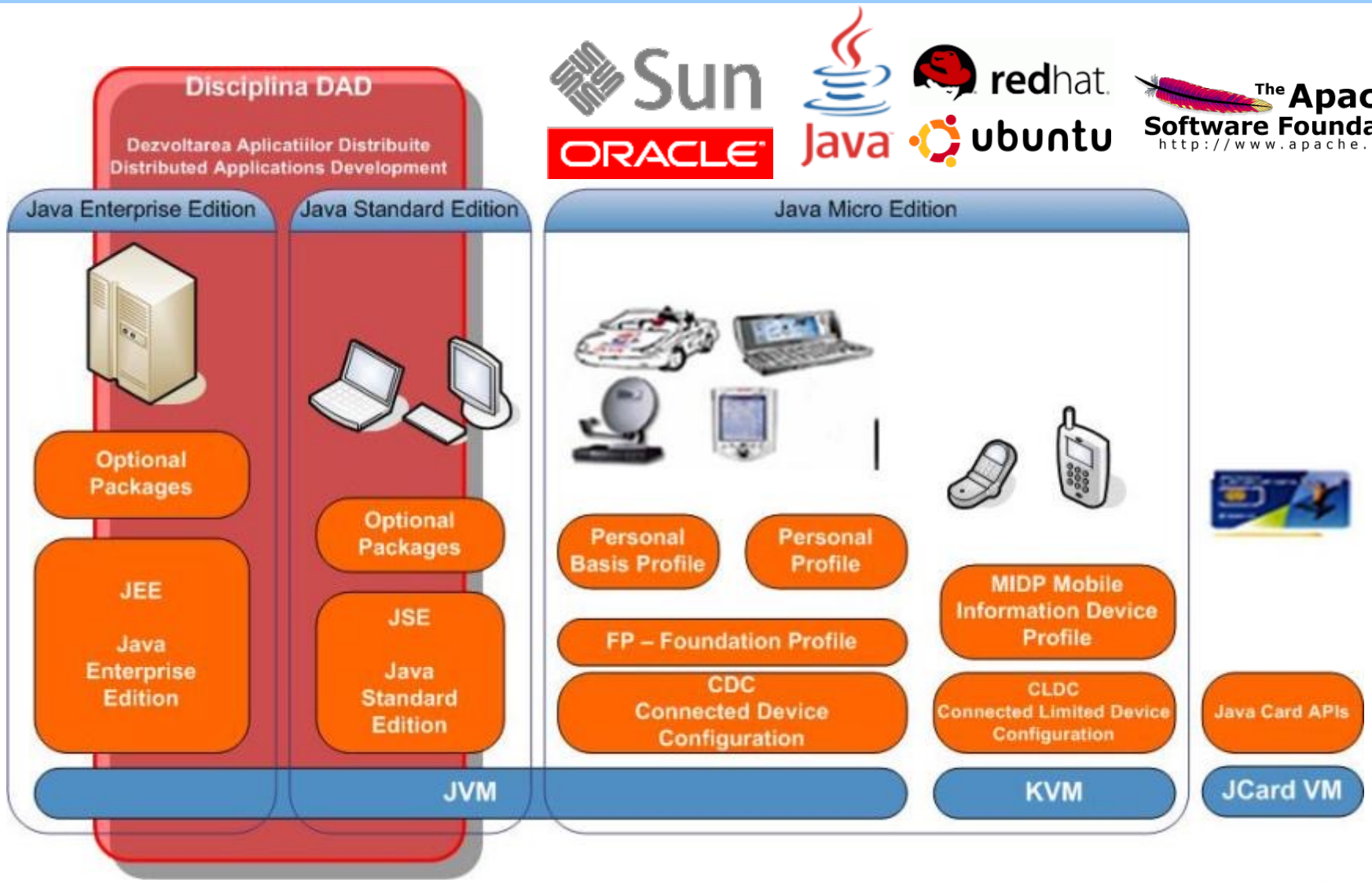
- concurrency of components
- lack of global clock
- independent failures of components

Leslie Lamport :-)

You know you have a distributed system when the crash of a computer you've never heard of stops you from getting any work done!

Prime motivation = to share the resources

Java – SE & EE 70% + C/C++ 5 % + Node.js 5 % + OS Admin 5 % + Distributed Frameworks 15 %



1.3 Distributed Systems

What are the challenges?

- heterogeneity of their components
- openness
- security
- scalability – the ability to work well when the load or the number of users increases
- failure handling
- concurrency of components
- transparency
- providing quality of service

1.3 Distributed Systems

What are the resources?

- Hardware
  - Not every single resource is for sharing
- Data
  - Databases
  - Proprietary software
  - Software production
  - Collaboration

Sharing Resources

• Different resources are handled in different ways, there are however some generic requirements:

– Namespace for identification

– Name translation to network address

– Synchronization of multiple access

1.3 Distributed Systems

Concept of Distributed Architecture

A distributed system can be demonstrated by the client-server architecture, which forms the base for multi-tier architectures; alternatives are the broker architecture such as CORBA, and the Service-Oriented Architecture (SOA). In this architecture, information processing is not confined to a single machine rather it is distributed over several independent computers.

There are several technology frameworks to support distributed architectures, including Java EE / Java-Kotlin Reactive Micro-services, .NET, CORBA, Scala Akka, Globus Toolkit Grid services, etc..

Middleware is an infrastructure that appropriately supports the development and execution of distributed applications.

1.3 Distributed Systems

Basis of Distributed Architecture

The basis of a distributed architecture is its transparency, reliability, and availability.

The following table lists the different forms of transparency in a distributed system

Transparency	Description
Access	Hides the way in which resources are accessed and the differences in data platform.
Location	Hides where resources are located.
Technology	Hides different technologies such as programming language and OS from user.
Migration / Relocation	Hide resources that may be moved to another location which are in use.
Replication	Hide resources that may be copied at several location.
Concurrency	Hide resources that may be shared with other users.
Failure	Hides failure and recovery of resources from user.
Persistence	Hides whether a resource (software) is in memory or disk.

1.3 Distributed Systems

Distributed Systems Advantages

It has following advantages:

- Resource sharing − Sharing of hardware and software resources.
- Openness − Flexibility of using hardware and software of different vendors.
- Concurrency − Concurrent processing to enhance performance.
- Scalability − Increased throughput by adding new resources.
- Fault tolerance − The ability to continue in operation after a fault has occurred.

Distributed Systems Disadvantages

Its disadvantages are:

- Complexity − They are more complex than centralized systems.
- Security − More susceptible to external attack.
- Manageability − More effort required for system management.
- Unpredictability − Unpredictable responses depending on the system organization and network load.

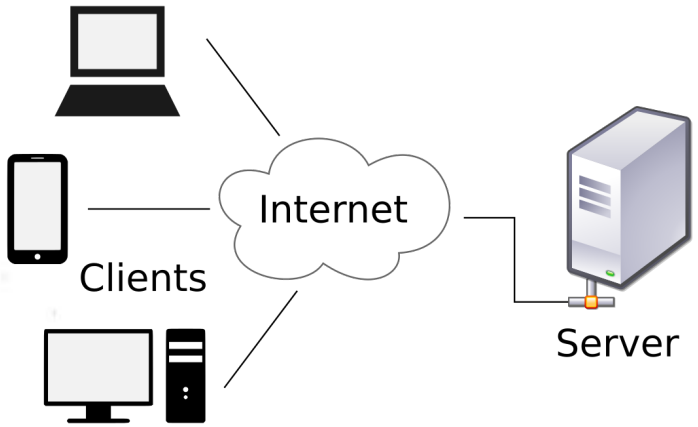
1.3 Distributed Systems

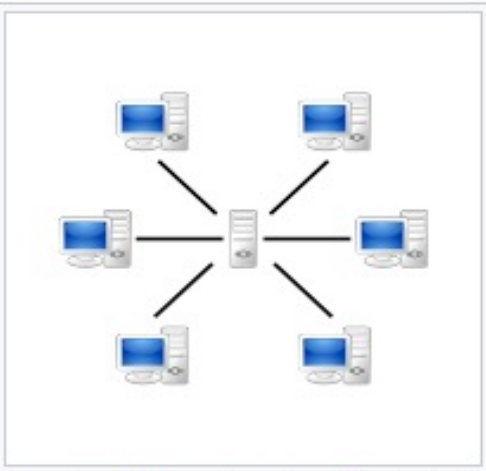
Client-Server Architecture

The client-server architecture is the most common distributed system architecture which decomposes the system into two major subsystems or logical processes −

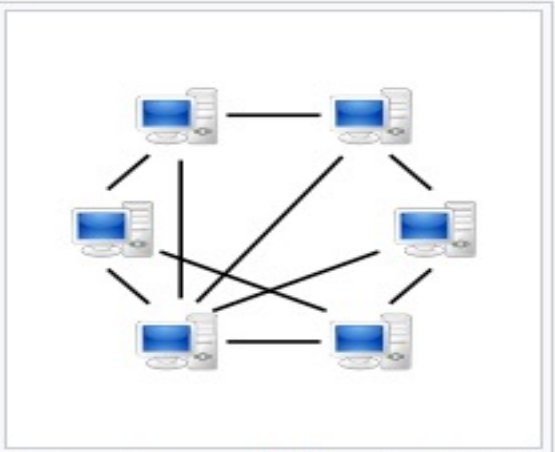
- Client − This is the first process that issues a request to the second process i.e. the server.
- Server − This is the second process that receives the request, carries it out, and sends a reply to the client.

In this architecture, the application is modelled as a set of services those are provided by servers and a set of clients that use these services. The servers need not to know about clients, but the clients must know the identity of servers.





A network based on the **client-server model**, where individual *clients* request services and resources from centralized servers



A **peer-to-peer (P2P) network** in which interconnected nodes ("peers") share resources amongst each other without the use of a centralized administrative system

1.3 Distributed Systems

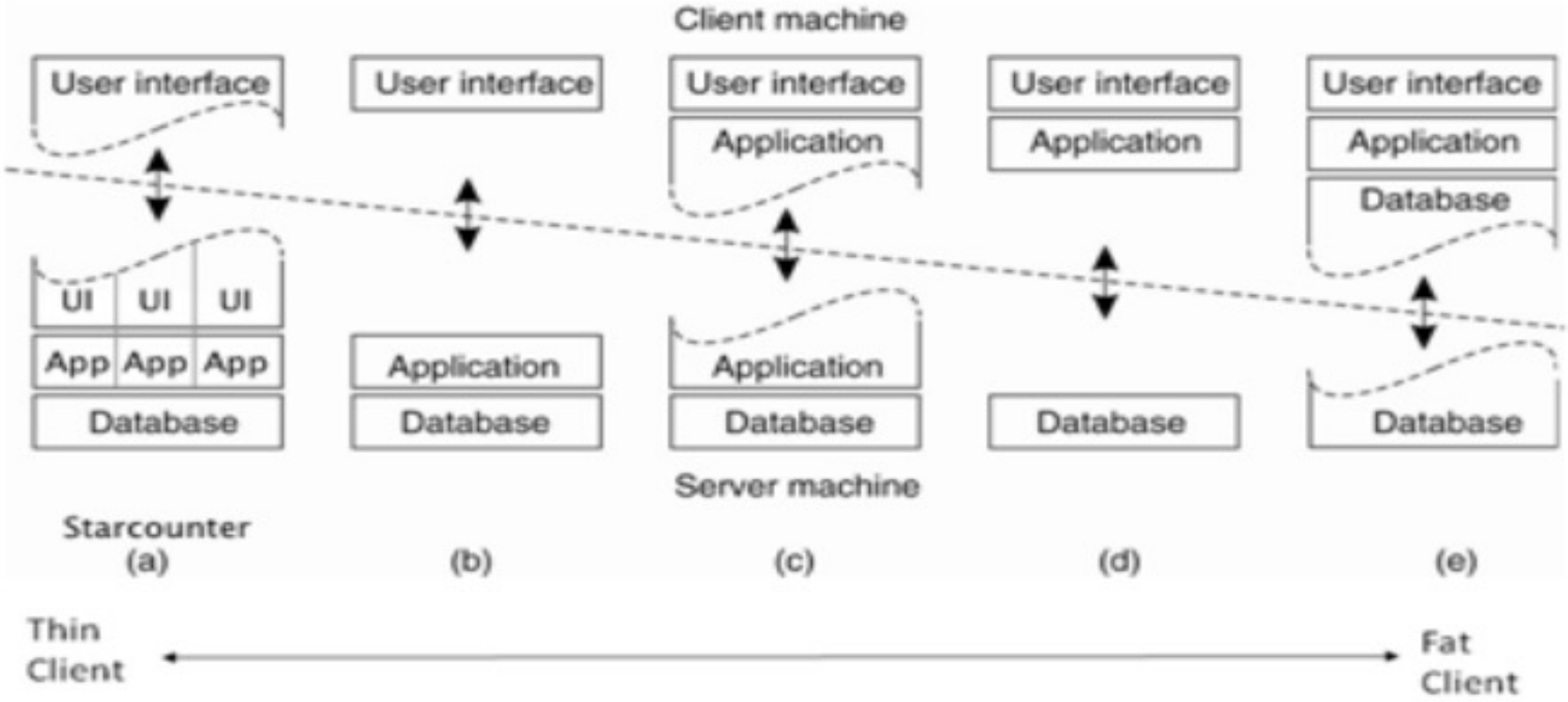
Thin-client model

In thin-client model, all the application processing and data management is carried by the server. The client is simply responsible for running the GUI software. It is used when legacy systems are migrated to client server architectures in which legacy system acts as a server in its own right with a graphical interface implemented on a client.
However, A major disadvantage is that it places a heavy processing load on both the server and the network.

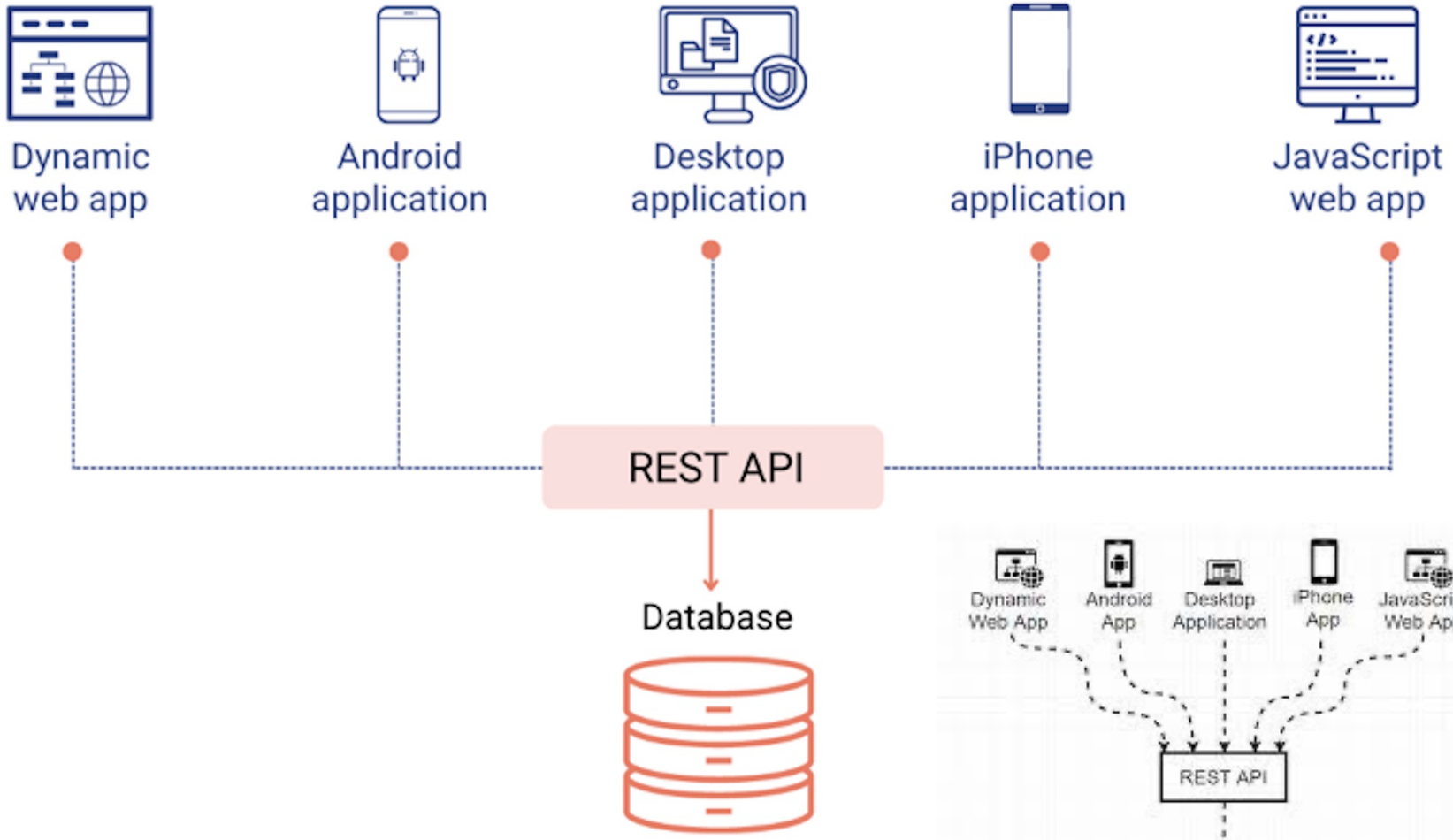
Thick/Fat-client model

In thick-client model, the server is in-charge of the data management. The software on the client implements the application logic and the interactions with the system user. It is most appropriate for new client-server systems where the capabilities of the client system are known in advance.
However, it is more complex than a thin client model especially for management, as all clients should have same copy/version of software application.

1.3 Distributed Systems Thin vs. Thick/Fat Client

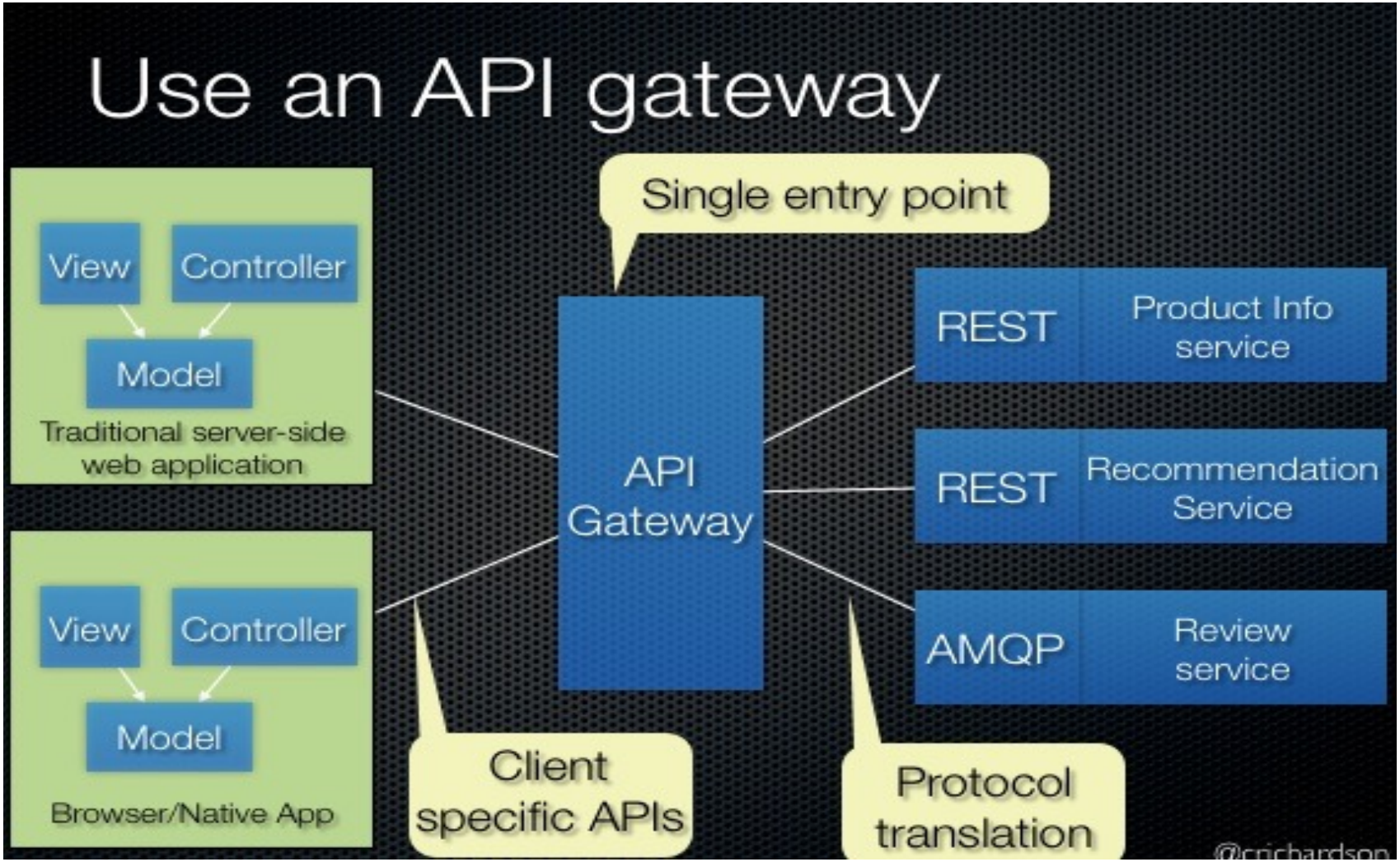


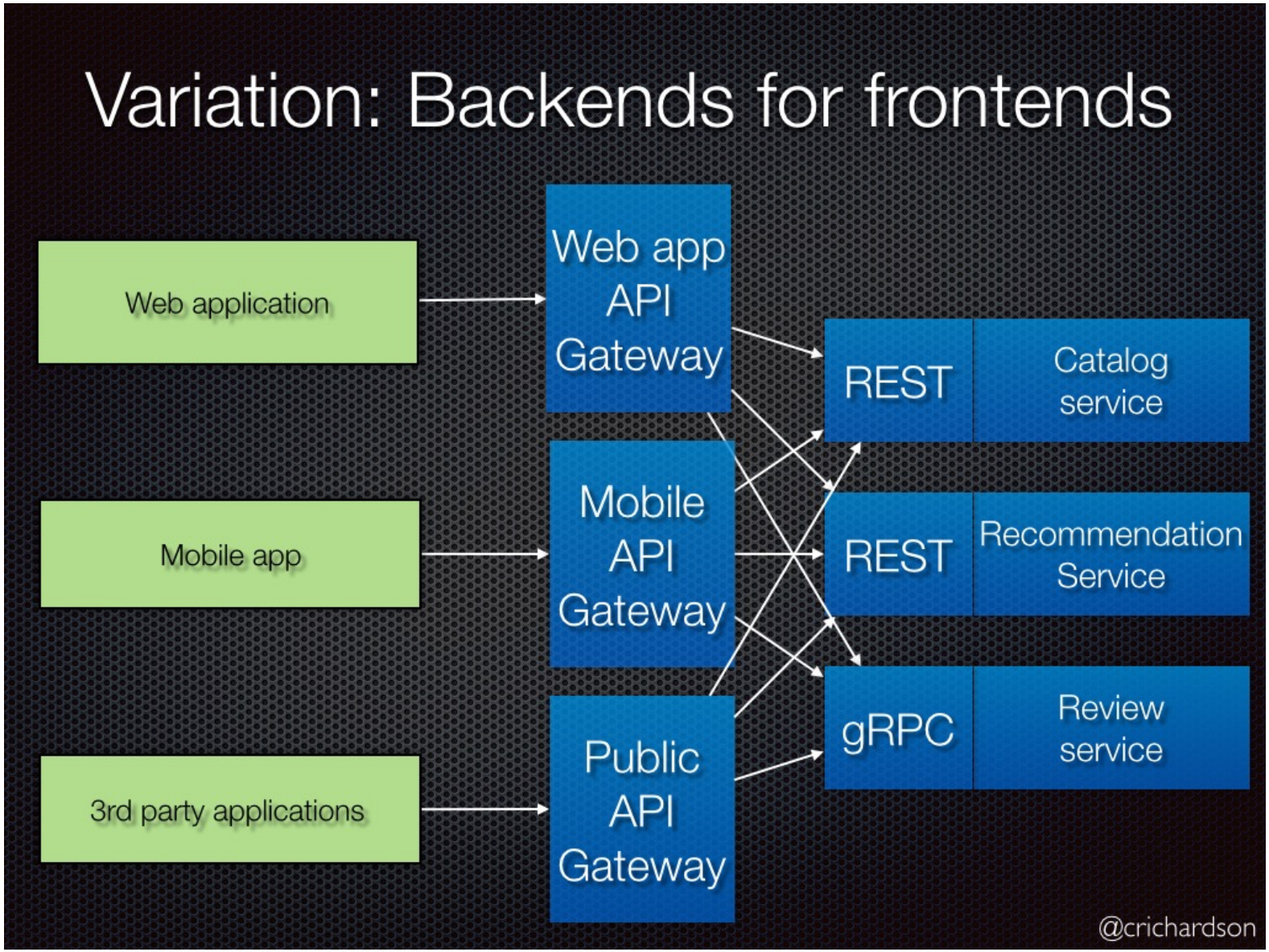
RESTful/REST API



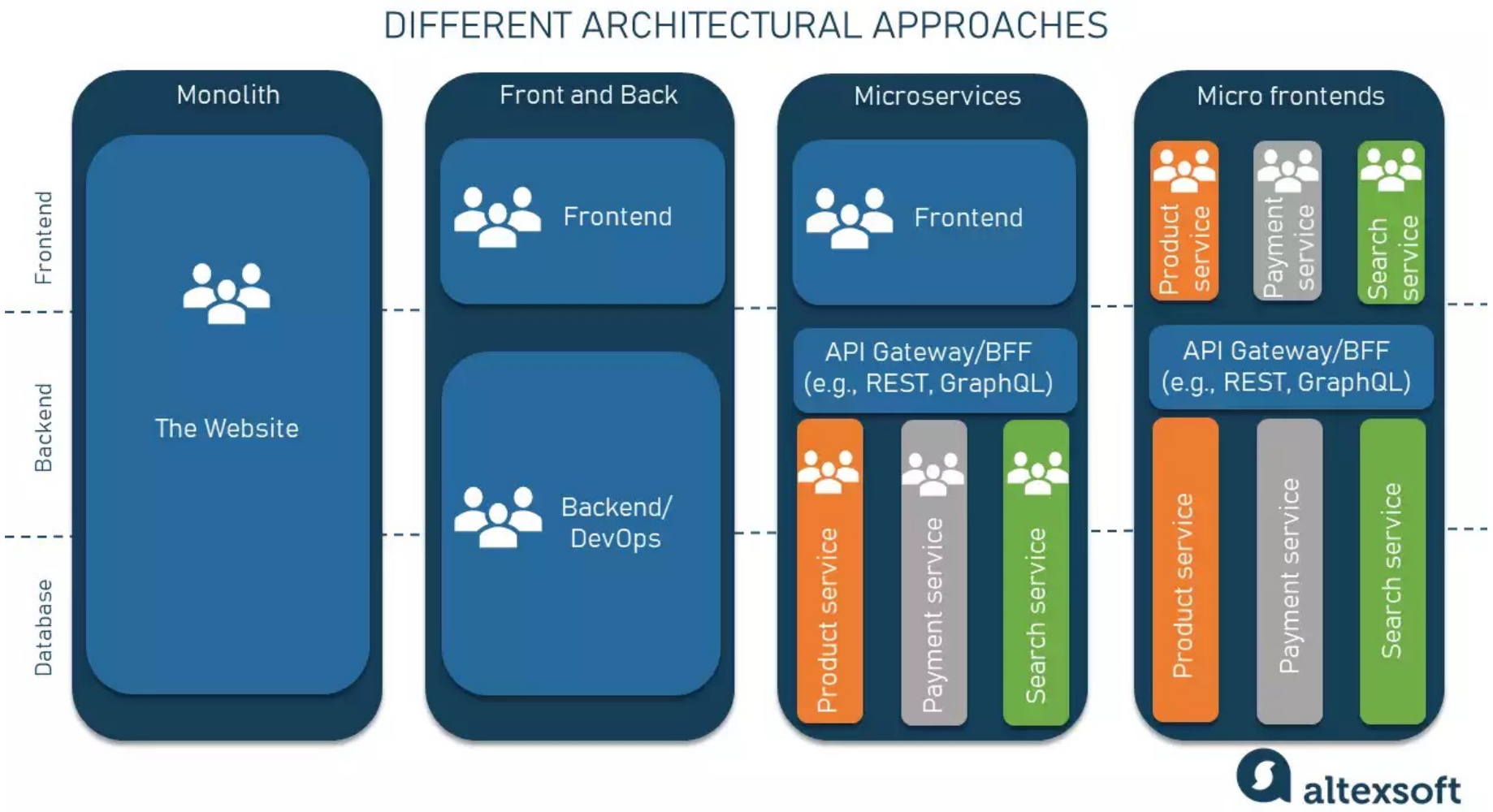
1.3 Distributed Systems

REST API – API GW/BFF (Backends for Front-ends)



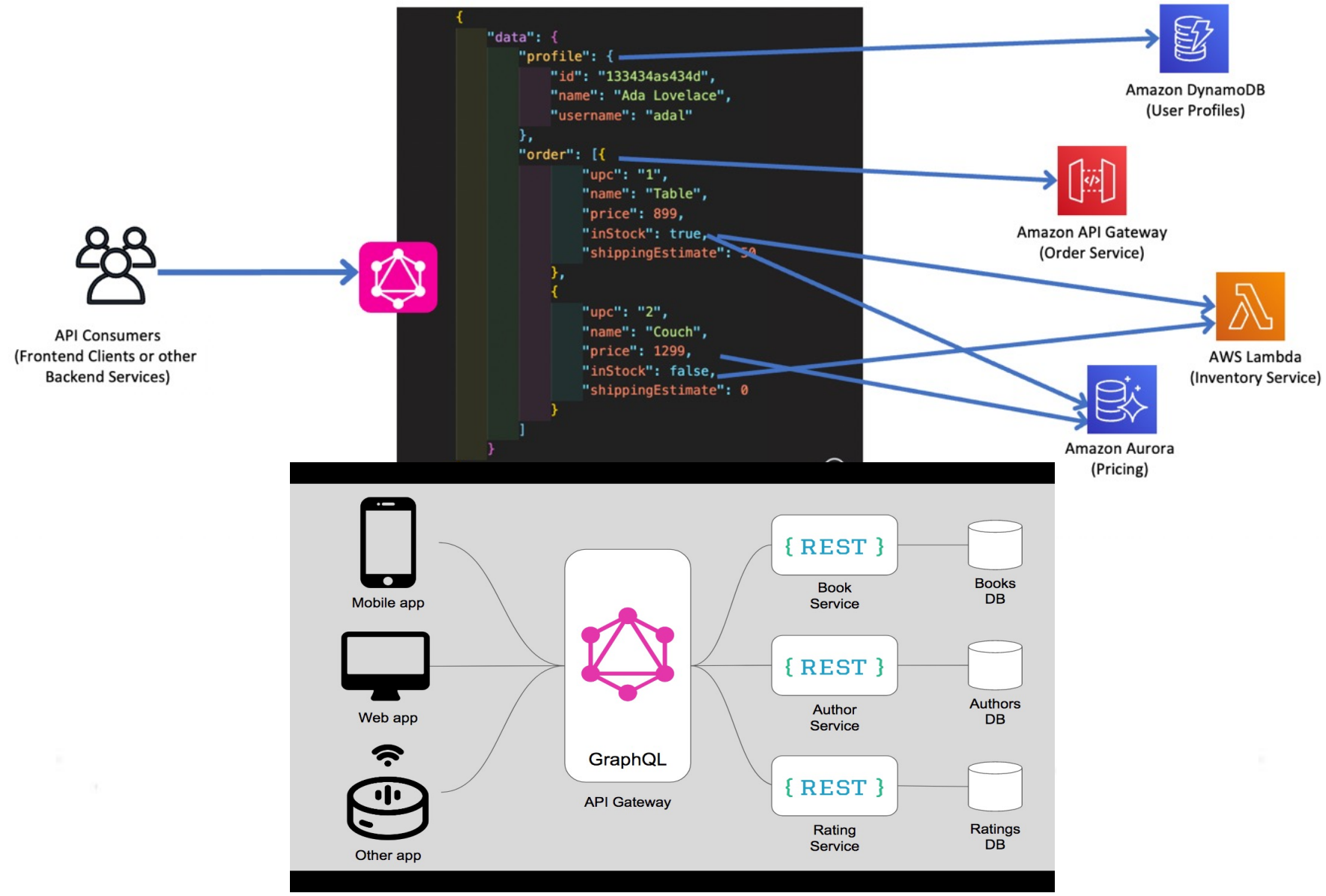


Microservices – API GW

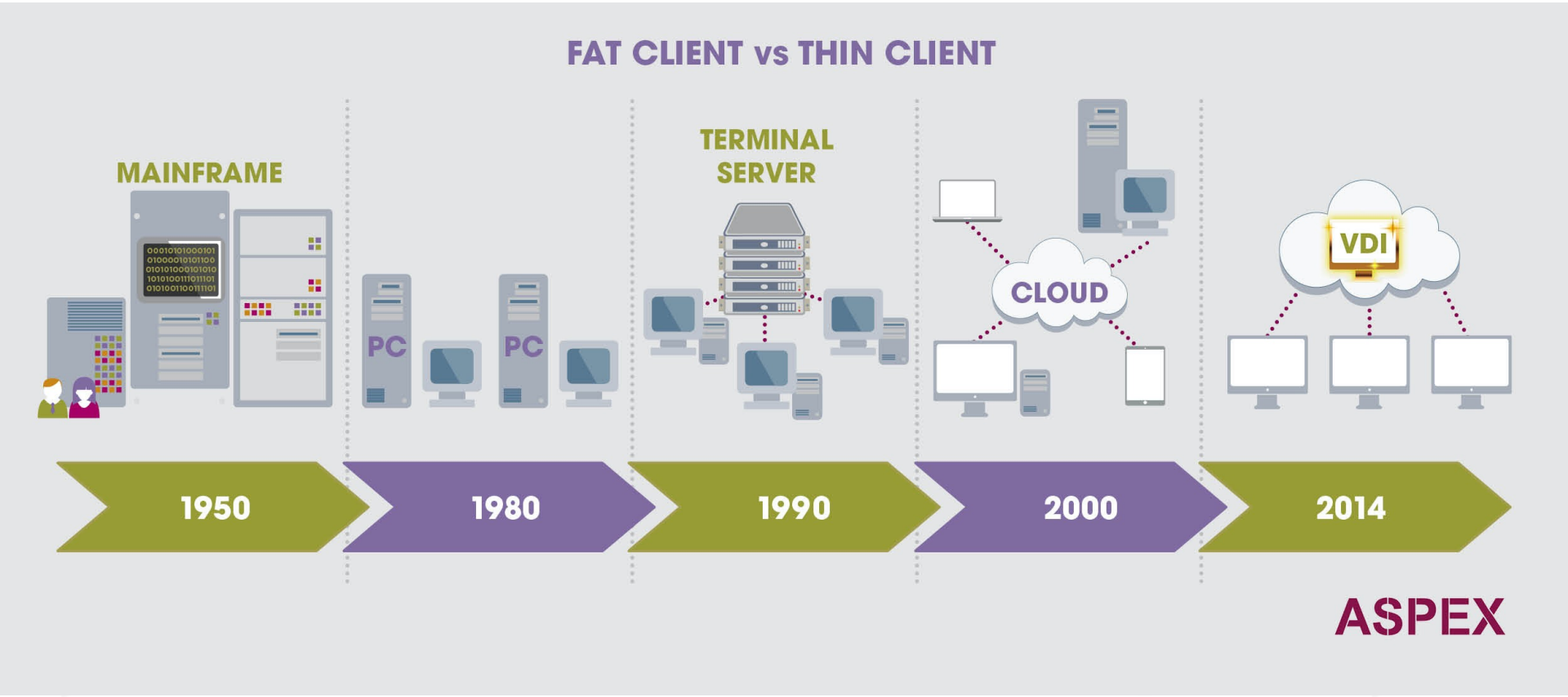


1.3 Distributed Systems

RESTful/REST API vs. GraphQL



Thin vs. Thick/Fat Client



1.3 Distributed Systems

Thin/Thick Clients Advantages:

- Separation of responsibilities such as user interface presentation and business logic processing.
- Reusability of server components and potential for concurrency
- Simplifies the design and the development of distributed applications
- It makes it easy to migrate or integrate existing applications into a distributed environment.
- It also makes effective use of resources when a large number of clients are accessing a high-performance server.

Thin/Thick Clients Disadvantages:

- Lack of heterogeneous infrastructure to deal with the requirement changes.
- Security complications.
- Limited server availability and reliability.
- Limited testability and scalability.
- Fat clients with presentation and business logic together.

1.3 Distributed Systems

Multi-Tier Architecture (n-tier Architecture)

Multi-tier architecture is a client–server architecture in which the functions such as presentation, application processing, and data management are physically separated. By separating an application into tiers, developers obtain the option of changing or adding a specific layer, instead of reworking the entire application. It provides a model by which developers can create flexible and reusable applications.

The most general use of multi-tier architecture is the ***three-tier architecture***. A three-tier architecture is typically composed of a presentation tier, an application tier, and a data storage tier and may execute on a separate processor.

Presentation Tier

Presentation layer is the topmost level of the application by which users can access directly such as webpage or Operating System GUI (Graphical User interface). The primary function of this layer is to translate the tasks and results to something that user can understand. It communicates with other tiers so that it places the results to the browser/client tier and all other tiers in the network.

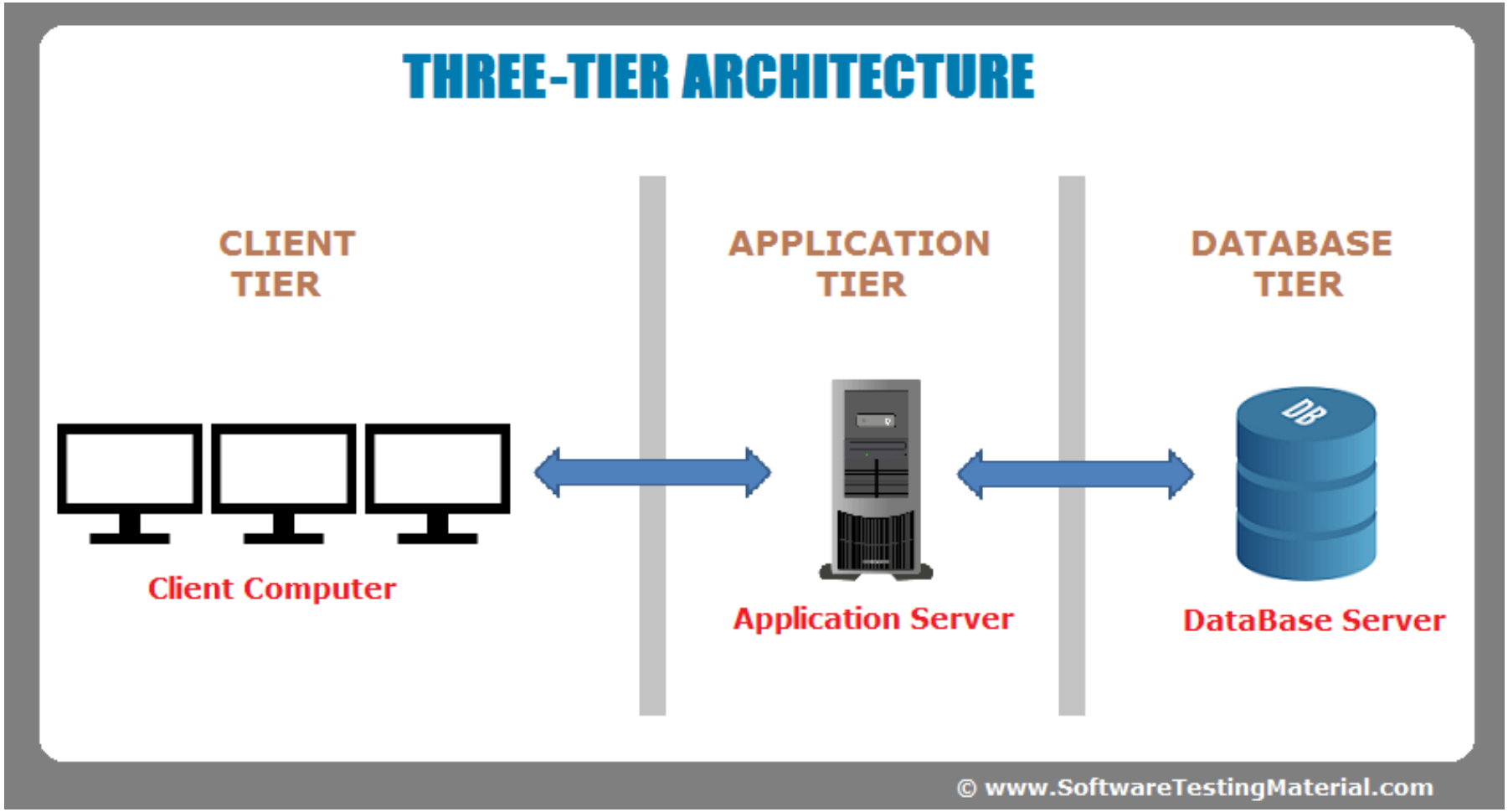
Application Tier (Business Logic, Logic Tier, or Middle Tier)

Application tier coordinates the application, processes the commands, makes logical decisions, evaluation, and performs calculations. It controls an application's functionality by performing detailed processing. It also moves and processes data between the two surrounding layers.

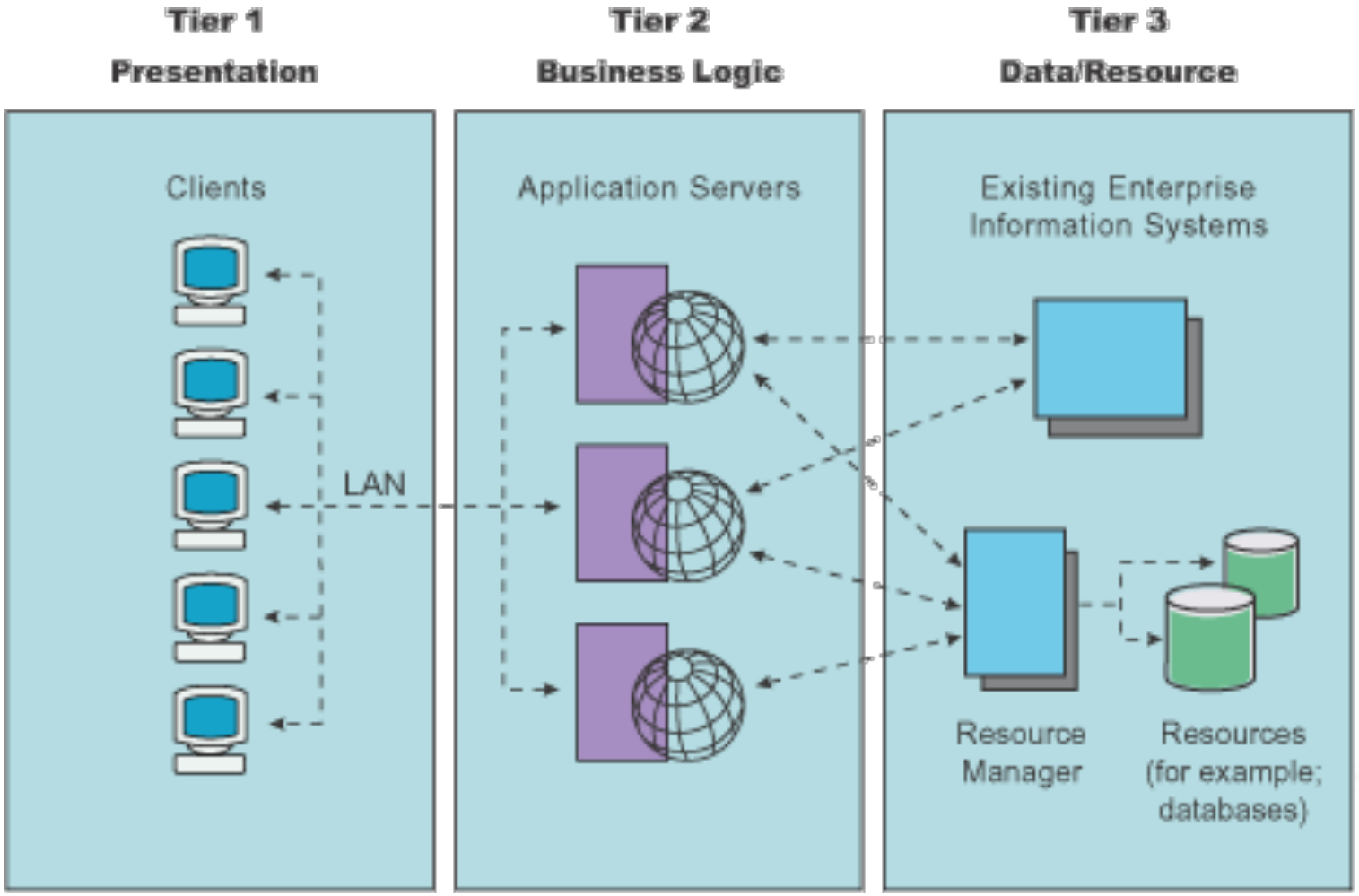
Data Tier

In this layer, information is stored and retrieved from the database or file system. The information is then passed back for processing and then back to the user. It includes the data persistence mechanisms (database servers, file shares, etc.) and provides API (Application Programming Interface) to the application tier which provides methods of managing the stored data.

3-Tier Architecture

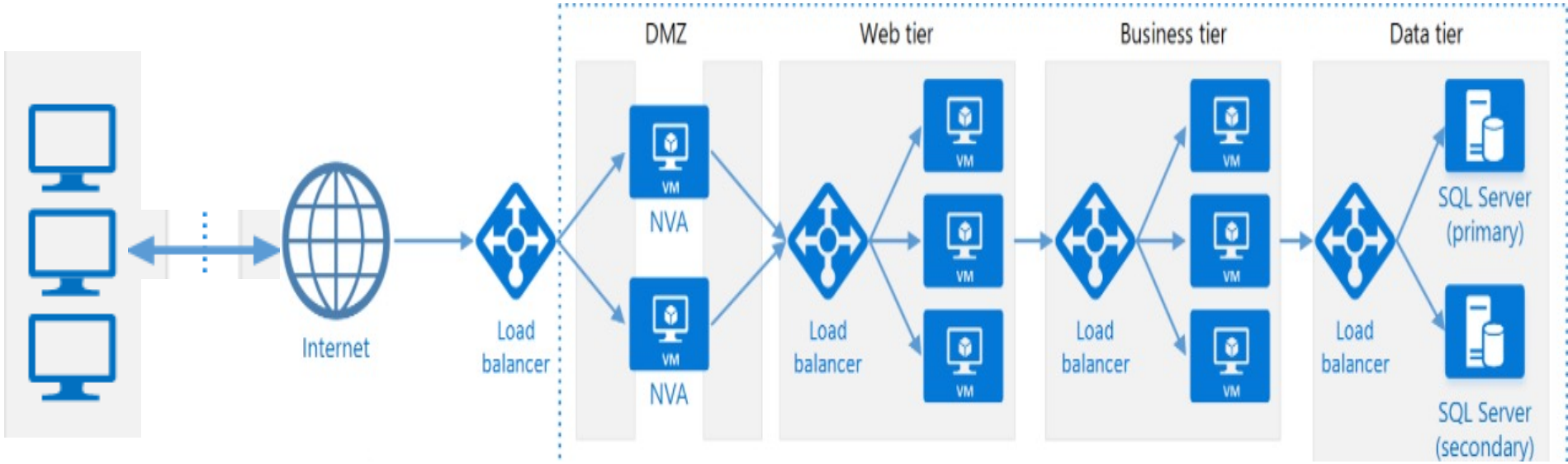


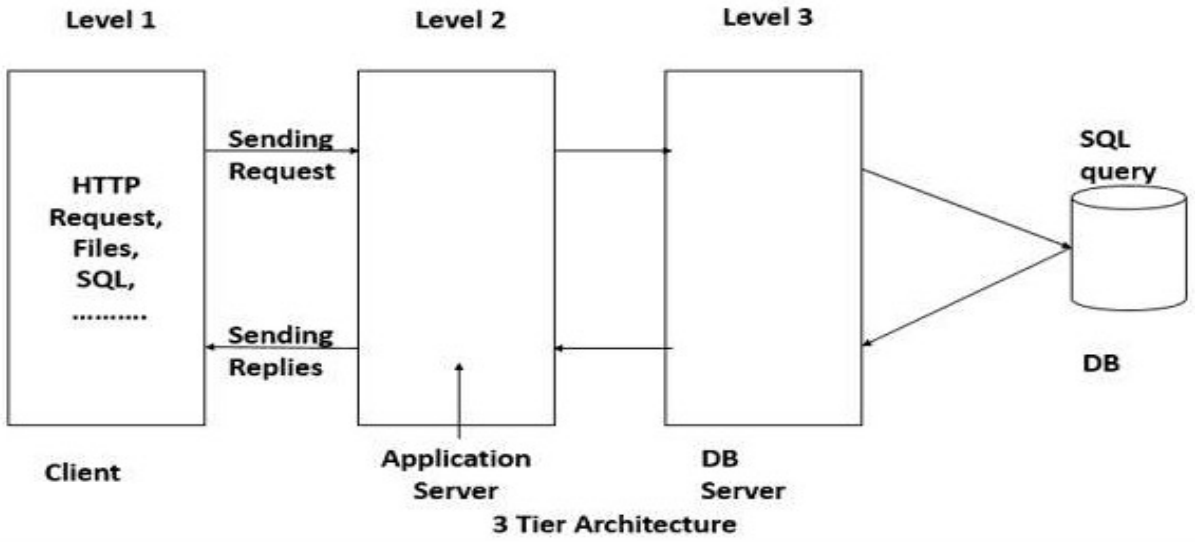
3-Tier Architecture



1.3 Distributed Systems

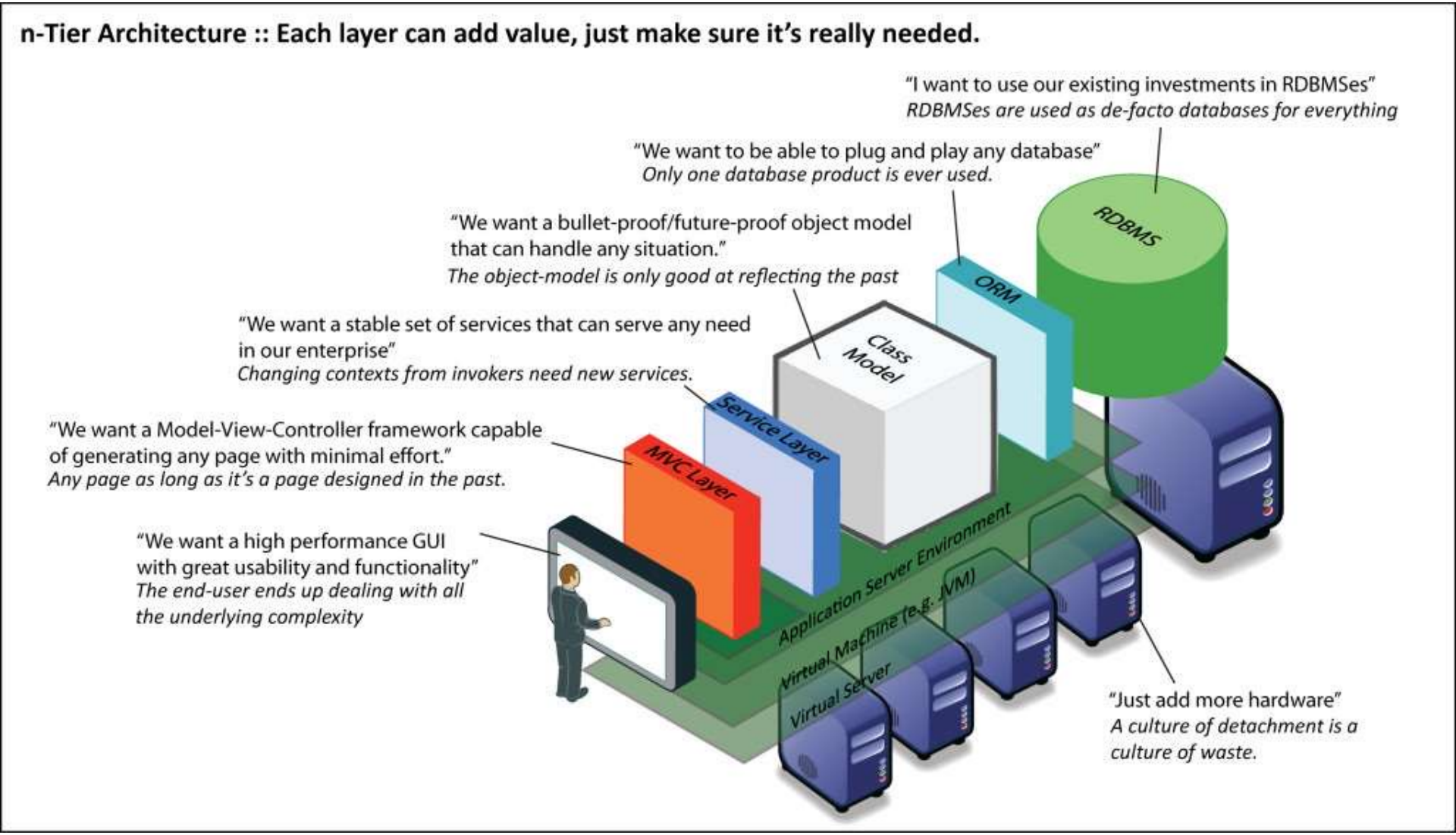
3-Tier/4-Tier Architecture





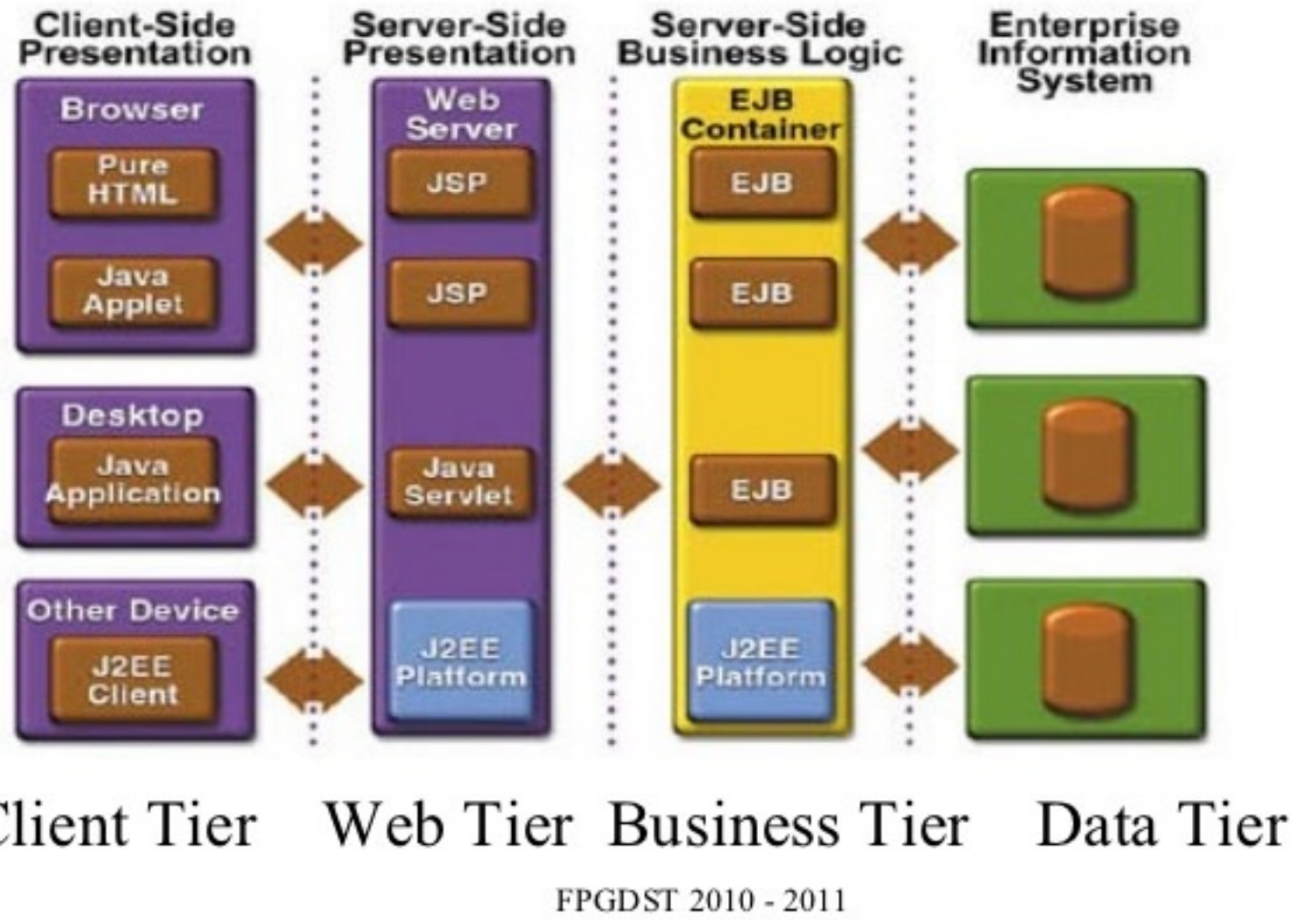
1.3 Distributed Systems

n-Tier Architecture



1.3 Distributed Systems

n-Tier Architecture



1.3 Distributed Systems

n-Tier Architecture

Advantages

- Better performance than a thin-client approach and is simpler to manage than a thick-client approach.
- Enhances the reusability and scalability − as demands increase, extra servers can be added.
- Provides multi-threading support and also reduces network traffic.
- Provides maintainability and flexibility.

Disadvantages

- Unsatisfactory Testability due to lack of testing tools.
- More critical server reliability and availability.

1.3 Distributed Systems

Broker Architectural Style

Broker Architectural Style is a middleware architecture used in distributed computing to coordinate and enable the communication between registered servers and clients. Here, object communication takes place through a middleware system called an object request broker (software bus).

However, client and the server do not interact with each other directly. Client and server have a direct connection to its proxy, which communicates with the mediator-broker. A server provides services by registering and publishing their interfaces with the broker and clients can request the services from the broker statically or dynamically by look-up.

CORBA (Common Object Request Broker Architecture) is a good implementation example of the broker architecture.

Broker Architectural Components

Components of Broker Architectural Style - The components of broker architectural style are discussed through following heads:

Broker

Broker is responsible for brokering the service requests, locating a proper server, transmitting requests, and sending responses back to clients. It also retains the servers' registration information including their functionality and services as well as location information.

Further, it provides APIs for clients to request, servers to respond, registering or unregistering server components, transferring messages, and locating servers.

Stub

Stubs are generated at the static compilation time and then deployed to the client side which is used as a proxy for the client. Client-side proxy acts as a mediator between the client and the broker and provides additional transparency between them and the client; a remote object appears like a local one.

The proxy hides the IPC (inter-process communication) at protocol level and performs marshaling of parameter values and un-marshaling of results from the server.

Skeleton

Skeleton is generated by the service interface compilation and then deployed to the server side, which is used as a proxy for the server. Server-side proxy encapsulates low-level system-specific networking functions and provides high-level APIs to mediate between the server and the broker.

Further, it receives the requests, unpacks the requests, unmarshals the method arguments, calls the suitable service, and also marshals the result before sending it back to the client.

Bridge

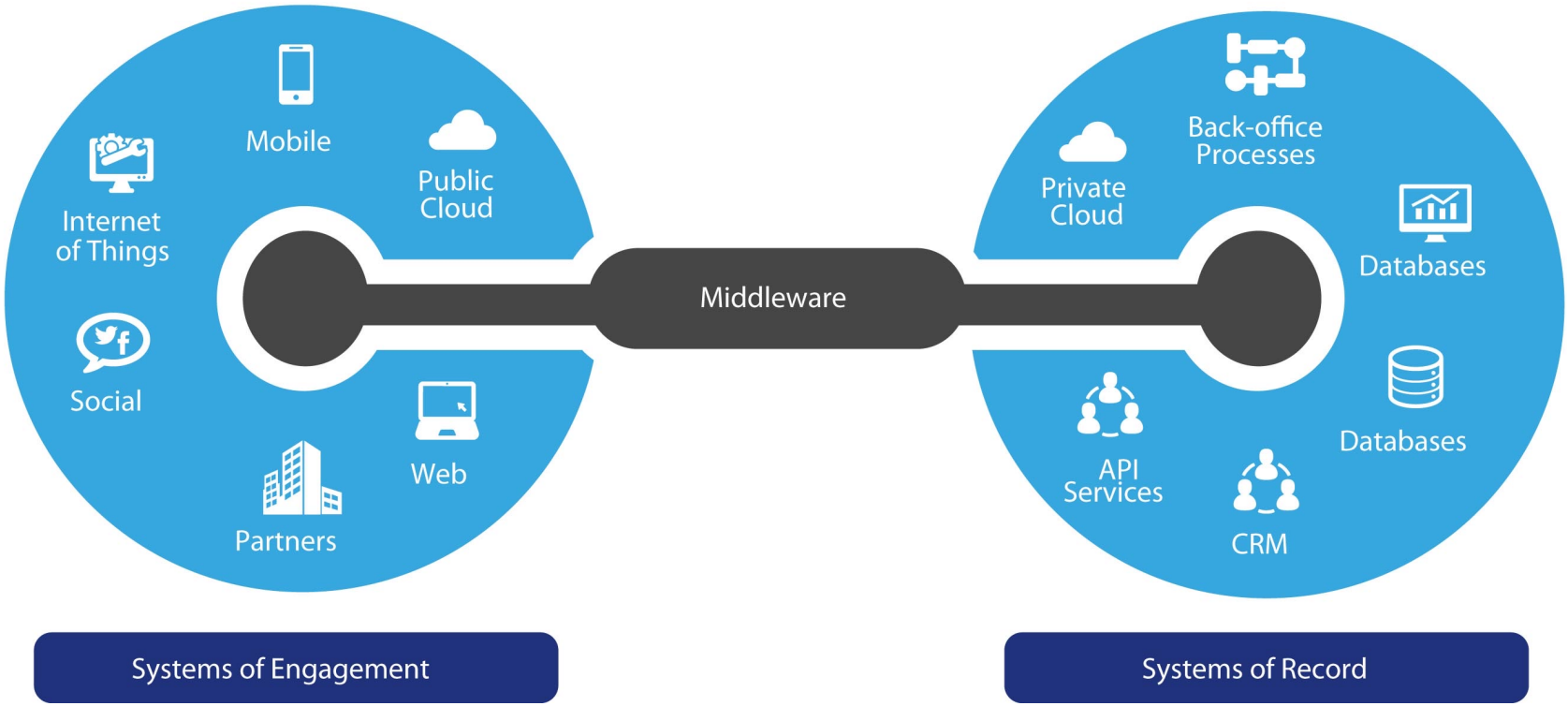
A bridge can connect two different networks based on different communication protocols. It mediates different brokers including DCOM, .NET remote, and Java CORBA brokers.

Bridges are optional component, which hides the implementation details when two brokers interoperate and take requests and parameters in one format and translate them to another format.

1.3 Distributed Systems

Middleware API

Middleware is computer software that provides services to software applications beyond those available from the operating system. It can be described as "software glue".



1.3 Distributed Systems

Middleware

The term is most commonly used for software that enables communication and management of data in distributed applications. An IETF workshop in 2000 defined middleware as "those services found above the transport (i.e. over TCP/IP) layer set of services but below the application environment" (i.e. below application-level APIs).[3] In this more specific sense *middleware* can be described as the dash ("-") in *client-server*, or the -*to*- in *peer-to-peer*.[*citation needed*] Middleware includes web servers, application servers, content management systems, and similar tools that support application development and delivery.

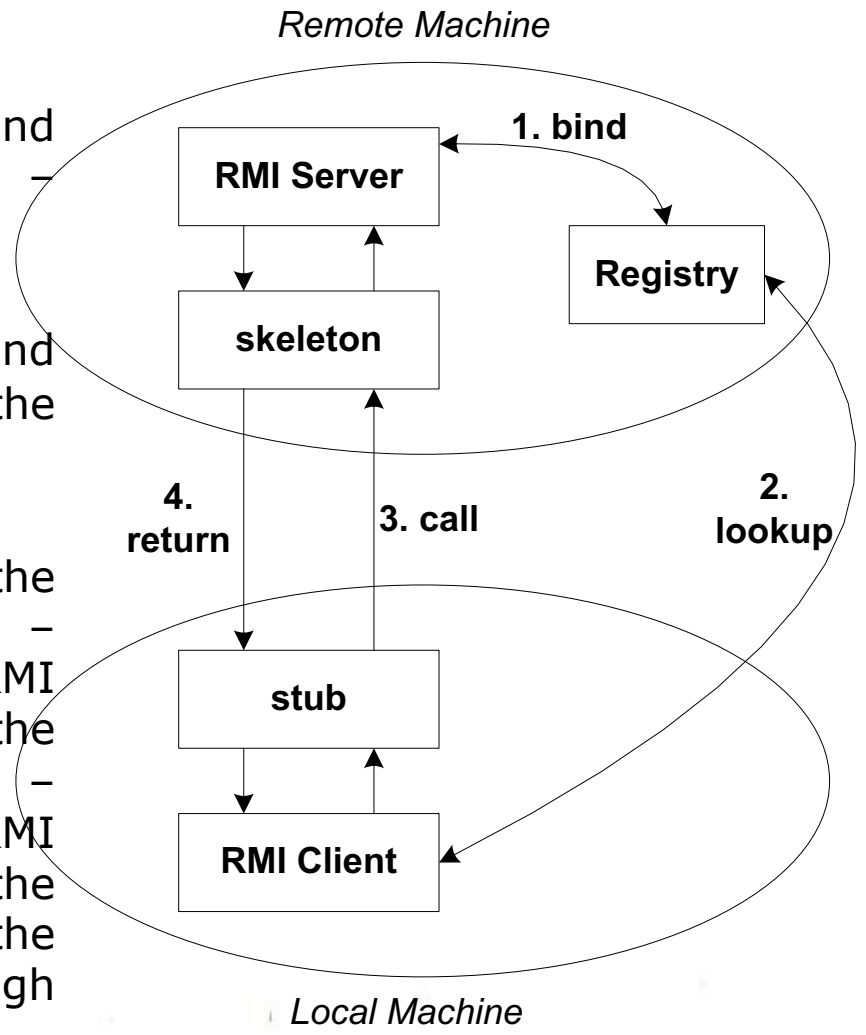
ObjectWeb defines middleware as: "The software layer that lies between the operating system and applications on each side of a distributed computing system in a network."[4]

Services that can be regarded as middleware include: enterprise application integration, data integration,
Remote Procedure Call (RPC)
Message Oriented Middleware (MOM),
Object Request Brokers (ORBs),
Service Oriented Architecture (SOA),
and the Enterprise Service Bus (ESB).

Database access services are often characterized as middleware. Some of them are language specific implementations and support heterogeneous features and other related communication features. Examples of database-oriented middleware include ODBC, JDBC and transaction processing monitors.

RPC/RMI Architecture

- RMI Server must register its name and address in the RMI Registry program – **bind**
- RMI Client is looking for the address and the name of the RMI server object in the RMI Registry program – **lookup**
- RMI Stub serializes and transmits the parameters of the call in sequence – ***"marshalling"*** to the RMI Skeleton. RMI Skeleton de-serializes and extracts the parameters from the received call – ***"unmarshalling".*** In the end, the RMI Skeleton calls the method inside the server object and send back the response to the RMI Stub through "marshalling" the return's parameter.

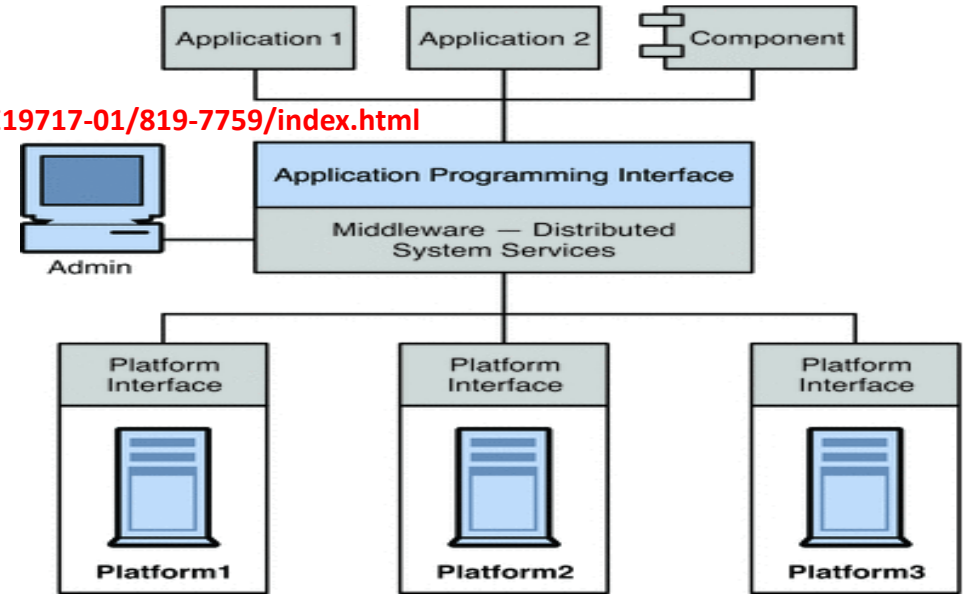


Middleware Technologies:

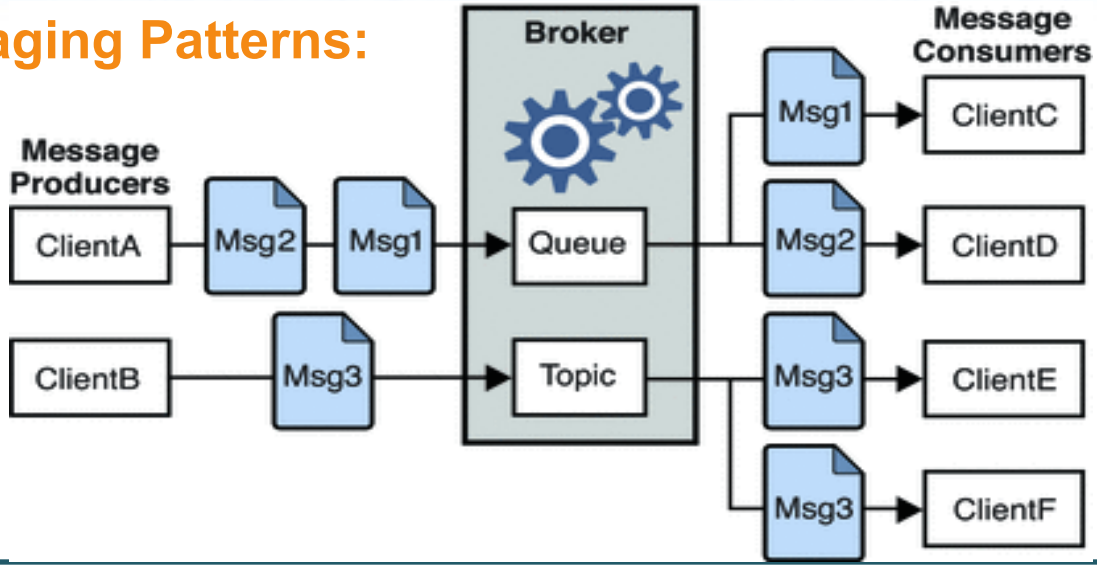
Middleware can be grouped into the following categories:

- ***Remote Procedure Call or RPC***-based middleware, which allows procedures in one application to call procedures in remote applications as if they were local calls. The middleware implements a linking mechanism that locates remote procedures and makes these transparently available to a caller. Traditionally, this type of middleware handled procedure-based programs; it now also includes object-based components.
- ***Message Oriented Middleware or MOM***-based middleware, which allows distributed applications to communicate and exchange data by sending and receiving messages asynchronously.
- ***Object Request Broker or ORB***-based middleware, which enables an application's objects to be distributed and shared across heterogeneous networks.

http://docs.oracle.com/cd/E19717-01/819-7759/index.html



JMS Messaging Patterns:



Clients A and B are message producers, sending messages to clients C, D, and E by way of two different kinds of destinations.

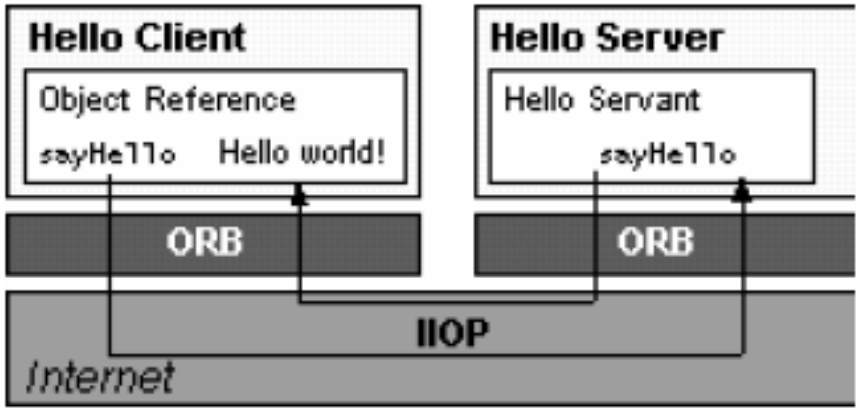
- Messaging between clients A, C, and D illustrates the point-to-point pattern. Using this pattern, a client sends a message to a queue destination from which only one receiver may get it. No other receiver accessing that destination can get that message.
- Messaging between clients B, E, and F illustrates the publish/subscribe pattern. Using this broadcast pattern, a client sends a message to a topic destination from which any number of consuming subscribers can retrieve it. Each subscriber gets its own copy of the message.

Message consumers in either domain can choose to get messages synchronously or asynchronously. Synchronous consumers make an explicit call to retrieve a message; asynchronous consumers specify a callback method that is invoked to pass a pending message. Consumers can also filter out messages by specifying selection criteria for incoming messages.

http://docs.oracle.com/cd/E19717-01/819-7759/index.html

1.3 Distributed Systems: CORBA

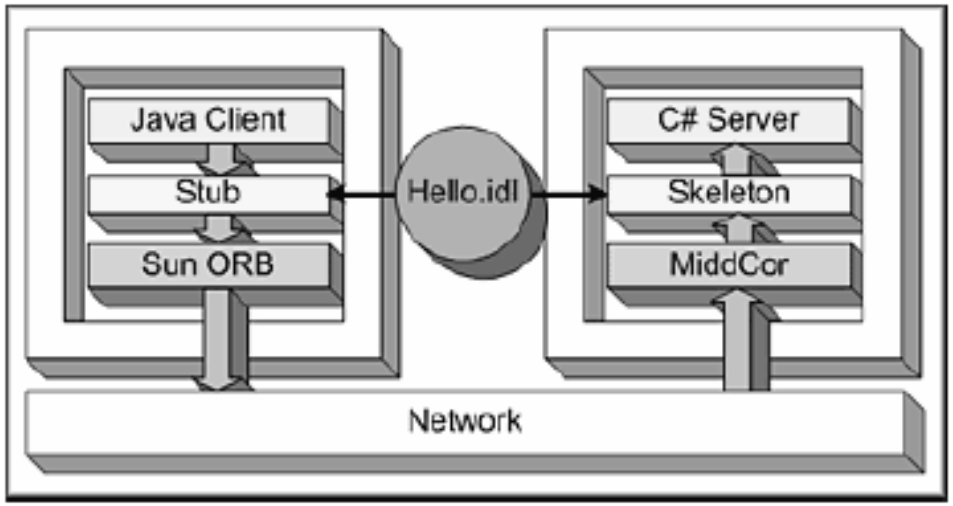
CORBA Architecture



From: http://java.sun.com/docs/books/tutorial/idl/hello/index.html

Hello.idl

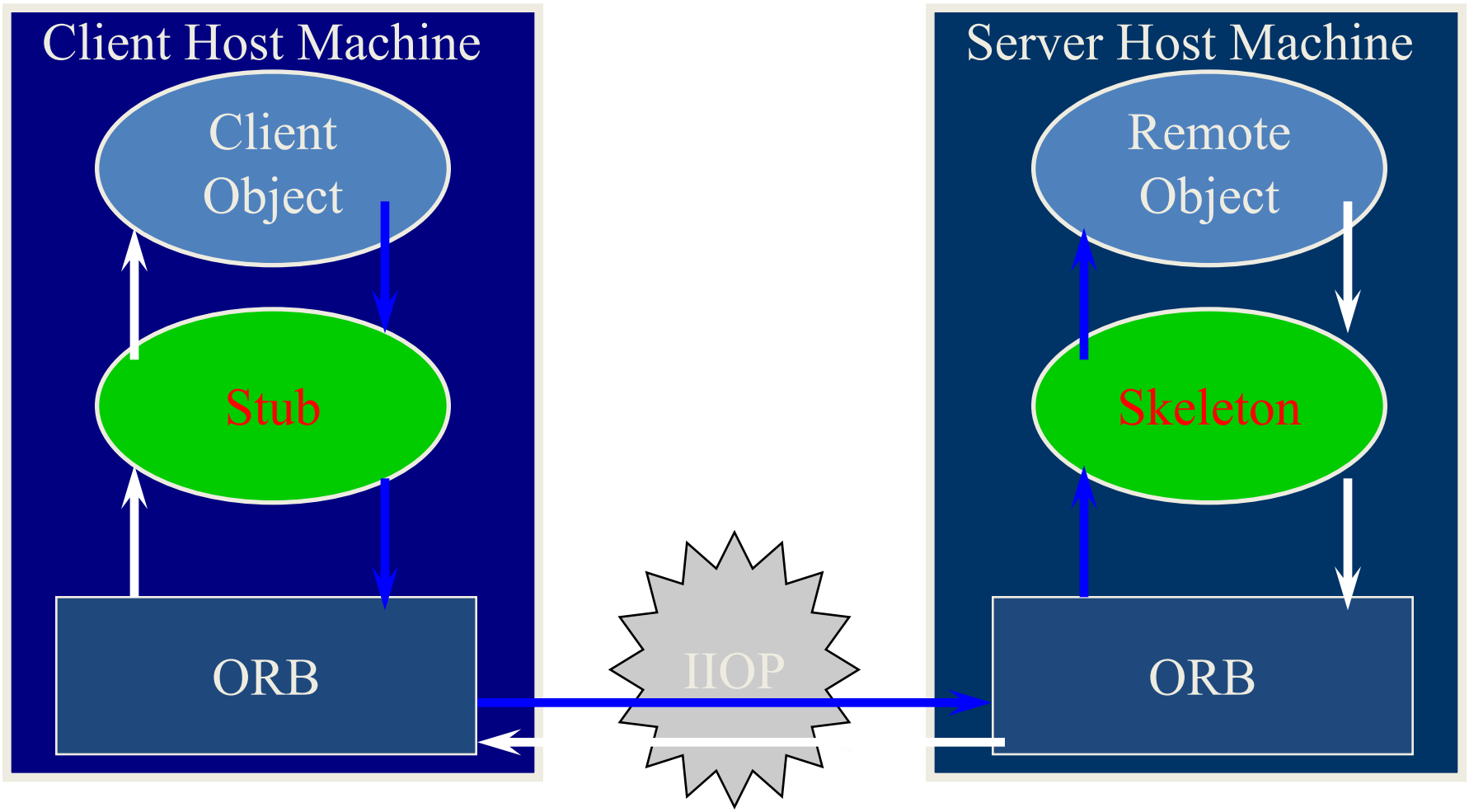
```
interface Hello {
    string sayHello();
};
```



From: http://www.molecular.com/news/recent_articles/051904_javaworld.aspx

1.3 Distributed Systems: CORBA

CORBA Architecture is replaced by gRPC?

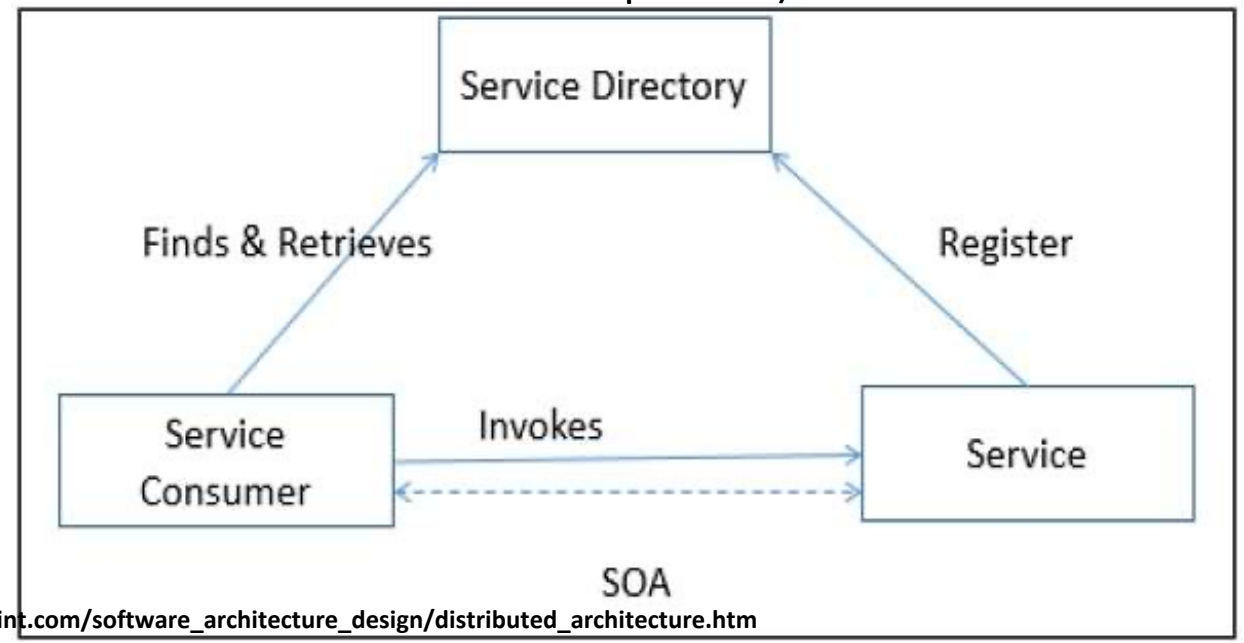


1.3 Distributed Systems: SOA

Service Oriented Architecture – SOA

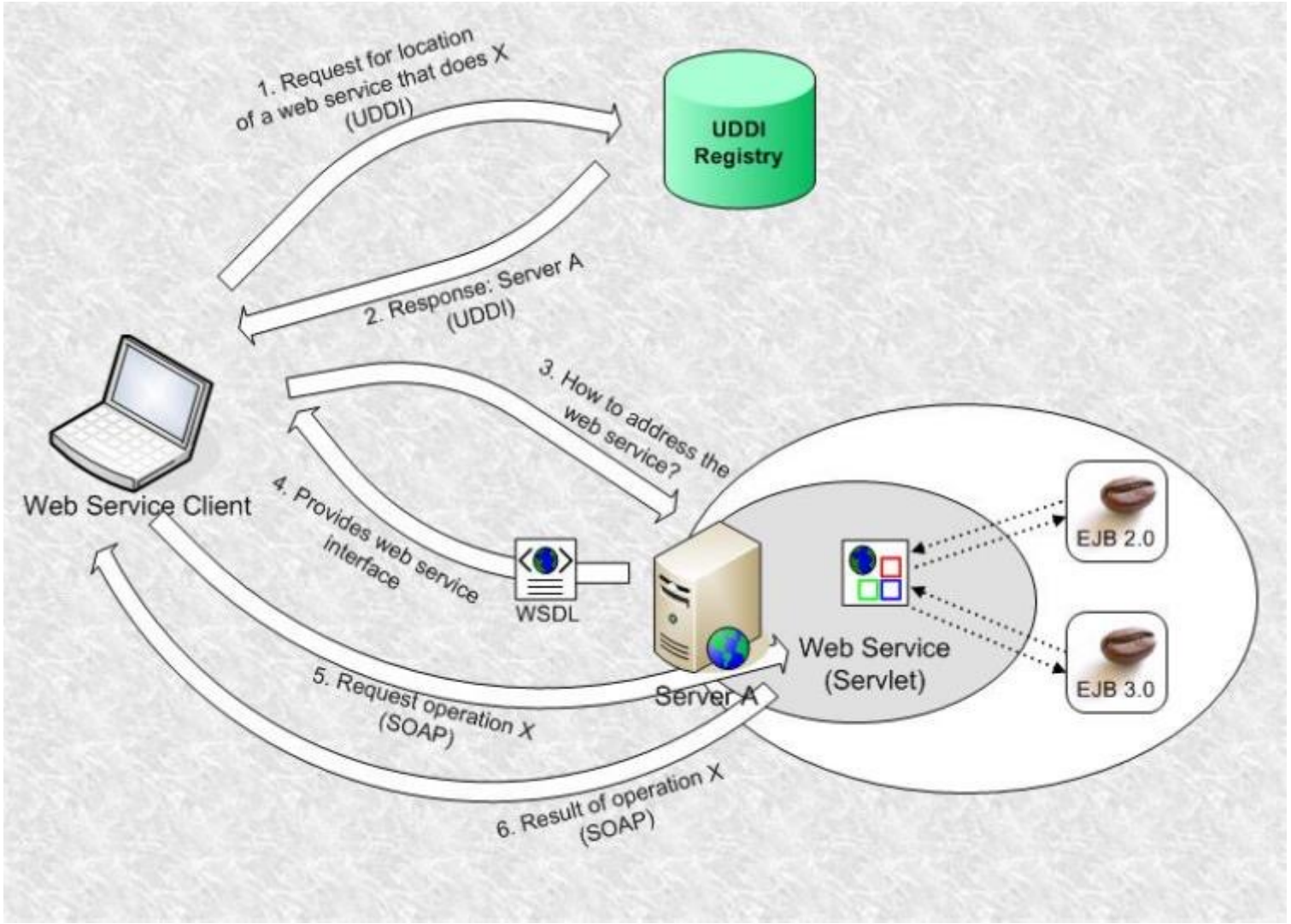
A service is a component of business functionality that is well-defined, self-contained, independent, published, and available to be used via a standard programming interface. The connections between services are conducted by common and universal message-oriented protocols such as the SOAP Web service protocol, which can deliver requests and responses between services loosely.

Service-oriented architecture is a client/server design which support business-driven IT approach in which an application consists of software services and software service consumers (also known as clients or service requesters).



1.3 Distributed Systems: SOA

Service Oriented Architecture – SOA



Service Oriented Architecture – SOA

Features of SOA

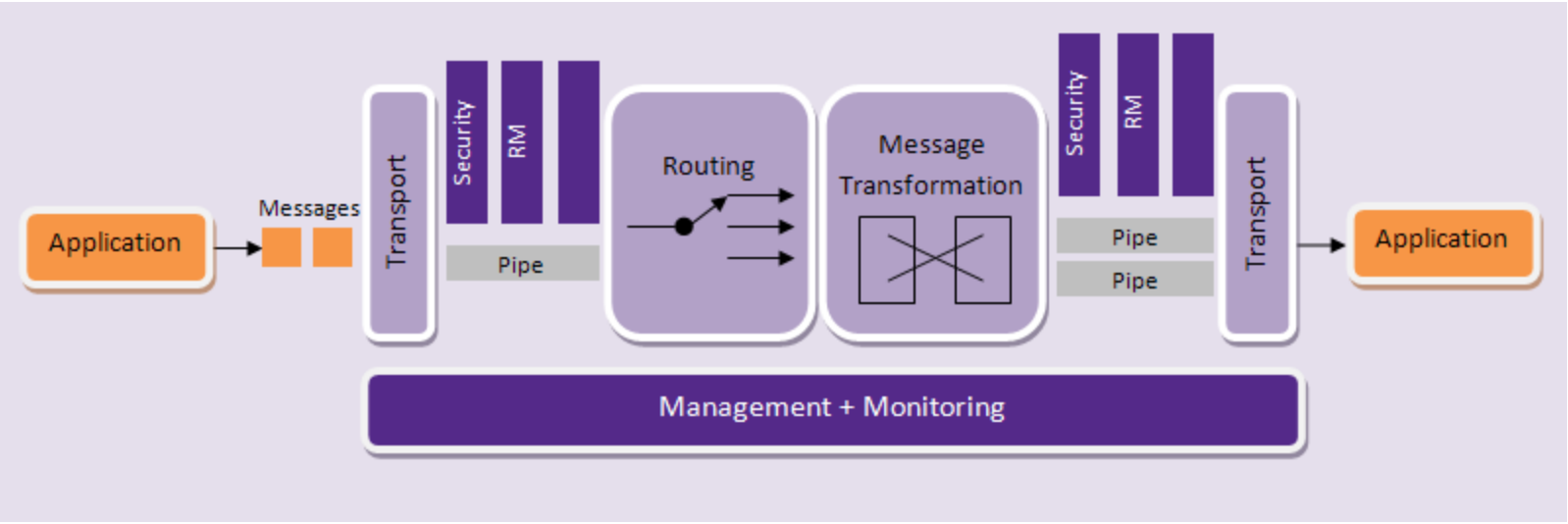
A service-oriented architecture provides the following features −

- Distributed Deployment − Expose enterprise data and business logic as loosely, coupled, discoverable, structured, standard-based, coarse-grained, stateless units of functionality called services.
- Composability − Assemble new processes from existing services that are exposed at a desired granularity through well defined, published, and standard complaint interfaces.
- Interoperability − Share capabilities and reuse shared services across a network irrespective of underlying protocols or implementation technology.
- Reusability − Choose a service provider and access to existing resources exposed as services.

1.3 Distributed Systems: ESB

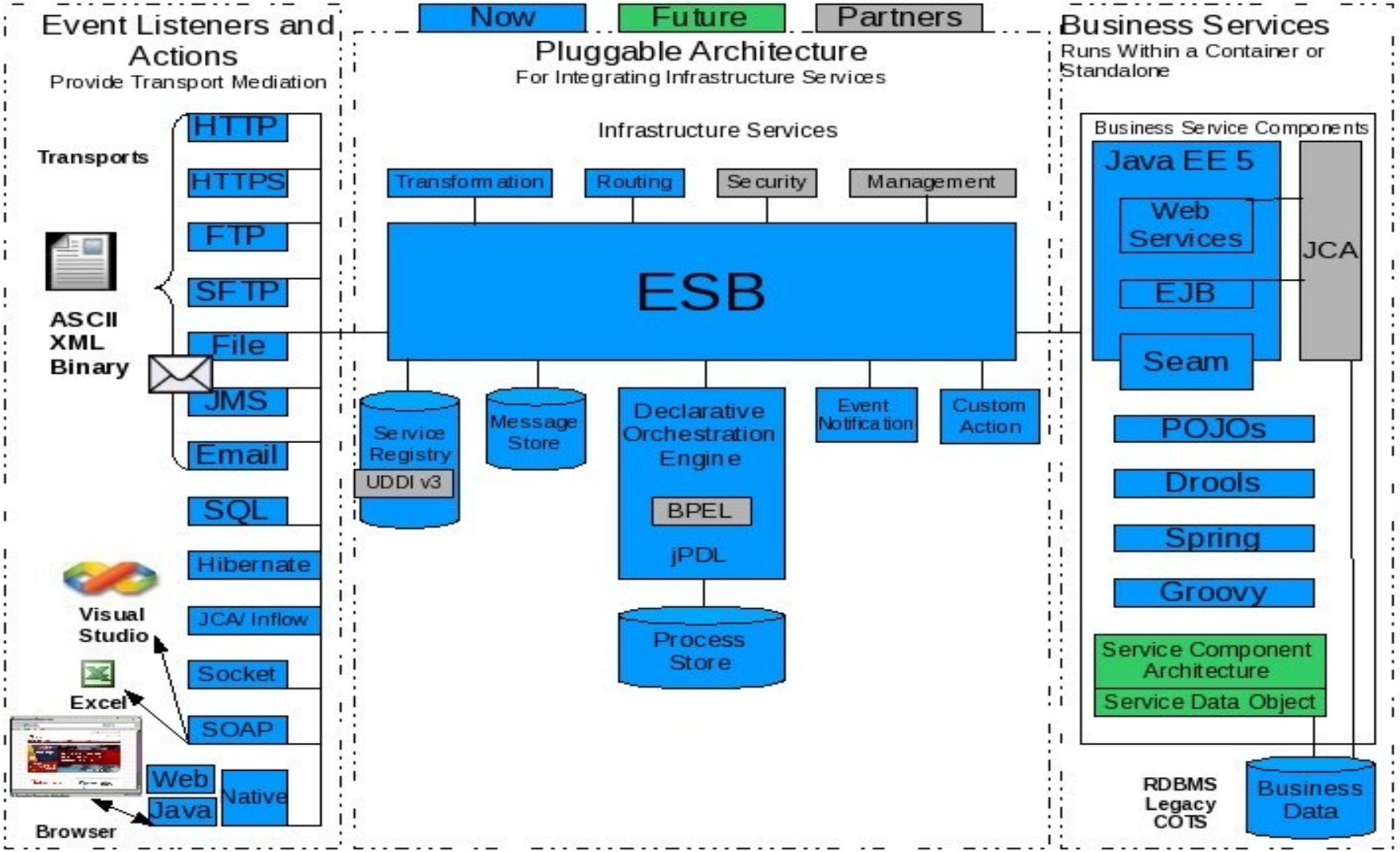
Enterprise Service Bus Architecture – ESB

An **enterprise service bus** (**ESB**) implements a communication system between mutually interacting software applications in a service-oriented architecture (SOA). As it implements a distributed computing architecture, it implements a special variant of the more general client-server model, wherein, in general, any application using ESB can behave as server or client in turns. ESB promotes agility and flexibility with regard to high-level protocol communication between applications. The primary goal of the high-level protocol communication is enterprise application integration (EAI) of heterogeneous and complex service or application landscapes (a view from the network level).



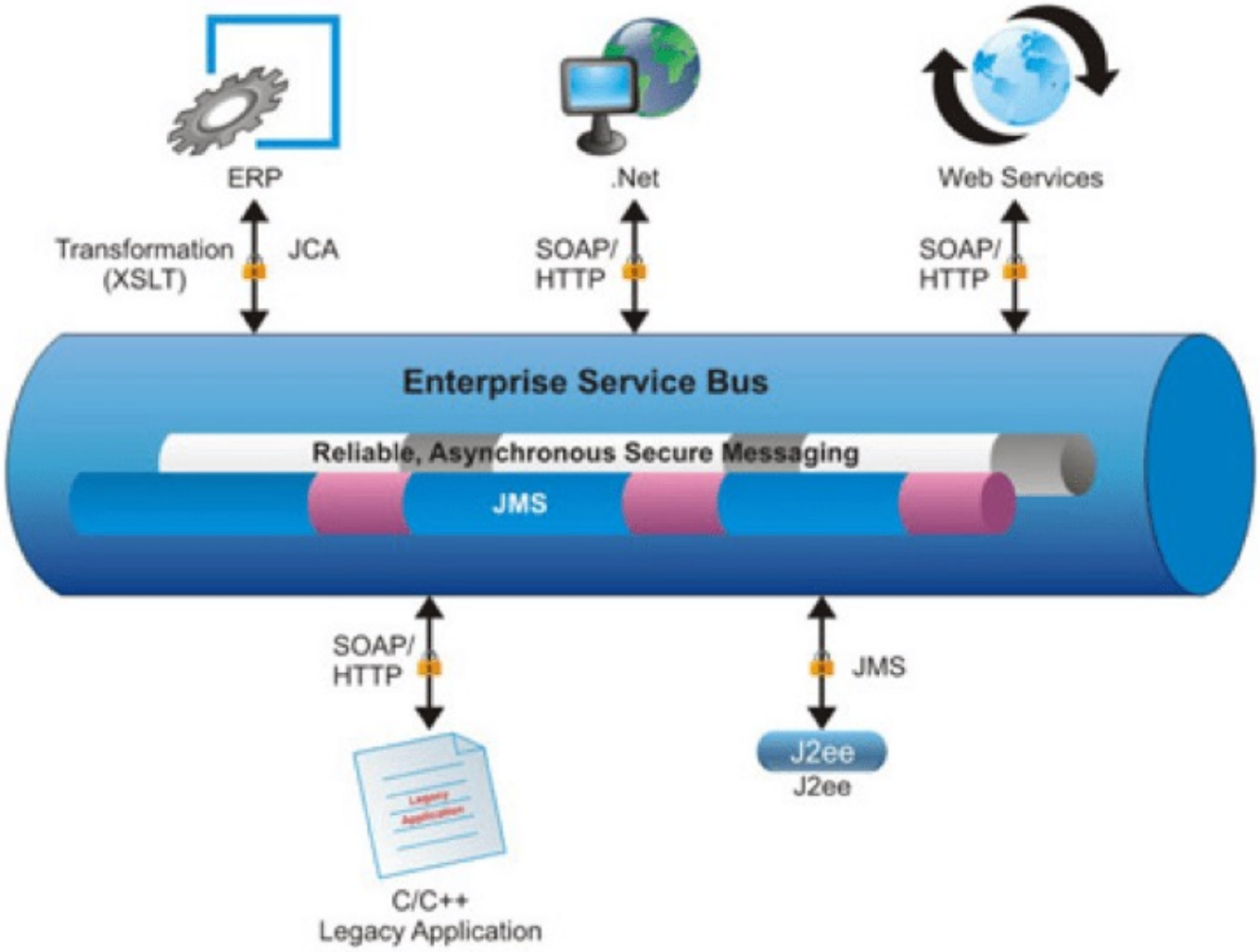
1.3 Distributed Systems: ESB

Enterprise Service Bus Architecture – ESB



1.3 Distributed Systems: ESB

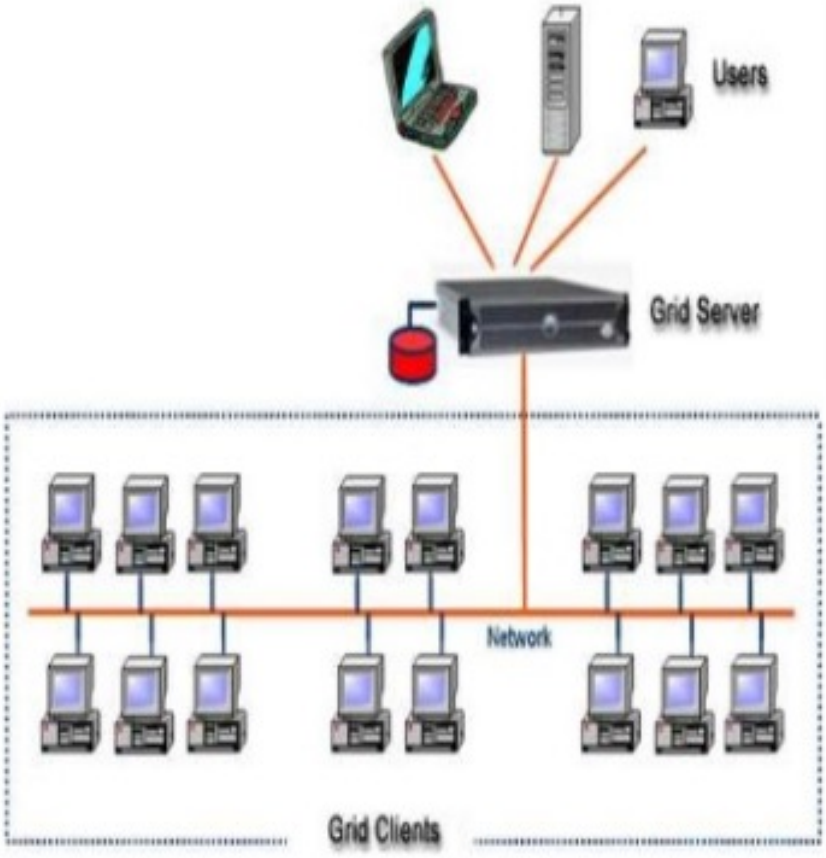
Enterprise Service Bus Architecture – ESB



1.3 Distributed Systems: GRID

Grid Computing

How Grid computing works ?

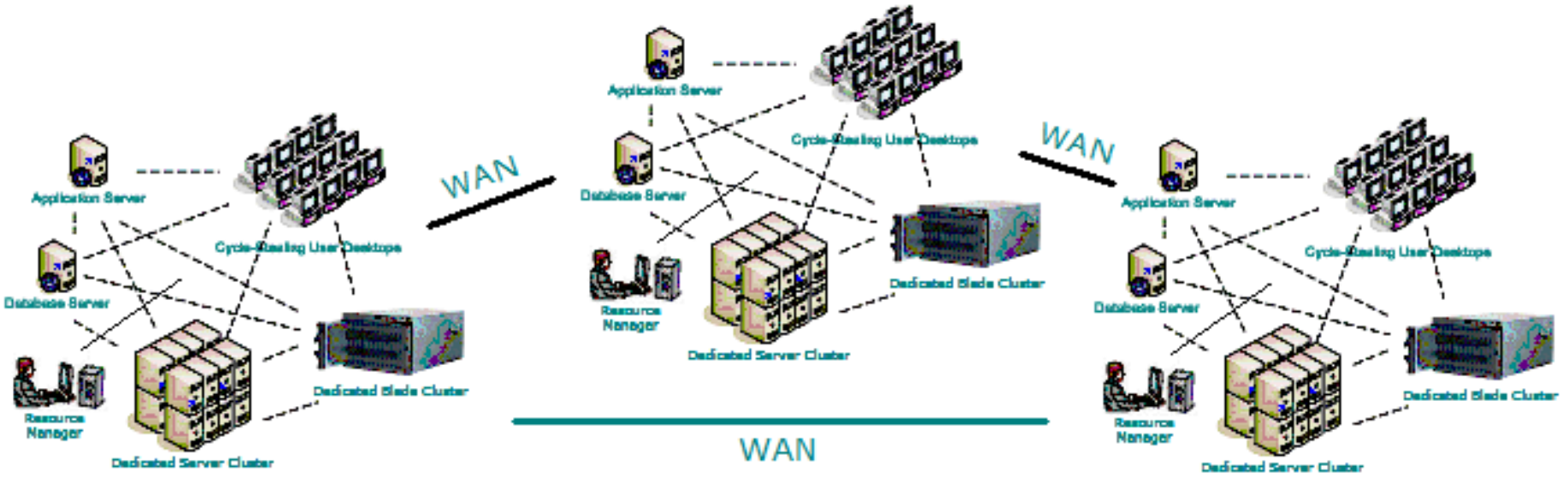


In general, a grid computing system requires:

- **At least one computer, usually a server, which handles all the administrative duties for the System**
- **A network of computers running special grid computing network software.**
- **A collection of computer software called middleware**

1.3 Distributed Systems: GRID

Grid Computing



1.4 Distributed Systems Challenges

1. Heterogeneity – variety and difference in:
- networks
- computer hardware
- OS
- programming languages
- implementations by different developers

2. OPENNESS

- computer system - can the system be extended and re-implemented in various ways?
- distributed system - can new resource-sharing services be added and made available for use by variety of client programs?

3. Security
- Confidentiality
- Integrity
- Availability

Main goal: High availability

4. Scalability
–the ability of the system system to work well when the system load or the number of users increases

Challenges with building scalable distributed systems:
- Controlling the cost of physical resources
- Controlling the performance loss
- Preventing software resources running out (like 32-bit internet addresses, which are being replaced by 128 bits)
- Avoiding performance bottlenecks

5. Failure handling
Techniques for dealing with failures
- Detecting failures
- Masking failures

- messages can be retransmitted
- disks can be replicated in a synchronous action
- Tolerating failures
- Recovery from failures
- Redundancy – redundant components

- at least two different routes
- database can be replicated in several servers

1.4 Distributed Systems Challenges

6. Concurrency – Several clients trying to access shared resource at the same time.

Any object with shared resources in a DS must be responsible that it operates correctly in a concurrent environment.

7. Transparency

Transparency – concealment from the user and the applications programmer of the separation of components in a Distributed System for the system to be perceived as a whole rather than a collection of independent components:

- Access transparency – access to local and remote re- sources identical
- Location transparency – resources accessed without knowing their physical or network location
- Concurrency transparency – concurrent operation of processes using shared resources without interference be- tween them
- Replication transparency – multiple instances seem like one
- Failure transparency – fault concealment
- Mobility transparency – movement of resources/clients within a system without affecting the operation of users or programs

8. Quality of service

Main nonfunctional properties of systems that affect *Quality of Service (QoS):*

- reliability
- security
- performance

*** Distributed Systems**

- ***Data Migration***
- ***Calculus / Business Logic Migration***
- Process Migration !?

Distributed Systems Challenges

In Distributed Systems & Cloud the core things are about:

- ***Data Migration***
- ***Calculus / Business Logic Migration***
- Process Migration !?

Cloud, Containers & Micro-services are parts of any Distributed Computing Systems

Section Conclusion

Fact: **DAD needs Java and ECMAScript/JS/node.js**

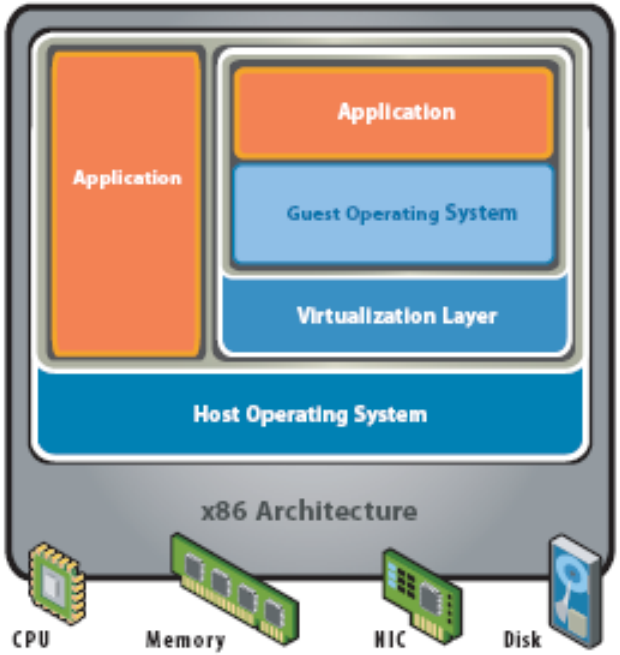
In few **samples** it is simple to remember: Types of the distributed architectural patterns of the distributed systems.

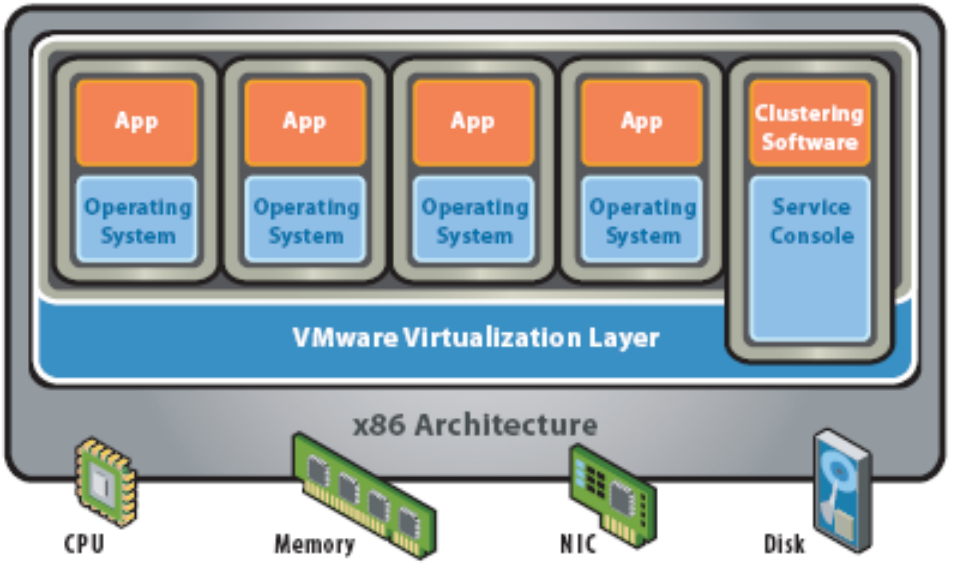
Computing Clouds: IaaS, “Native Clouds”: PaaS, CaaS, FaaS | (SaaS), Dev-Ops & INFRA: Jenkins - Bamboo

Computing Cloud, Dev-Ops & INFRA

Distributed Systems: Cloud Architecture

Cloud Concepts – Intro – Virtualization Overview





Hosted Architecture

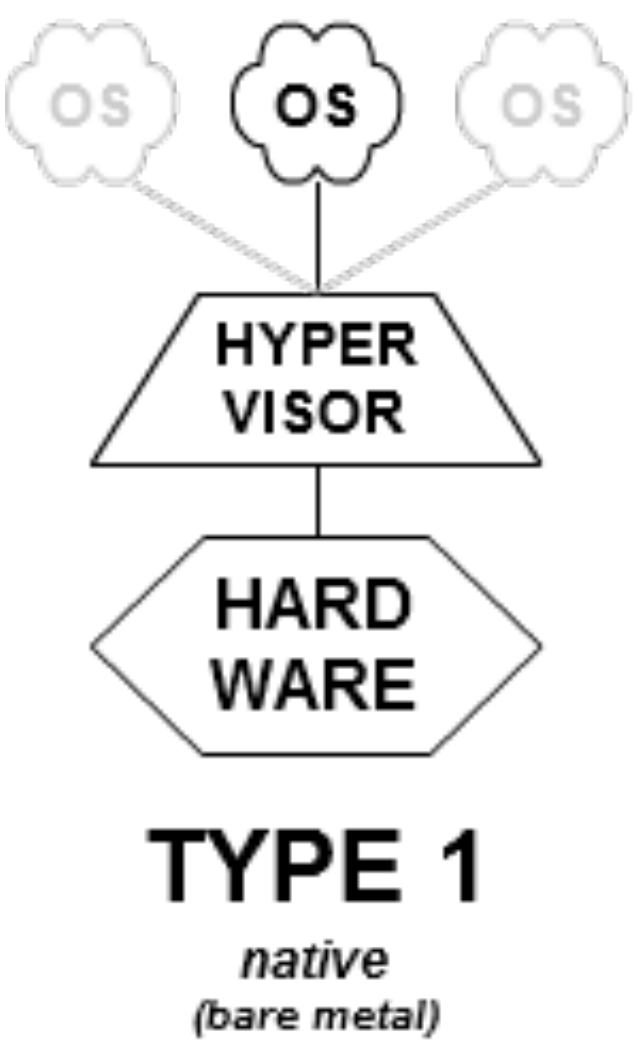
- Installs and runs as an application
- Relies on host OS for device support and physical resource management

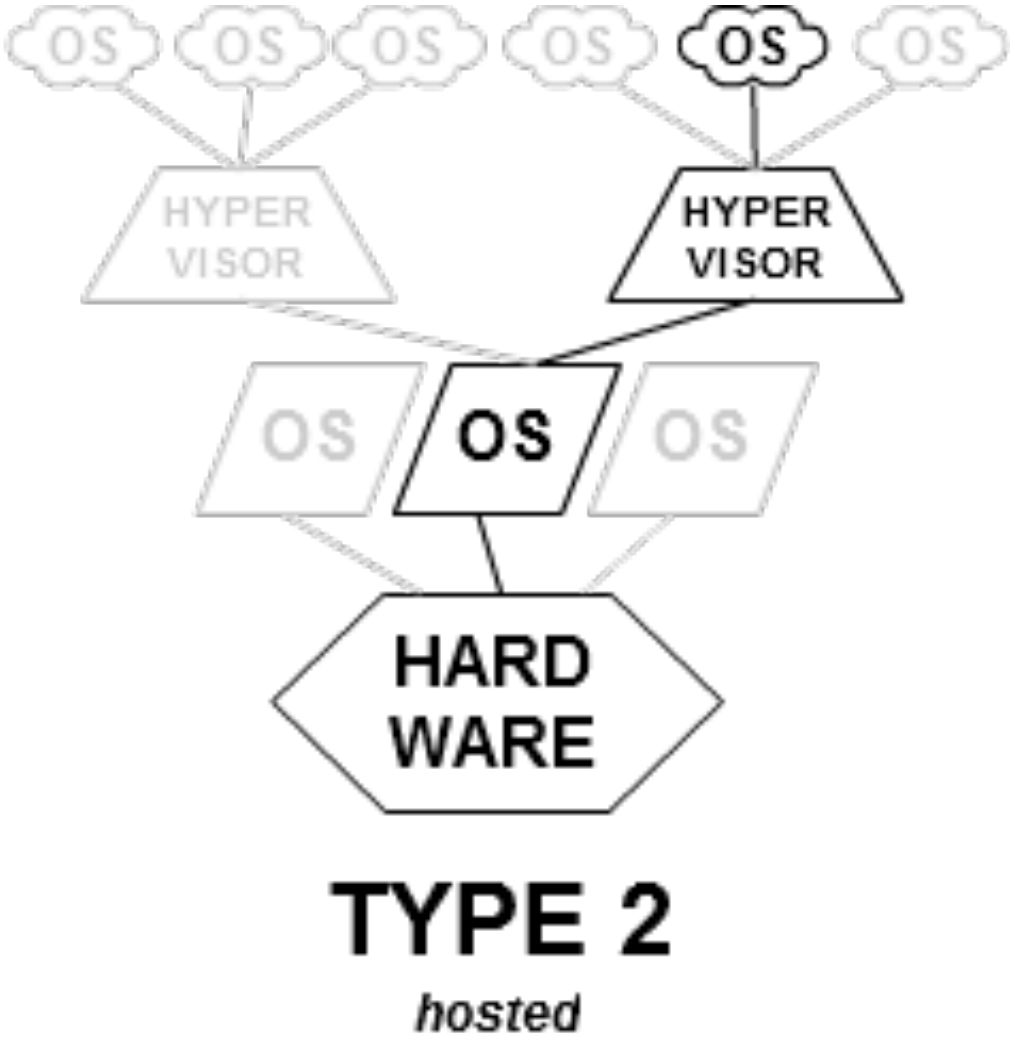
Bare-Metal (Hypervisor) Architecture

- Lean virtualization-centric kernel
- Service Console for agents and helper applications

Figure 2: Virtualization Architectures

Cloud Concepts – Intro – Virtualization Overview

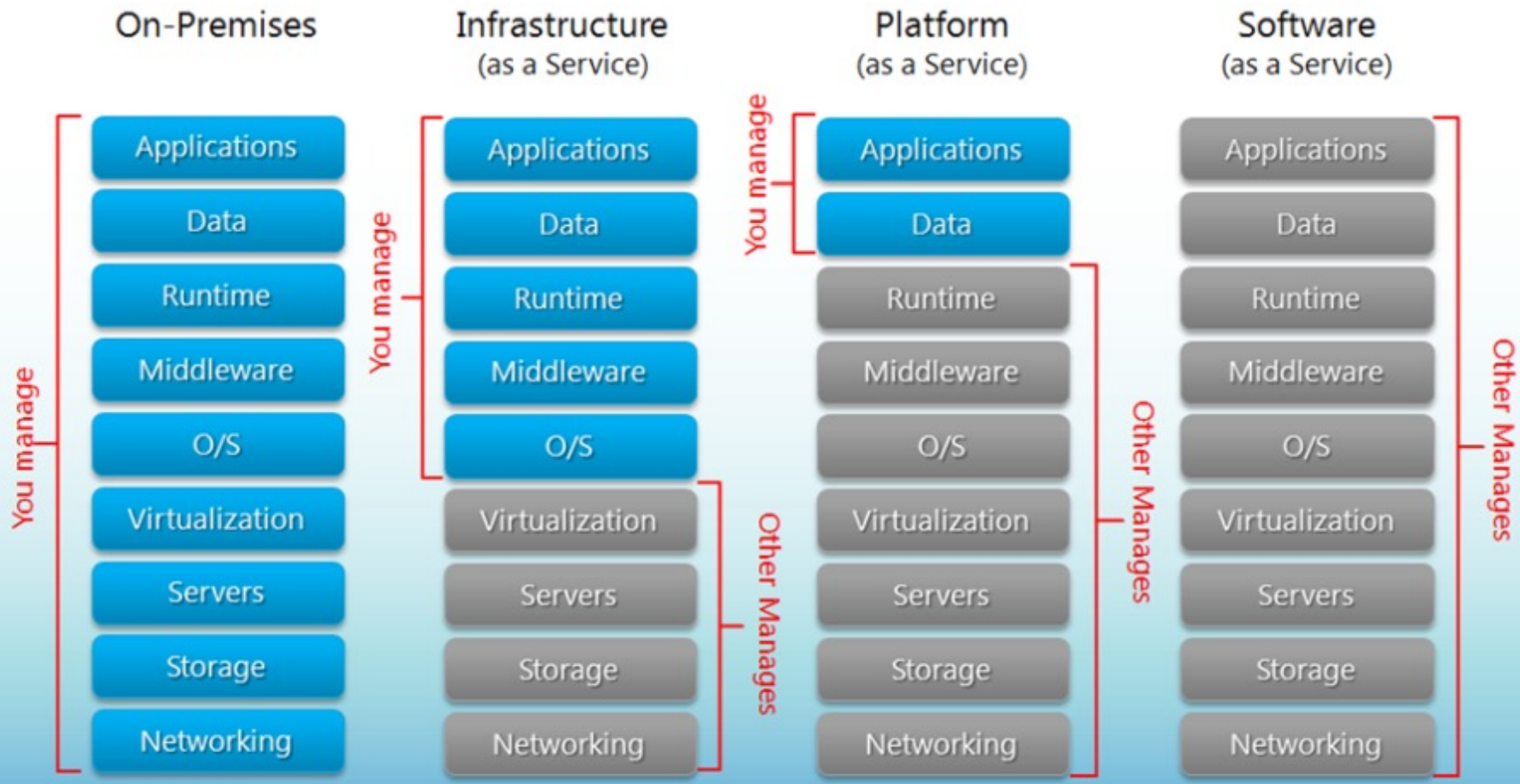




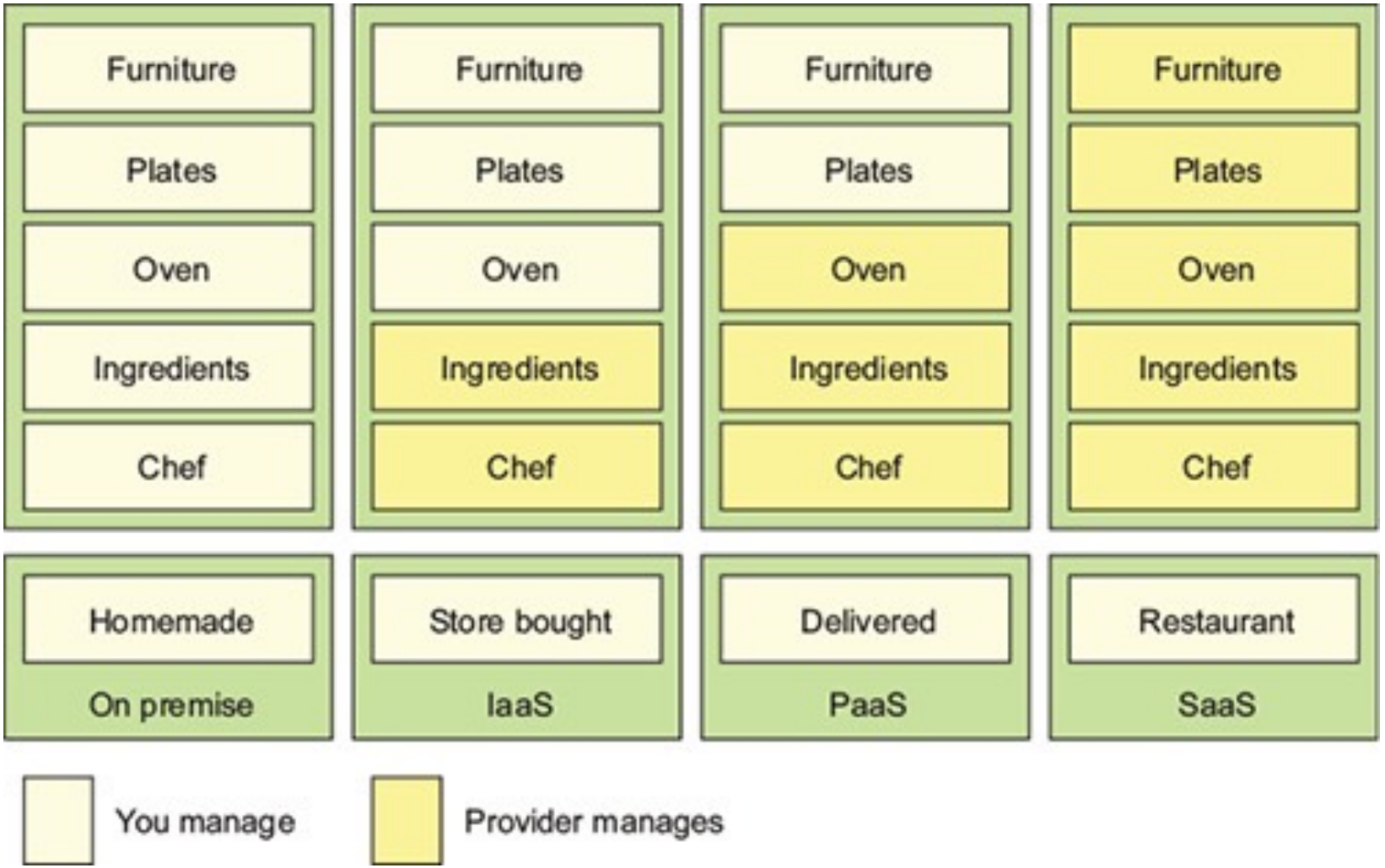
Distributed Systems: Cloud

Separation of Responsibilities

Cloud Concepts – Intro – IaaS, PaaS, SaaS

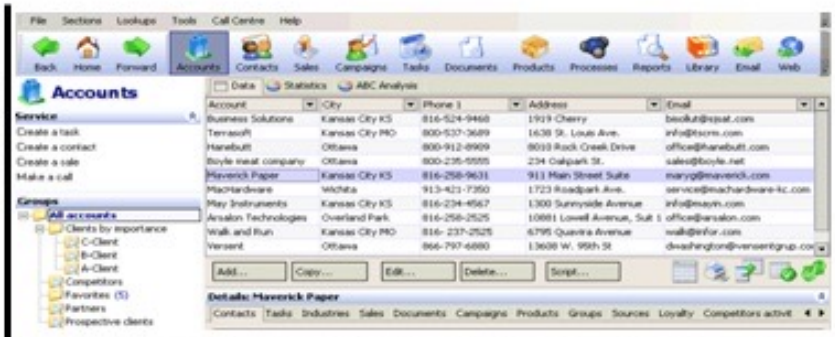


Distributed Systems: Cloud Analogy with Pizza



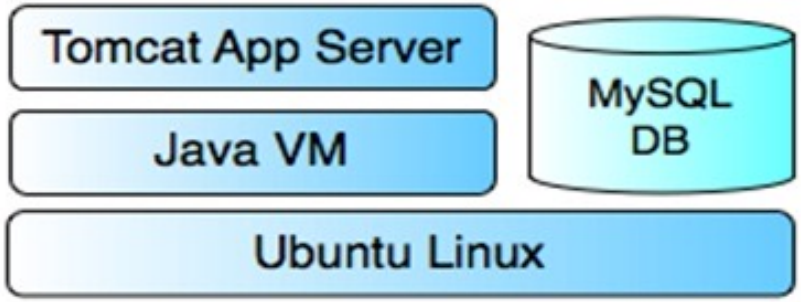
Cloud Concepts – Intro – IaaS, PaaS, SaaS

SaaS



SalesForce.com, Google Apps

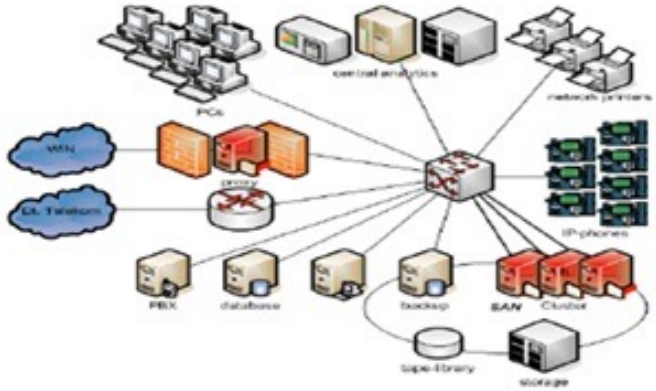
PaaS



Google App Engine for: Java, Ruby, Python & GO

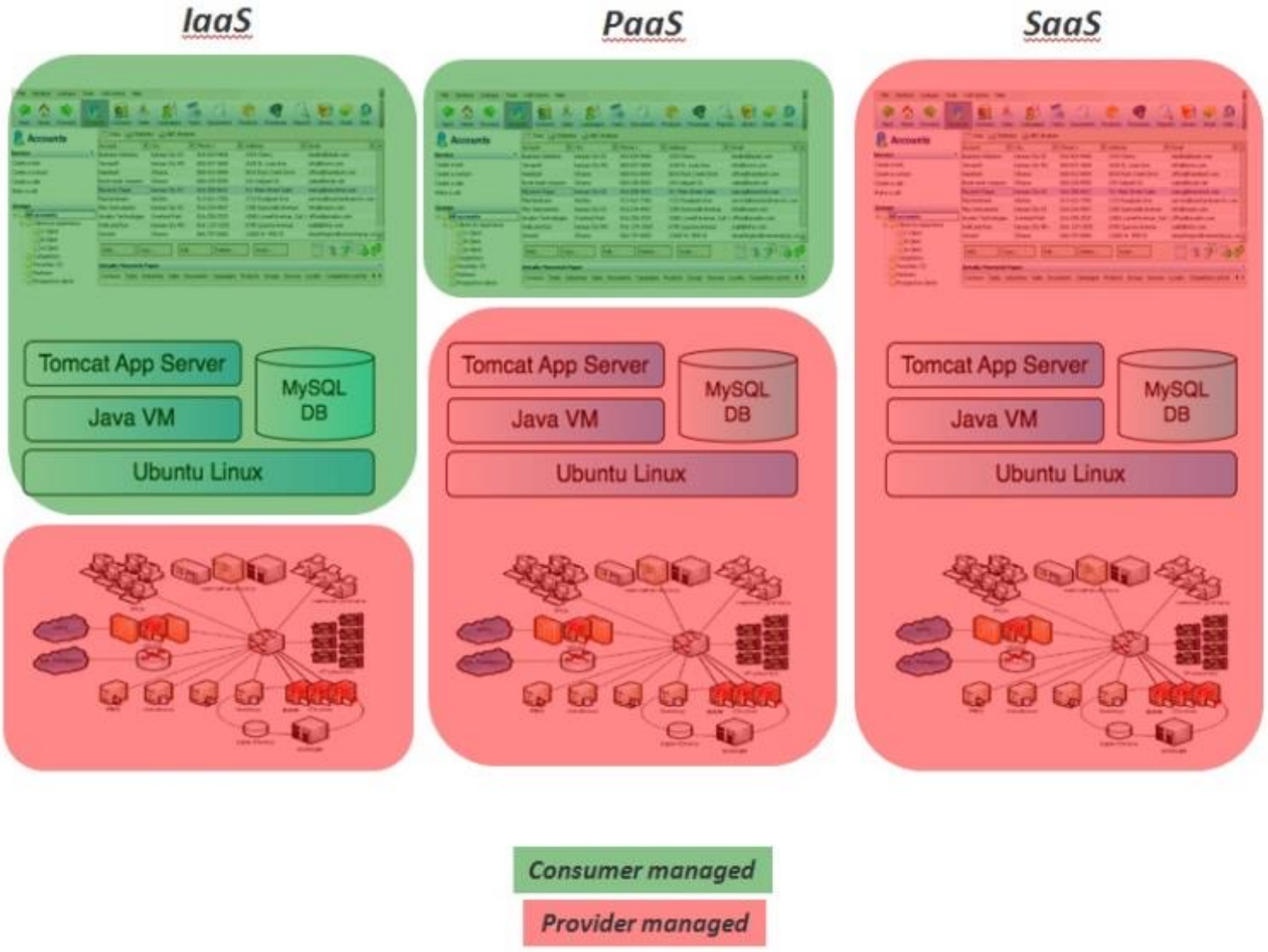
VMForce.com, MS Azure

IaaS

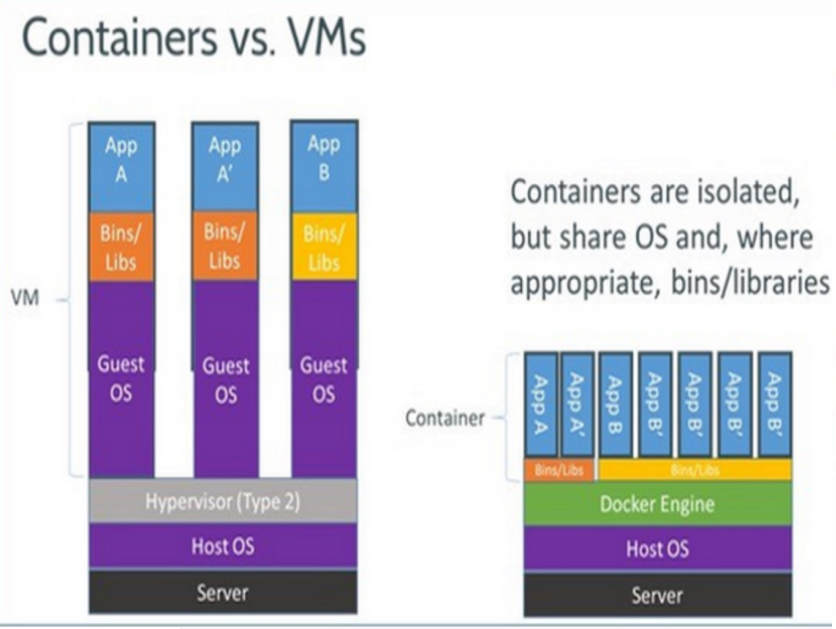


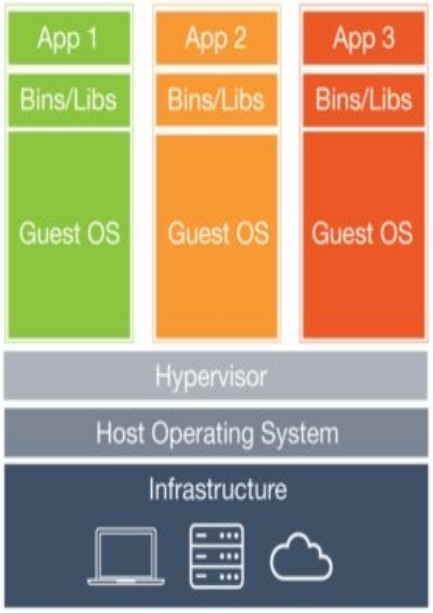
vCloud Express/Datacenter, Amazon EC2

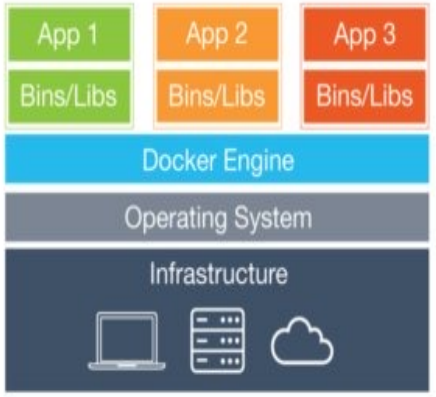
Cloud Concepts – Intro – IaaS, PaaS, SaaS

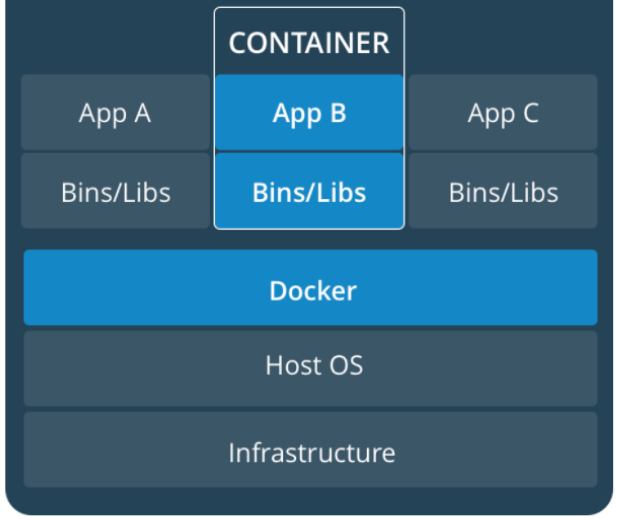


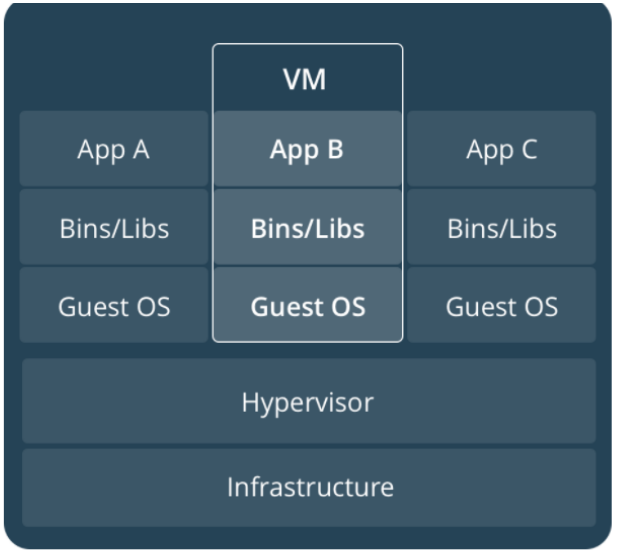
Deployment: Std. OS vs. VMs vs. Docker Containers



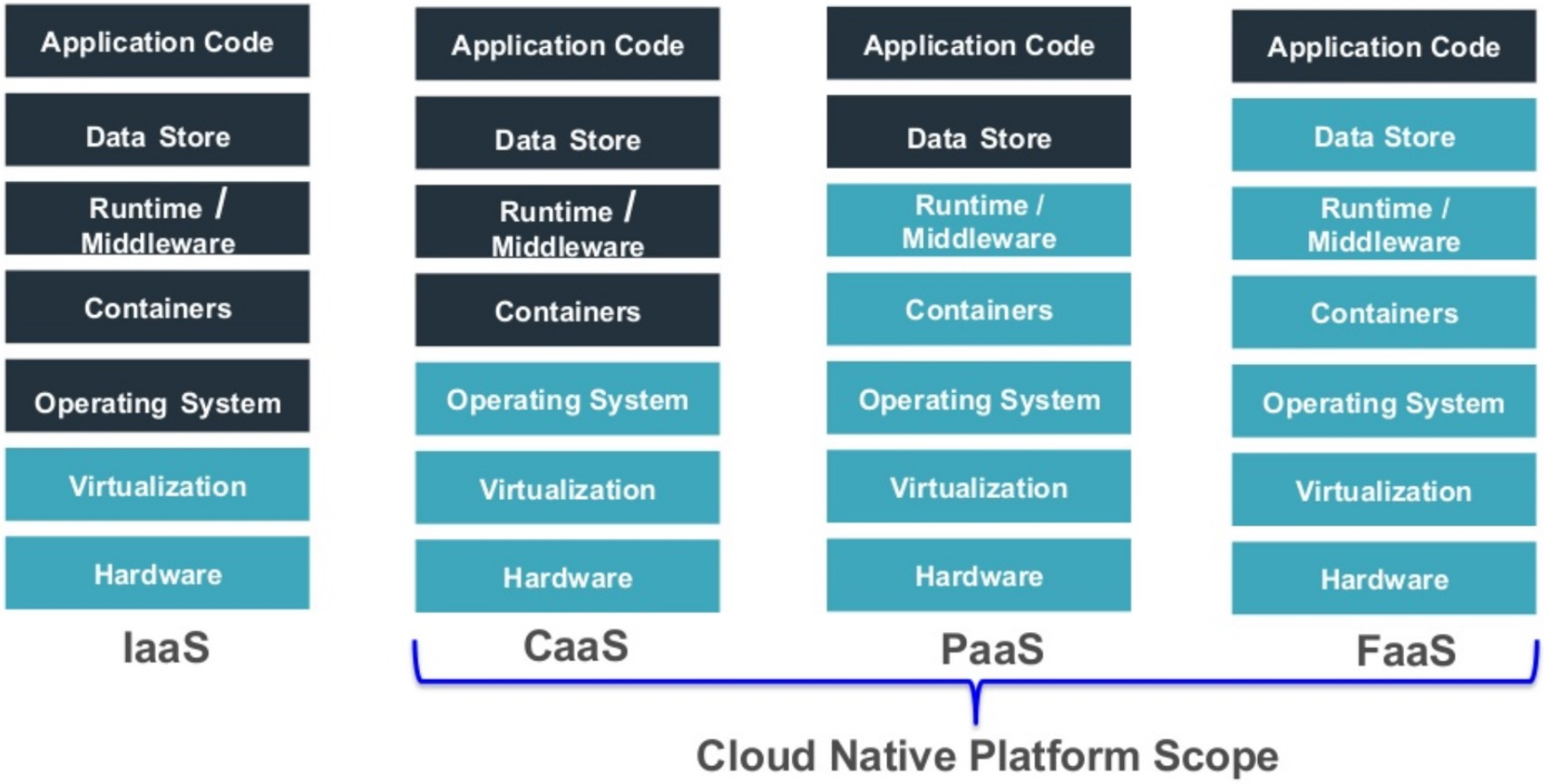






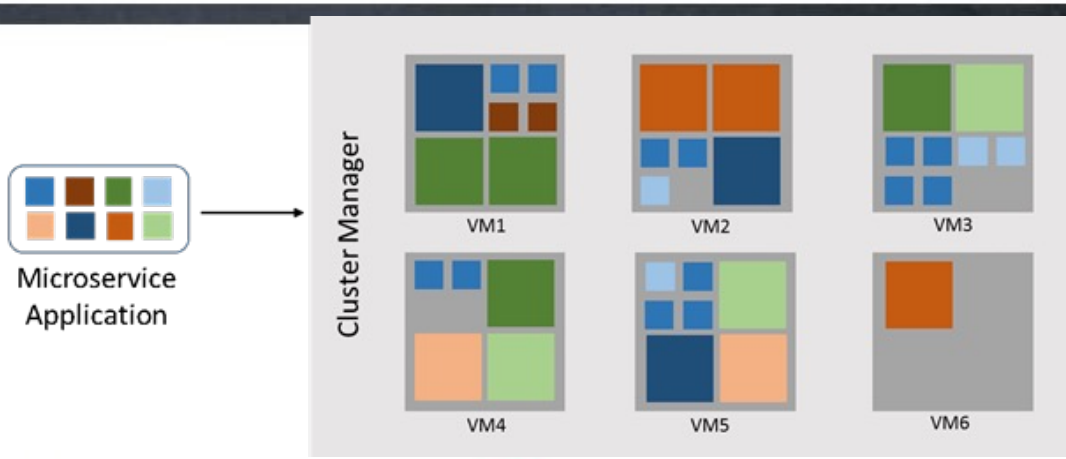


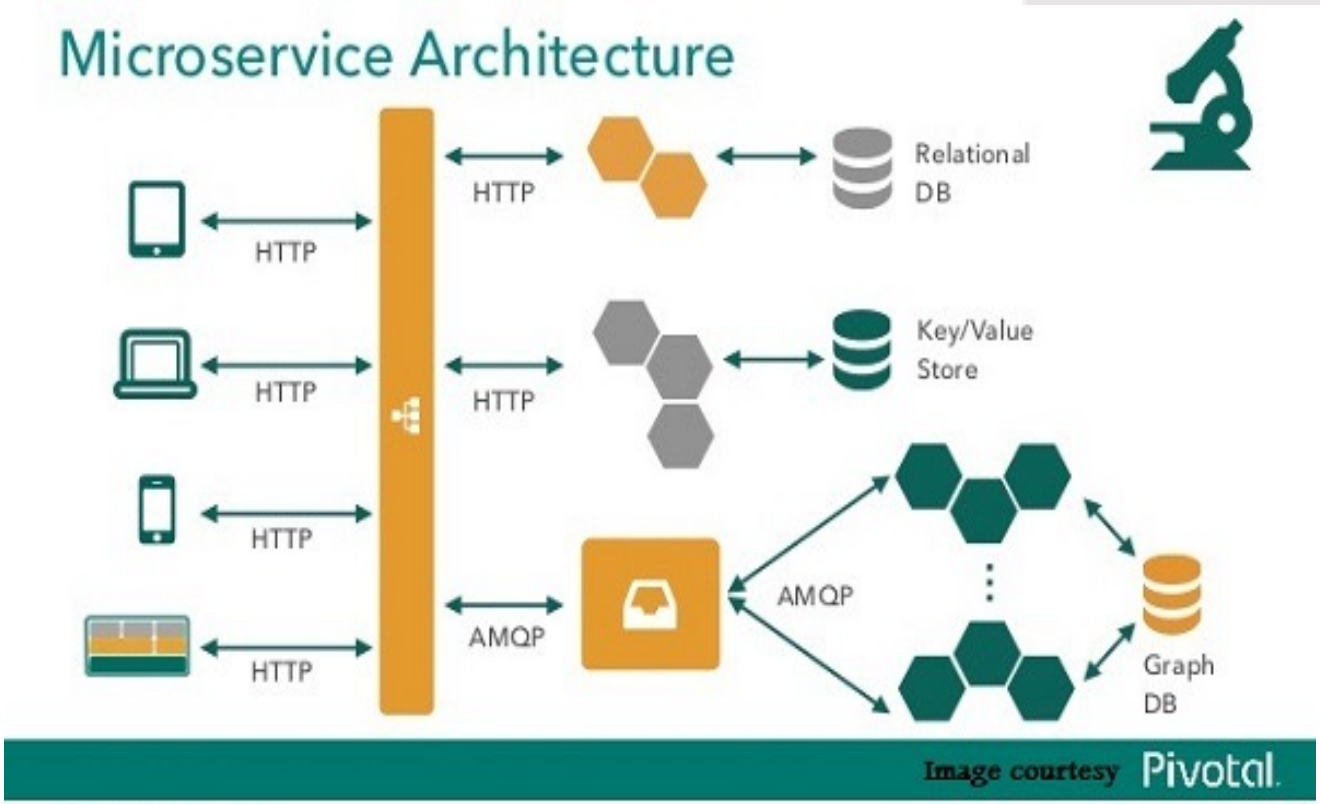
Cloud Concepts – Intro – IaaS, PaaS, CaaS, SaaS



Distributed Systems: Micro-services

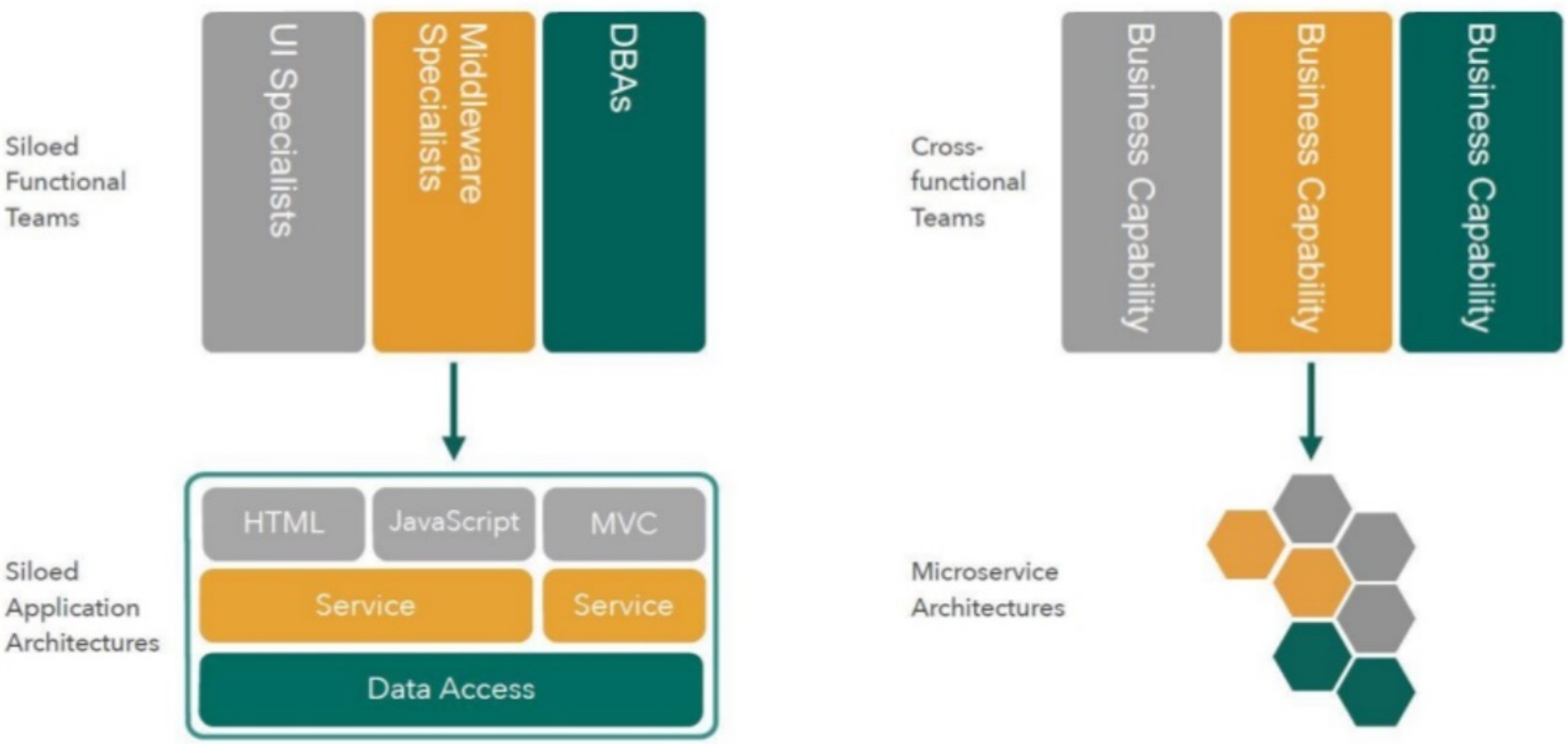
Micro-services Architecture





Distributed Systems

Micro-services Architecture



Computing Clouds
for easy sharing

Section Conclusions

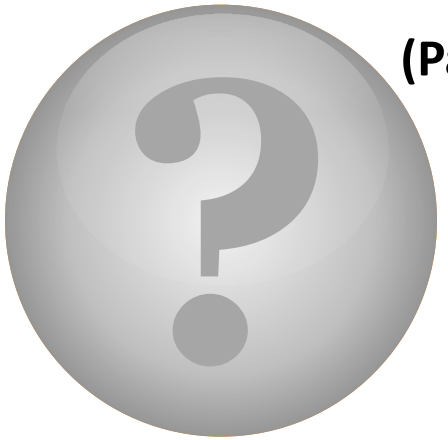
Computing Clouds

Share knowledge, Empowering Minds

Communicate & Exchange Ideas

Some “myths”:

(Distributed Systems).Equals(Distributed Computing) == true?



(Parallel System).Equals(Parallel Computing) == true?

(Parallel System == Distributed System) != true?

(Sequential vs. Parallel vs. Concurrent vs. Distributed *Programming*) *? (Different) : (Same)*

Questions & Answers!

But wait…

There’s More!

```
if (HTC !=  HPC)
    HTC  (High Throughput Computing) >
    MTC (Many Task  Computing) >
    HPC (High Performance Computing);
```

… Will be continued! - In the next lectures …

Thanks!

HTTP(s) and Java Servlet/JSP

Lecture 4

S3 - Summary of Web/ServerSide Development in JEE

Section 3

presentation

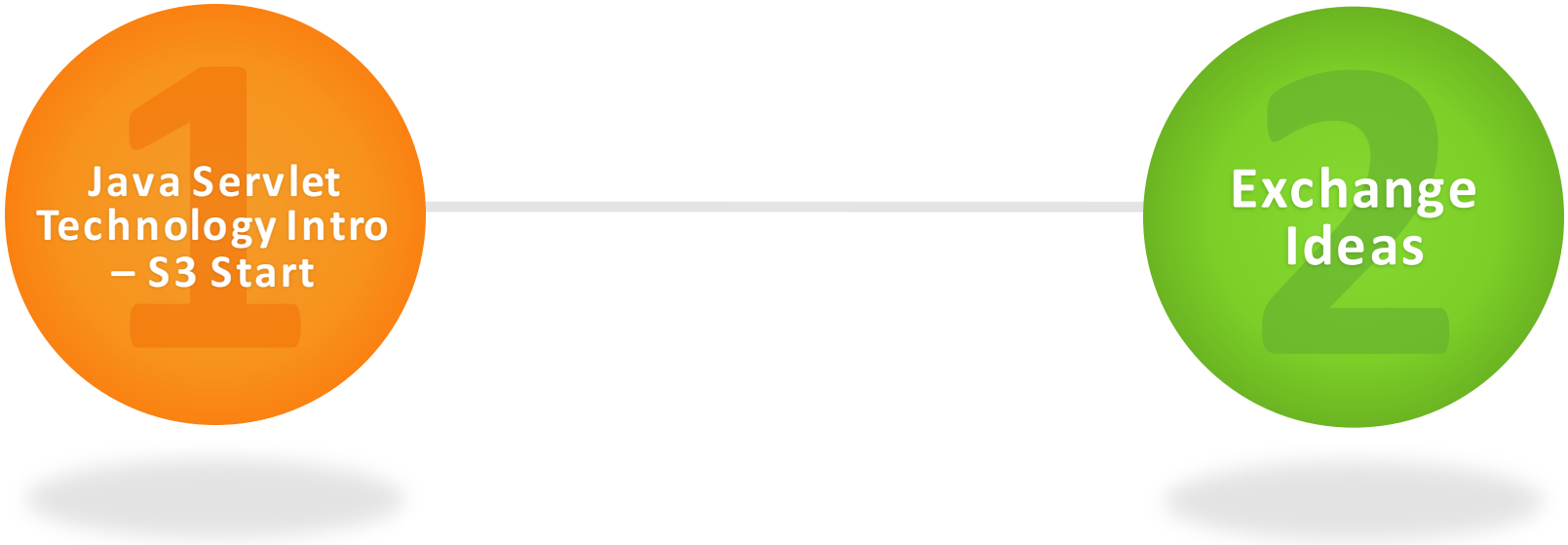
DAD – Distributed Applications Development

Cristian Toma

D.I.C.E/D.E.I.C – Department of Economic Informatics & Cybernetics
www.dice.ase.ro

Cristian Toma – Business Card

Agenda for Lecture 4



DAD Section 3 - Summary of Web Development in JEE, Servlet Lifecycle, Java Servlet Samples

Java Servlet Tech Recapitulation

1. Java Servlet Technology

What is Java Servlet?

Sun: Java Servlet technology provides Web developers with a simple, consistent mechanism for extending the functionality of a Web server and for accessing existing business systems.

WiKi: Servlets are Java programming language objects that dynamically process requests and construct responses. The Java Servlet API allows a software developer to add dynamic content to a Web server using the Java platform. The generated content is commonly HTML, but may be other data such as XML.

The web server for the Java servlet simple tests is Apache Tomcat in Ubuntu Linux and Apache TomEE+ / Tom PluME
http://tomcat.apache.org
http://tomee.apache.org/index.html

1. Java Servlet Technology

What is Java Servlet?

* Java Servlets Intro & Development Cycle

  - Java Servlet Structure
  - Java Servlet sample that generates “Plain Text”
  - Compiling and testing Java Servlet
  - A Simple Servlet Generating HTML

* Processing the Request: Form Data

  - Introduction (Format, URL-encoding, GET, POST)
  - Example: Reading Specific Parameters
  - Example: Making Table of All Parameters

1. Java Servlet Technology

What is Java Servlet?

* HTTP Request Headers
  - Common Request Headers
  - Sample: Java Servlet for displaying HTML table of the Request Headers

* HTTP Status Codes & HTTP Response Headers
  - Overview: Status Codes & Response Headers
  - Set Status Codes from Java Servlets
  - Set Response Headers from Java Servlets
  - Sample: Refresh at each 3 seconds based on Response Headers

* Handling Cookies
  - Cookies Intro
  - Java Servlet Cookie API
  - Sample: Set/Get Cookie for Internet Explorer & Mozilla

* Session Tracking
  - Session Tracking Overview
  - Java Servlet Session Tracking API + Sample

1. Java Servlet Technology

Java Servlet API

```
import java.io.IOException;
import java.io.PrintWriter;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class HelloWorld extends HttpServlet {
  public void doGet(HttpServletRequest request, HttpServletResponse response)
      throws ServletException, IOException {
    PrintWriter out = response.getWriter();
    out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 " +
                                          "Transitional//EN\">\n" +
                "<html>\n" +
                "<head><title>Hello WWW</title></head>\n" +
                "<body>\n" +
                "<h1>Hello WWW</h1>\n" +
                "</body></html>");
  }
}
```

Java Servlet Technology Intro

Servlet Example: Showing Request Headers

Request Method: GET
Request URI: /servlet/hall.ShowRequestHeaders
Request Protocol: HTTP/1.0

Header Name	Header Value
Connection	Keep-Alive
User-Agent	Mozilla/4.05 [en] (WinNT; I)
Host	webdev.apl.jhu.edu
Accept	image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, image/png, */*
Accept-Language	en
Accept-Charset	iso-8859-1,*,utf-8
Cookie	searchString=java servlet cookies; numResults=10; searchEngine=infoseek

Java Servlet Technology Intro

Servlet Example: Showing Request Headers

Request Method: GET
Request URI: /servlet/hall.ShowRequestHeaders
Request Protocol: HTTP/1.1

Header Name	Header Value
Accept	image/gif, image/x-xbitmap, image/jpeg, image/pjpeg, application/vnd.ms-excel, application/msword, application/vnd.ms-powerpoint, */*
Accept-Language	en-us
Accept-Encoding	gzip, deflate
User-Agent	Mozilla/4.0 (compatible; MSIE 4.01; Windows NT)
Host	webdev.apl.jhu.edu
Connection	Keep-Alive

Java Servlet Technology Intro – Response Codes and Errors

```
      String url =
        response.encodeURL(searchSpec.makeURL(searchString,
                                              numResults));
      response.sendRedirect(url);
      return;
    }
}
response.sendError(response.SC_NOT_FOUND,
                   "No recognized search engine specified.");
```

Java Servlet Technology Intro

Status Code	Associated Message
100	Continue
101	Switching Protocols
200	OK
201	Created
202	Accepted
203	Non-Authoritative Information
204	No Content
205	Reset Content
206	Partial Content
300	Multiple Choices
301	Moved Permanently
302	Found
303	See Other
304	Not Modified
305	Use Proxy
307	Temporary Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
405	Method Not Allowed
406	Not Acceptable
407	Proxy Authentication Required
408	Request Timeout
409	Conflict
410	Gone
411	Length Required
412	Precondition Failed
413	Request Entity Too Large
414	Request URI Too Long
415	Unsupported Media Type
416	Requested Range Not Satisfiable
417	Expectation Failed
500	Internal Server Error
501	Not Implemented
502	Bad Gateway
503	Service Unavailable
504	Gateway Timeout
505	HTTP Version Not Supported

Java Servlet Technology Intro – Response Headers

Header
Allow
Content-Encoding
Content-Length
Content-Type
Date
Expires
Last-Modified
Location
Refresh
Server
Set-Cookie
WWW-Authenticate

Java Servlet Technology Intro – Cookies and Session Tracking

There are tech issues with HTTP, because it is a ”stateless“ protocol.

Usually this may be solved as it follows:

1. Cookies. Most used way to transform HTTP from “stateless” to “state-full”. The objects associated to the cookie are NOT going through the network and are stored on the web server side.

2. URL Rewriting. For each HTTP request there is attached in the end or the URL an unique char string generated by the web server.

3. Hidden form fields. Are used HTML tags such as:

<INPUT TYPE="HIDDEN" NAME="session" VALUE="...">

1. Java Servlet Technology

Java Servlet Technology Intro – Cookies and Session Tracking

```
//create cookie 1 - implicit value in seconds of cookie is within the session
Cookie userCookie = new Cookie("CookieGigel", "CucuBau");
response.addCookie(userCookie);

//create cookie 2 - is per year
Cookie userCookie2 = new Cookie("CookieIon", "IONIONION");
userCookie2.setMaxAge(SECONDS_PER_YEAR);
response.addCookie(userCookie2);

…

Cookie[] cookies = request.getCookies();
if (cookies != null) {

    for(int i=0; i<cookies.length; i++) {
     Cookie cookie = cookies[i];
     if ("CookieGigel".equals(cookie.getName())) {…
```

1. Java Servlet Technology

Java Servlet Technology Intro – Cookies and Session Tracking

```
public void processRequest(HttpServletRequest request, HttpServletResponse response)
throws ServletException, IOException {

   HttpSession session = request.getSession(true);
   response.setContentType("text/html");
   PrintWriter out = response.getWriter();
   String title = "Show Session"; String heading;
   Integer accessCount = new Integer(0);

   if (session.isNew()) {
     heading = "Welcome, Newcomer";
   } else {
     heading = "Welcome Back";
     Integer oldAccessCount =(Integer)session.getAttribute("accessCount");
     if (oldAccessCount != null) {
       accessCount = new Integer(oldAccessCount.intValue() + 1);
     }
   }
   session.setAttribute("accessCount", ""+accessCount);
```

Section Conclusion

Fact: **DAD needs Web Programming**

In few **samples** it is simple to remember: Java Servlet Programming with HTTP protocol analysis in real time for request headers, responses' codes and headers, session tracking – generates standards HTML pages as entering gate for distributed computing and systems.

Java Server Page – JSP & Java Servlet Technology Intro

Communicate & Exchange Ideas

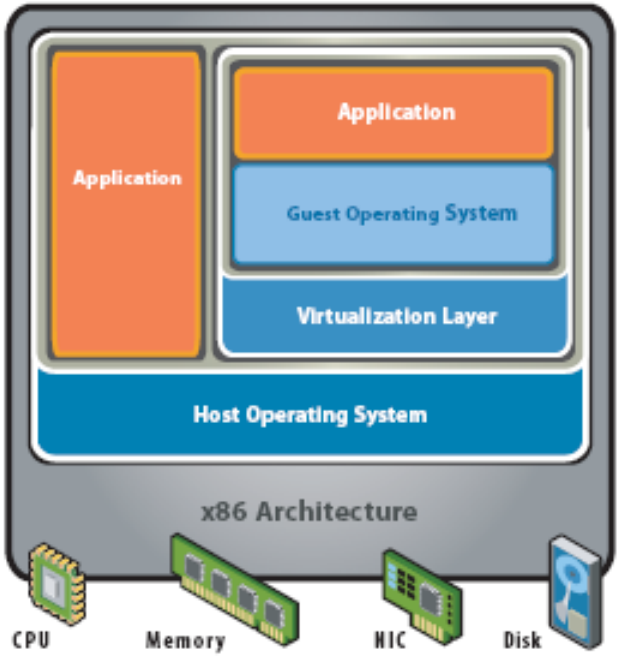
Questions & Answers!

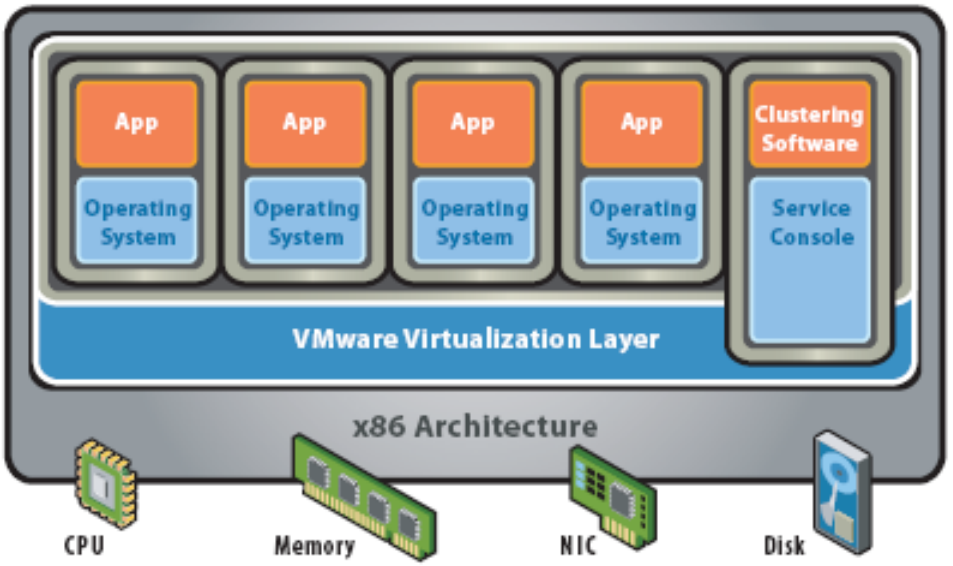
But wait…

There's More!

Distributed Systems: Cloud Architecture

Cloud Concepts – Intro – Virtualization Overview





Hosted Architecture

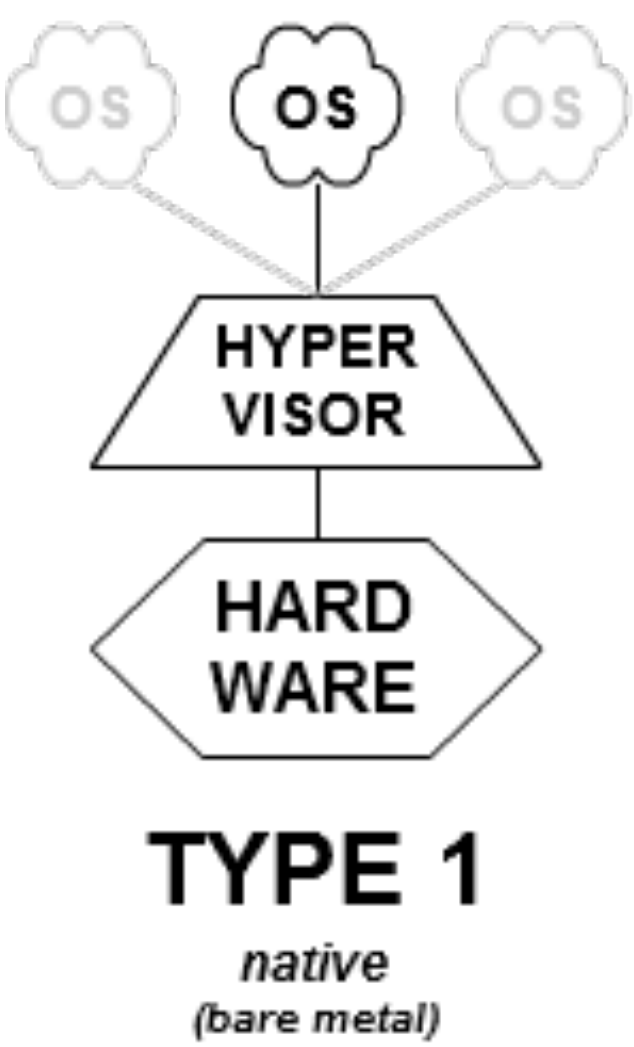
- Installs and runs as an application
- Relies on host OS for device support and physical resource management

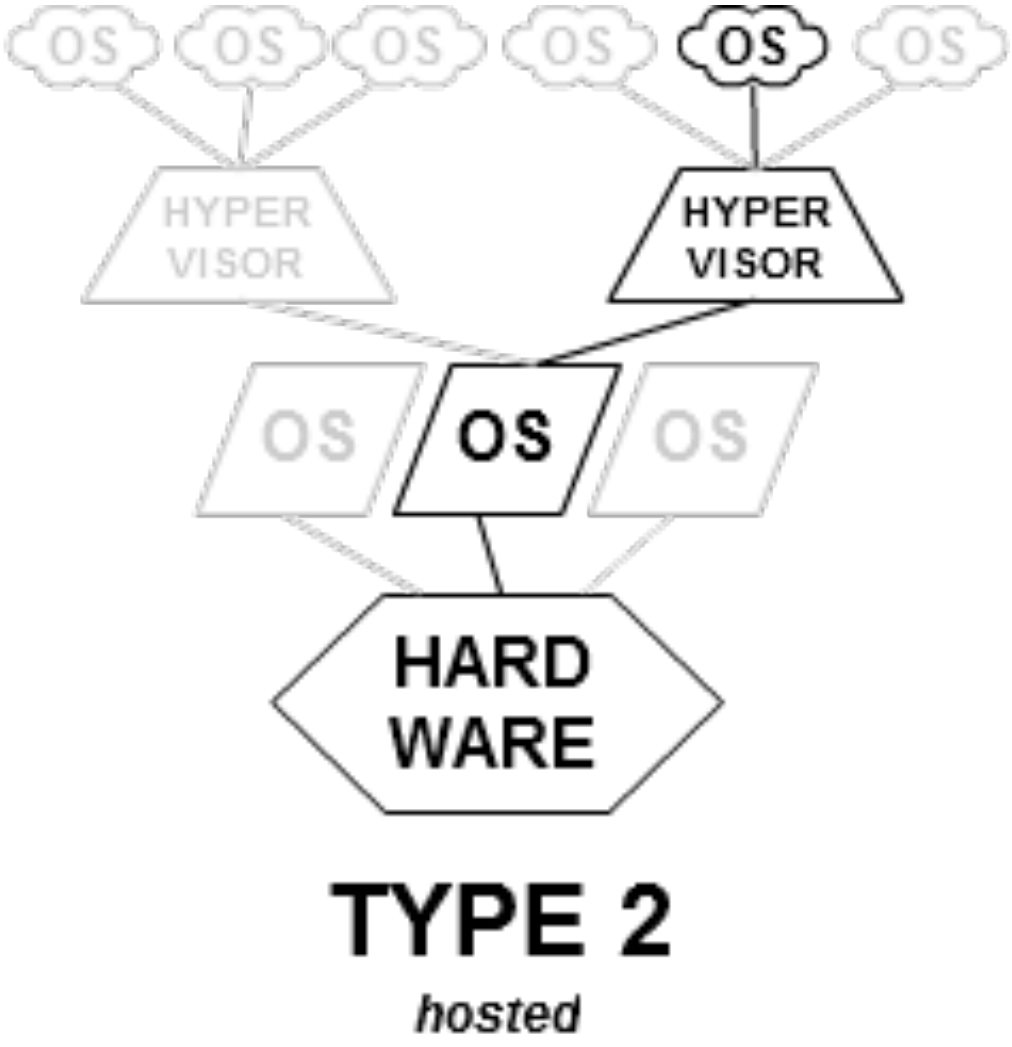
Bare-Metal (Hypervisor) Architecture

- Lean virtualization-centric kernel
- Service Console for agents and helper applications

Figure 2: Virtualization Architectures

Cloud Concepts – Intro – Virtualization Overview

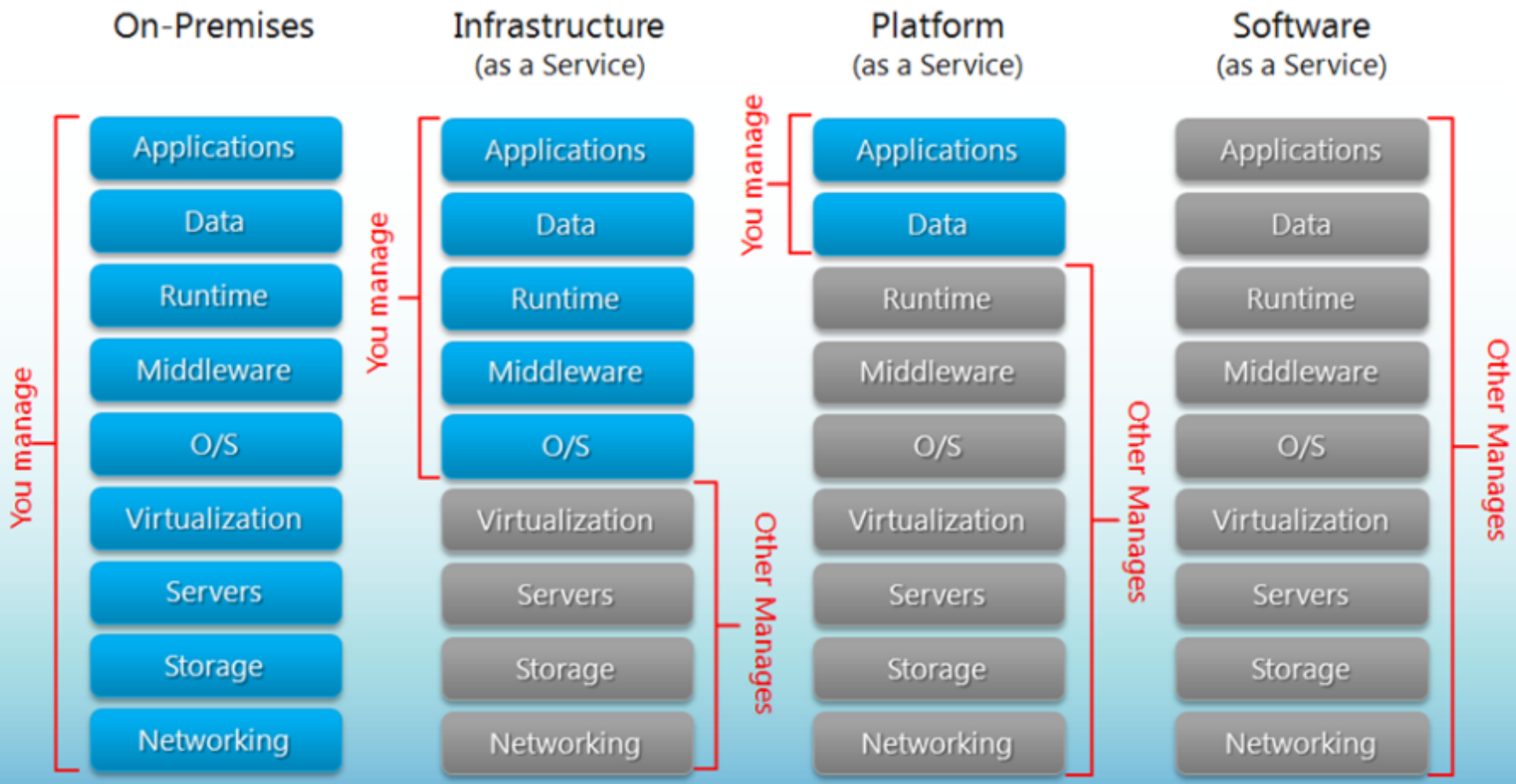




Distributed Systems: Cloud

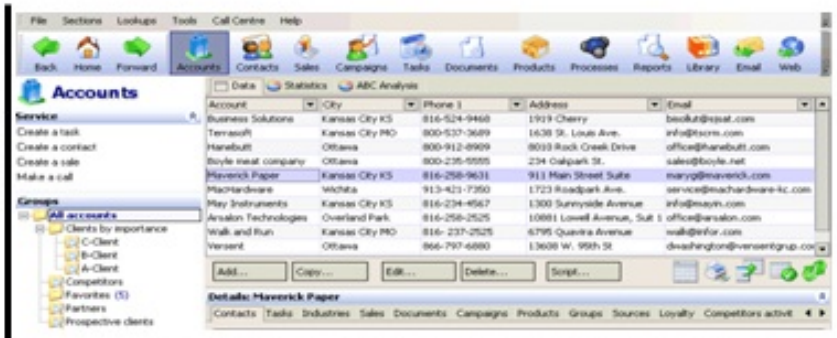
Separation of Responsibilities

Cloud Concepts – Intro – IaaS, PaaS, SaaS



Cloud Concepts – Intro – IaaS, PaaS, SaaS

SaaS



SalesForce.com, Google Apps

PaaS

Tomcat App Server

Java VM

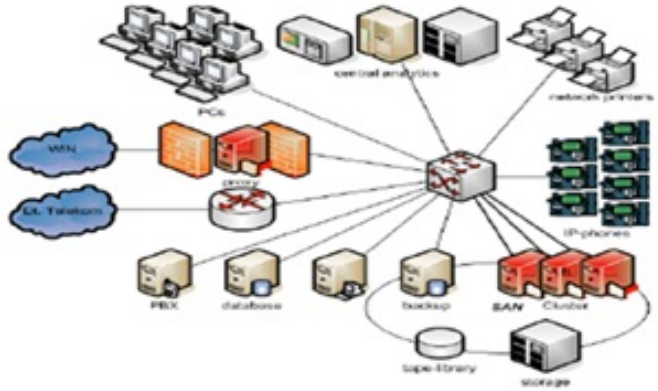
MySQL DB

Ubuntu Linux

Google App Engine for: Java, Ruby, Python & GO

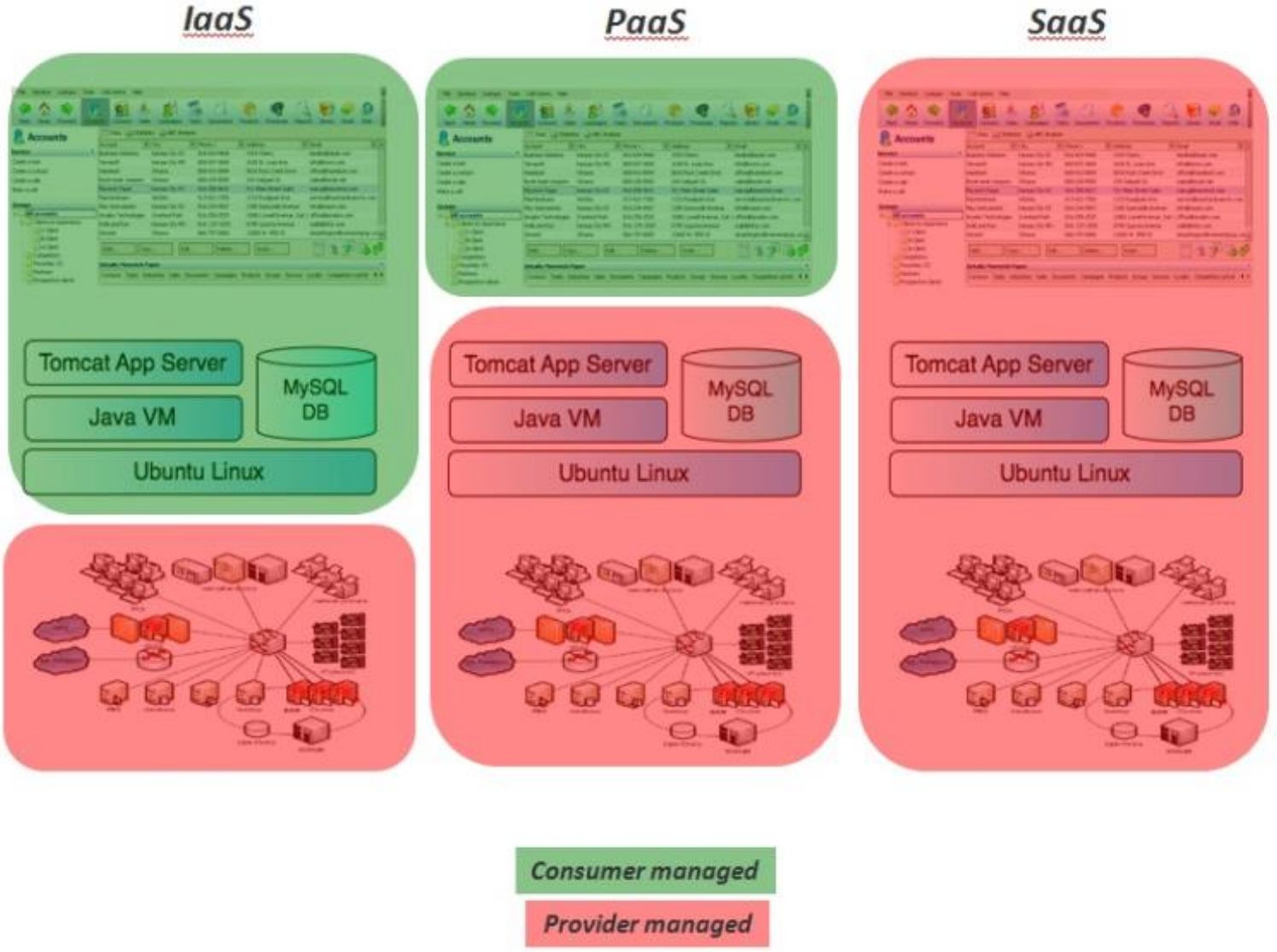
VMForce.com, MS Azure

IaaS



vCloud Express/Datacenter, Amazon EC2

Cloud Concepts – Intro – IaaS, PaaS, SaaS



Cloud Concepts – Intro – using Google App Engine Cloud PaaS

https://cloud.google.com/appengine/docs/standard/java/quickstart

https://ssh.cloud.google.com/cloudshell/editor

secitc@cloudshell:~$ gcloud projects create wserv001
it must be created from Web GUI:
https://console.cloud.google.com/projectcreate?previousPage=%2Fprojectselector2%2Fbilling%3Flang%3Djava%26st%3Dtrue%26_ga%3D2.187602464.925220531.1582115517-284239323.1582115517%26pli%3D1&project=&folder=&organizationId=0

secitc@cloudshell:~$ gcloud projects describe wserv0001

createTime: '2020-02-19T12:38:54.623Z'
lifecycleState: ACTIVE
name: wserv001
projectId: wserv001projectNumber: '163714956555'

Cloud Concepts – Intro – using Google App Engine Cloud PaaS

secitc@cloudshell:~$ **gcloud app create --project=wserv0001**
Please enter your numeric choice: 6
Please choose the region where you want your App Engine application located:
[1] asia-east2 (supports standard and flexible) [2] asia-northeast1 (supports standard and flexible) [3] asia-northeast2 (supports standard and flexible) [4] asia-south1 (supports standard and flexible) [5] australia-southeast1 (supports standard and flexible) [6] europe-west (supports standard and flexible) …
Creating App Engine application in project [wserv0001] and region [europe-west]....done.Success! The app is now created. Please use `gcloud app deploy` to deploy your first app.

secitc@cloudshell:~$ **git clone https://github.com/GoogleCloudPlatform/getting-started-java.git**
secitc@cloudshell:~$ **pwd**
/home/secitc
secitc@cloudshell:~$ **ls**
getting-started-java README-cloudshell.txt

Cloud Concepts – Intro – using Google App Engine Cloud PaaS

secitc@cloudshell:~$ **cd getting-started-java/appengine-standard-java8/helloworld**

localhost
secitc@cloudshell:~$ **mvn package appengine:run -DprojectId=wserv0001**

real instance
secitc@cloudshell:~$ **gcloud config set project wserv0001**
when is finished:
secitc@cloudshell:~$ **gcloud config unset project**

modify wserv0001 in pom.xml and then:
secitc@cloudshell:~$ **mvn package appengine:deploy**

https://wserv0001.appspot.com
https://wserv0001.appspot.com/hello
secitc@cloudshell:~$ **gcloud app browse**

Thanks!

DAD – Distributed Application Development
End of Lecture 4 – Section 3

Lecture 5

S3 - Summary of Server Side/Web Development in JEE

presentation

DAD – Distributed Applications Development

Cristian Toma

D.I.C.E/D.E.I.C – Department of Economic Informatics & Cybernetics
www.dice.ase.ro

Cristian Toma – Business Card

Agenda for Lecture 5

DAD Section 3 - Summary of Web Development in JEE, JSP Architecture, Lifecycle, Samples

JSP - Java Server Page Intro & Recap

1. Java Server Pages – JSP Technology

What is JSP – Java Server Pages?

- JSP Architecture
- JSP Life-cycle
- JSP Syntax + Directive + DEMO
- JSP Predefined Variables
- JSP Predefined Directives + Actions
- JSP Beans
- DEMO JSP & JSPX

JSP Technology Necessity

With Java Servlet technology is easy to:

- Get the data from HTML form
- Get the headers values from HTTP request
- Set the error status code and header from HTTP response
- Use of “cookies” & “session tracking” – “state-full” HTTP
- Sharing the data between Java servlets

With Java Servlet technology is NOT easy to:

- Generate HTML tags using the method “println” from ‘PrintWriter’ class
- Maintenance of HTML generated code

JSP Approach

Ideas:

– Use of standard HTML code in most of the pages sections

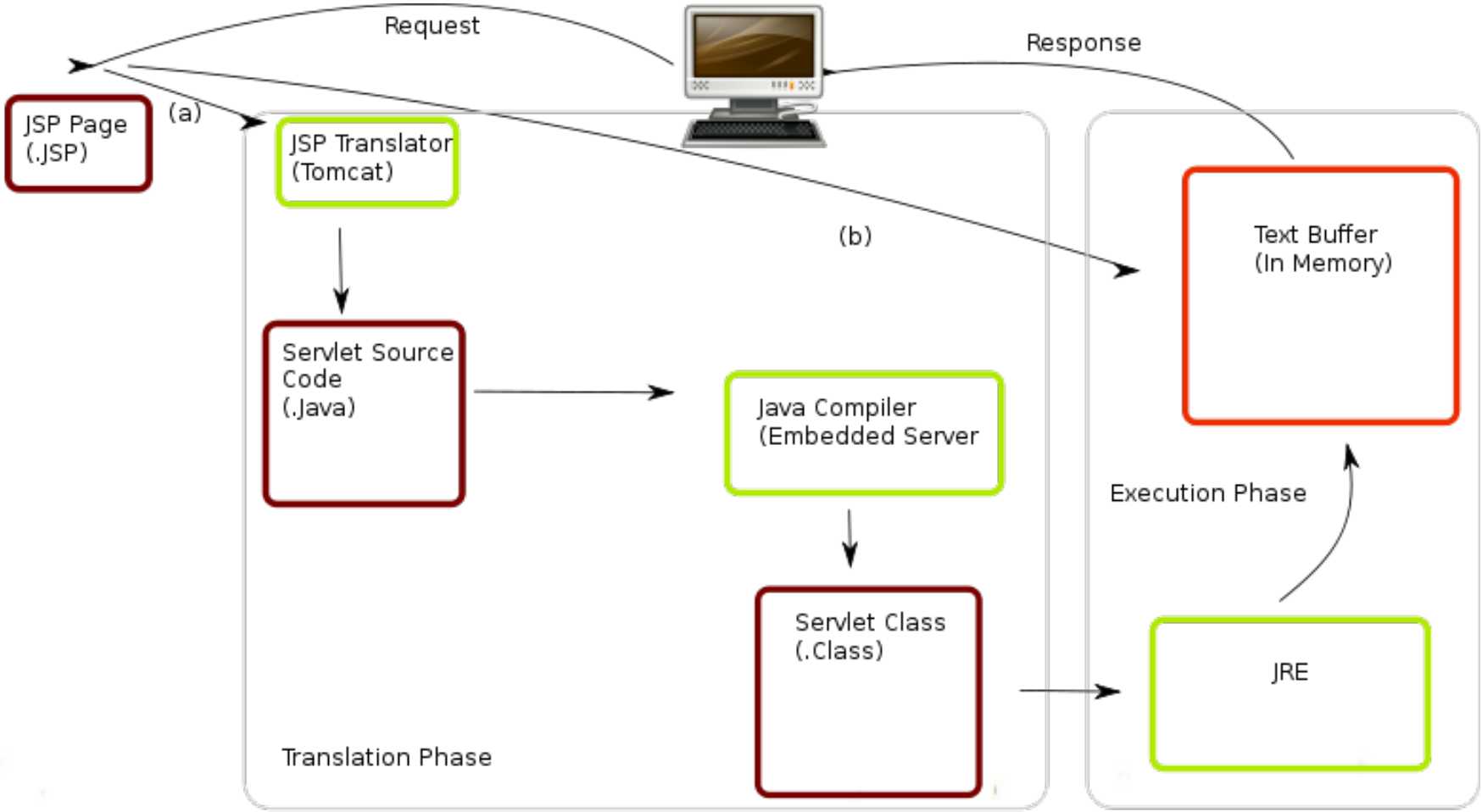
– The entire JSP page is “translated” into Java servlet, and the servlet’s methods are called for each HTTP request

- **Sample:**

```
<!DOCTYPE …>
<HTML>
<HEAD>
 <TITLE>Order Confirmation</TITLE>
 <LINK REL=STYLESHEET HREF="JSP-Styles.css“ TYPE="text/css">
</HEAD>
<BODY>
 <H2>Order Confirmation</H2>
 Thanks for ordering <I><%= request.getParameter("title") %></I>
</BODY>
</HTML>
```

1. Java Server Pages – JSP Technology

JSP Translation Process



JSP Container
(a) Translation occurs at this point if JSP has been changed or is new
(b) if not, translation is skipped.

1. Java Server Pages – JSP Technology

JSP Lifecycle

	Page first written	Request #1	Request #2	Server restarted	Request #3	Request #4	Page modified	Request #5	Request #6
JSP page translated into servlet		Yes	No		No	No		Yes	No
Servlet compiled		Yes	No		No	No		Yes	No
Servlet instantiated and loaded into server's memory		Yes	No		Yes	No		Yes	No
init (or equivalent) called		Yes	No		Yes	No		Yes	No
doGet (or equivalent) called		Yes	Yes		Yes	Yes		Yes	Yes

1. Java Server Pages – JSP Technology

DEMO Sample – test01.jsp:

Although in Java Servlet the may do as much as in JSP (because JSP is Java Servlet), it is recommended to use JSP for:

- Writing easy HTML code
- Debugging and maintaining the HTML code
- There is nothing about environment variables, just simple copy the JSP files in a web server directory

test.jsp

```
<HTML>
<HEAD>
 <TITLE>Test jsp</TITLE>
</HEAD>
<BODY>
<H2>JSP Expressions</H2>
<UL>
<LI>Current time: <%= new java.util.Date() %>
<LI>Server: <%= application.getServerInfo() %>
<LI>Session ID: <%= session.getId() %>
<LI>The <CODE>testParam</CODE> form parameter: <%= request.getParameter("testParam") %>
</UL>
</BODY>
</HTML>
```

1. Java Server Pages – JSP Technology

JSP API

JSP Element	Syntax	Interpretation	Notes
JSP Expression	`<%= expression %>`	Expression is evaluated and placed in output.	XML equivalent is `<jsp:expression>` `expression` `</jsp:expression>`. Predefined variables are `request`, `response`, `out`, `session`, `application`, `config`, and `pageContext` (available in scriptlets also).
JSP Scriptlet	`<% code %>`	Code is inserted in `service` method.	XML equivalent is `<jsp:scriptlet>` `code` `</jsp:scriptlet>`.
JSP Declaration	`<%! code %>`	Code is inserted in body of servlet class, outside of `service` method.	XML equivalent is `<jsp:declaration>` `code` `</jsp:declaration>`.

1. Java Server Pages – JSP Technology

JSP API

JSP Element	Syntax	Interpretation	Notes
JSP page Directive	`<%@ page att="val" %>`	Directions to the servlet engine about general setup.	XML equivalent is `<jsp:directive.page att="val"\>`. Legal attributes, with default values in bold, are: • import="*package.class*" • contentType="*MIME-Type*" • isThreadSafe="**true**|false" • session="**true**|false" • buffer="*size*kb|none" • autoflush="**true**|false" • extends="*package.class*" • info="*message*" • errorPage="*url*" • isErrorPage="true|**false**" • language="java"
JSP `include` Directive	`<%@ include file="url" %>`	A file on the local system to be included when the JSP page is translated into a servlet.	XML equivalent is `<jsp:directive.include file="url"\>`. The URL must be a relative one. Use the `jsp:include` action to include a file at request time instead of translation time.

1. Java Server Pages – JSP Technology

JSP API

JSP Element	Syntax	Interpretation	Notes
JSP Comment	`<%-- comment --%>`	Comment; ignored when JSP page is translated into servlet.	If you want a comment in the resultant HTML, use regular HTML comment syntax of `<-- comment -->`.
The `jsp:include` Action	`<jsp:include page="relative URL" flush="true"/>`	Includes a file at the time the page is requested.	If you want to include the file at the time the page is translated, use the `page` directive with the `include` attribute instead. **Warning**: on some servers, the included file must be an HTML file or JSP file, as determined by the server (usually based on the file extension).

1. Java Server Pages – JSP Technology

JSP API

JSP Element	Syntax	Interpretation	Notes

The `jsp:useBean` Action	`<jsp:useBean att=val*/>` or
	`<jsp:useBean att=val*>`
	`...`
	`</jsp:useBean>`
		Find or build a Java Bean.	Possible attributes are:
			• id="*name*"
			• scope="**page**|request|session|application"
			• class="*package.class*"
			• type="*package.class*"
			• beanName="*package.class*"

The `jsp:setProperty` Action	`<jsp:setProperty att=val*/>`	Set bean properties, either explicitly or by designating that value comes from a request parameter.	Legal attributes are • name="*beanName*" • property="*propertyName*|*" • param="*parameterName*" • value="*val*"

The `jsp:getProperty` Action	`<jsp:getProperty`
	`name="propertyName"`
	`value="val"/>`
		Retrieve and output bean properties.	

1. Java Server Pages – JSP Technology

JSP API

JSP Element	Syntax	Interpretation	Notes
The `jsp:forward` Action	`<jsp:forward page="relative URL"/>`	Forwards request to another page.	
The `jsp:plugin` Action	`<jsp:plugin attribute="value"*> ... </jsp:plugin>`	Generates `OBJECT` or `EMBED` tags, as appropriate to the browser type, asking that an applet be run using the Java Plugin.	

JSP API

Predefined Objects in JSP

In order to simplify the Java source code in the scriptlets and expressions, in JSP there are 8 predefined objects.

1 request

This is the HttpServletRequest associated with the request, and lets you look at the request parameters (via getParameter), the request type (GET, POST, HEAD, etc.), and the incoming HTTP headers (cookies, Referer, etc.). Strictly speaking, request is allowed to be a subclass of ServletRequest other than HttpServletRequest, if the protocol in the request is something other than HTTP. This is almost never done in practice.

2 response

This is the HttpServletResponse associated with the response to the client. Note that, since the output stream (see out below) is buffered, it is legal to set HTTP status codes and response headers, even though this is not permitted in regular servlets once any output has been sent to the client.

JSP API

3 out

This is the PrintWriter used to send output to the client. However, in order to make the response object (see the previous section) useful, this is a buffered version of PrintWriter called JspWriter. Note that you can adjust the buffer size, or even turn buffering off, through use of the buffer attribute of the page directive. This was discussed in Section 5. Also note that out is used almost exclusively in scriptlets, since JSP expressions automatically get placed in the output stream, and thus rarely need to refer to out explicitly.

4 session

This is the HttpSession object associated with the request. Recall that sessions are created automatically, so this variable is bound even if there was no incoming session reference. The one exception is if you use the session attribute of the page directive to turn sessions off, in which case attempts to reference the session variable cause errors at the time the JSP page is translated into a servlet.

JSP API

5 application

This is the ServletContext as obtained via getServletConfig().getContext().

6 config

This is the ServletConfig object for this page.

7 pageContext

JSP introduced a new class called PageContext to encapsulate use of server-specific features like higher performance JspWriters. The idea is that, if you access them through this class rather than directly, your code will still run on "regular" servlet/JSP engines. It is used also for Java Bean synchronization and session tracking info storage for not allocating more beans for one page session.

8 page

This is simply a synonym for this, and is not very useful in Java. It was created as a placeholder for the time when the scripting language could be something other than Java.

JSP API

ACTIONS in JSP

The actions in JSP uses XML for controlling the behavior of the web server Java Servlet container. Using the JSP actions, the programmer can dynamically insert files, reuse Java Beans components, redirect the web browser to other web resources or generate HTML with JavaScript or Java applets.

- jsp:include – Include a file in the HTTP request time.
- jsp:useBean – Use or instantiate a JavaBean component.
- jsp:setProperty – Sets the JavaBean field value using a property.
- jsp:getProperty – Gets the JavaBean field value using a property.
- jsp:forward – Redirect an HTTP request to a new page/web resource.
- jsp:plugin - Generate the specific web browser code which contains OBJECT or EMBED HTML tag for Java Applets that are running at the client web browser.

Section Conclusion

Fact: **DAD needs Web Programming**

In few **samples** it is simple to remember: Java Server Pages Programming with HTTP protocol analysis in real time for request headers, responses' codes and headers, session tracking – generates standards HTML pages as entering gate for distributed computing and systems.

DAD Section 3 - Summary of Web Development in JEE, JSP Tag libs, JSP MVC Concept, Web Server Clusters

Java Server Page Tech Advanced Topics

2. Advanced Java Server Page – JSP Tech Topics

JSP DEMO Tag-lib

http://jakarta.apache.org/taglibs/doc/standard-doc/intro.html

Using JSTL: JSTL includes a wide variety of tags that fit into discrete functional areas. To reflect this, as well as to give each area its own namespace, JSTL is exposed as multiple tag libraries.
The URIs for the libraries are as follows:

Core: http://java.sun.com/jsp/jstl/core

XML: http://java.sun.com/jsp/jstl/xml

Internationalization: http://java.sun.com/jsp/jstl/fmt

SQL: http://java.sun.com/jsp/jstl/sql

Functions: http://java.sun.com/jsp/jstl/functions

2. Advanced Java Server Page – JSP Tech Topics

JSP DEMO Tag-lib

Area	Subfunction	Prefix
Core	Variable support	c
	Flow control	
	URL management	
	Miscellaneous	
XML	Core	x
	Flow control	
	Transformation	
I18N	Locale	fmt
	Message formatting	
	Number and date formatting	
Database	SQL	sql
Functions	Collection length	fn
	String manipulation	

Area	Function	Tags	Prefix
Core	Variable support	remove set	c
	Flow control	choose when otherwise forEach forTokens if	
	URL management	import param redirect param url param	
	Miscellaneous	catch out	

2. Advanced Java Server Page – JSP Tech Topics

JSP DEMO Tag-lib

Area	Function	Tags	Prefix
I18N	Setting Locale	setLocale requestEncoding	fmt
	Messaging	bundle message param setBundle	
	Number and Date Formatting	formatNumber formatDate parseDate parseNumber setTimeZone timeZone	

Area	Function	Tags	Prefix
XML	Core	out parse set	x
	Flow control	choose when otherwise forEach if	
	Transformation	transform param	

2. Advanced Java Server Page – JSP Tech Topics

JSP DEMO Tag-lib

Area	Function	Tags	Prefix
Database	Setting the data source	`setDataSource`	`sql`

	SQL	`query`
		`  dateParam`
		`  param`
		`transaction`
		`update`
		`  dateParam`
		`  param`	

2. Advanced Java Server Page – JSP Tech Topics

JSP MVC – Model View Controller – Model 1 – Architecture Design Pattern

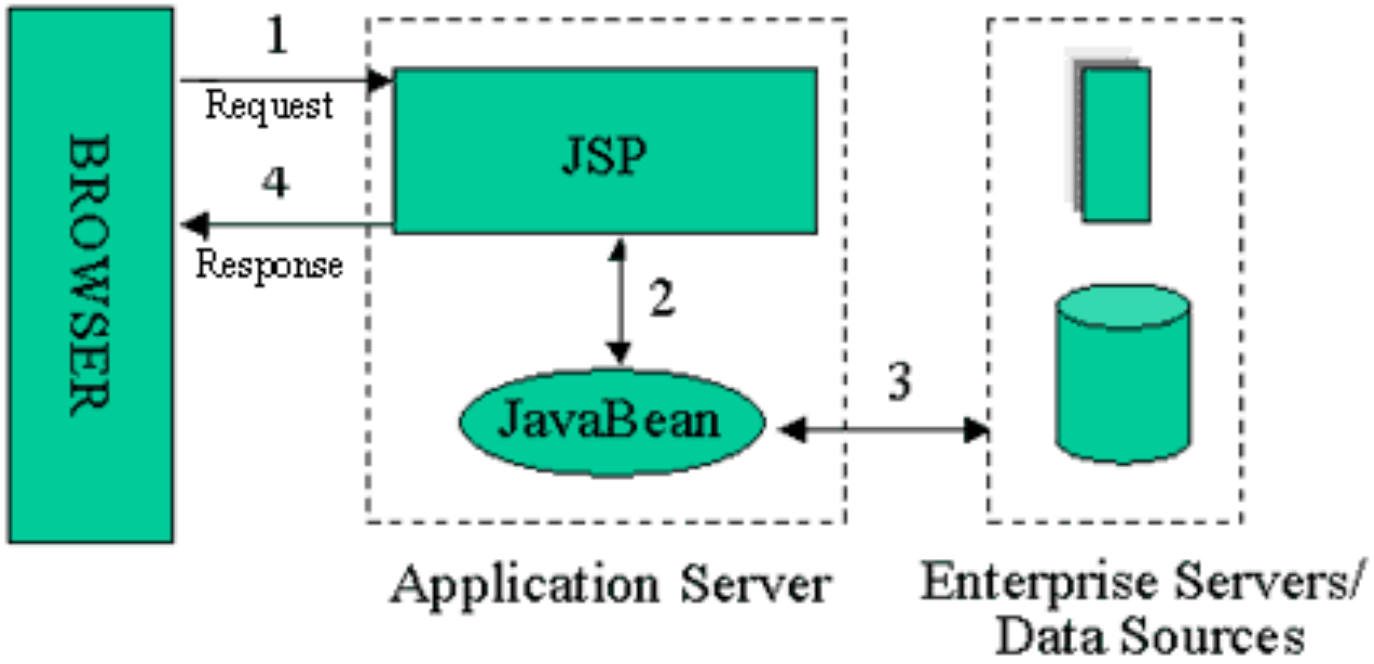
The early JSP specifications advocated two philosophical approaches for building applications using JSP technology. These approaches, termed the JSP Model 1 and Model 2 architectures, differ essentially in the location at which the bulk of the request processing was performed.

In the Model 1 architecture, shown in Figure of Model 1, the JSP page alone is responsible for processing the incoming request and replying back to the client. There is still separation of presentation from content, because all data access is performed using beans. Although the Model 1 architecture should be perfectly suitable for simple applications, it may not be desirable for complex implementations. Indiscriminate usage of this architecture usually leads to a significant amount of scriptlets or Java code embedded within the JSP page, especially if there is a significant amount of request processing to be performed. While this may not seem to be much of a problem for Java developers, it is certainly an issue if your JSP pages are created and maintained by designers -- which is usually the norm on large projects.

Ultimately, it may even lead to an unclear definition of roles and allocation of responsibilities, causing easily avoidable project-management headaches.

2. Advanced Java Server Page – JSP Tech Topics

JSP MVC – Model View Controller – Model 1 – Architecture Design Pattern



2. Advanced Java Server Page – JSP Tech Topics

JSP MVC – Model View Controller – Model 2 – Architecture Design Pattern

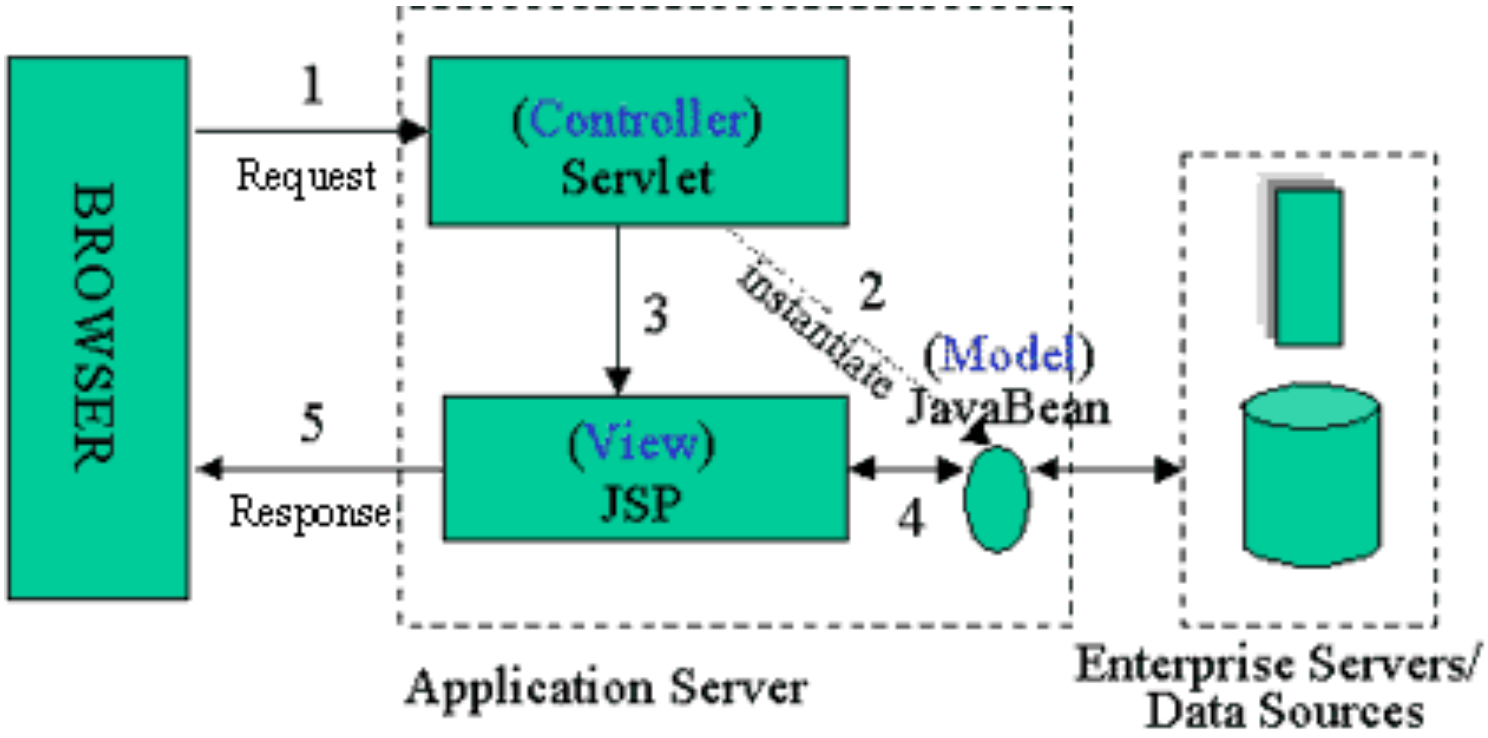
The Model 2 architecture, shown in Figure of Model 2, is a hybrid approach for serving dynamic content, since it combines the use of both servlets and JSP. It takes advantage of the predominant strengths of both technologies, using JSP to generate the presentation layer and servlets to perform process-intensive tasks.

Here, the servlet acts as the *controller* and is in charge of the request processing and the creation of any beans or objects used by the JSP, as well as deciding, depending on the user's actions, which JSP page to forward the request to. Note particularly that there is no processing logic within the JSP page itself; it is simply responsible for retrieving any objects or beans that may have been previously created by the servlet, and extracting the dynamic content from that servlet for insertion within static templates.

This approach typically results in the cleanest separation of presentation from content, leading to clear delineation of the roles and responsibilities of the developers and page designers on your programming team. In fact, the more complex your application, the greater the benefits of using the Model 2 architecture should be.

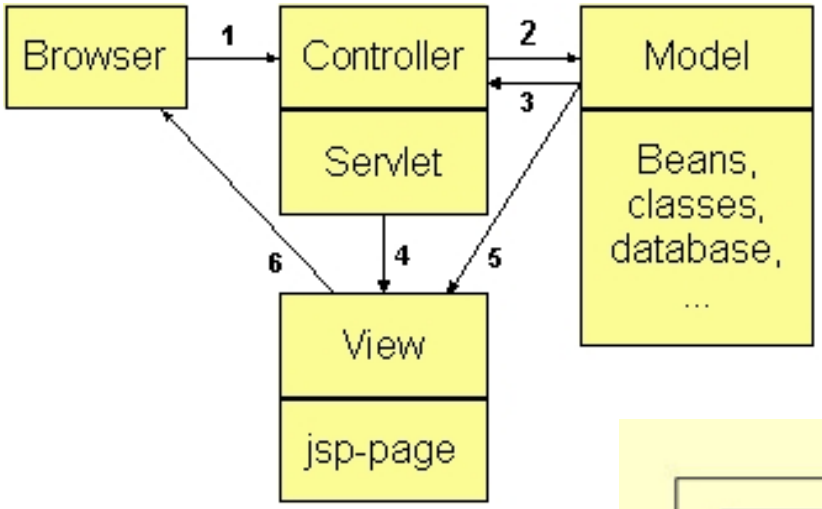
2. Advanced Java Server Page – JSP Tech Topics

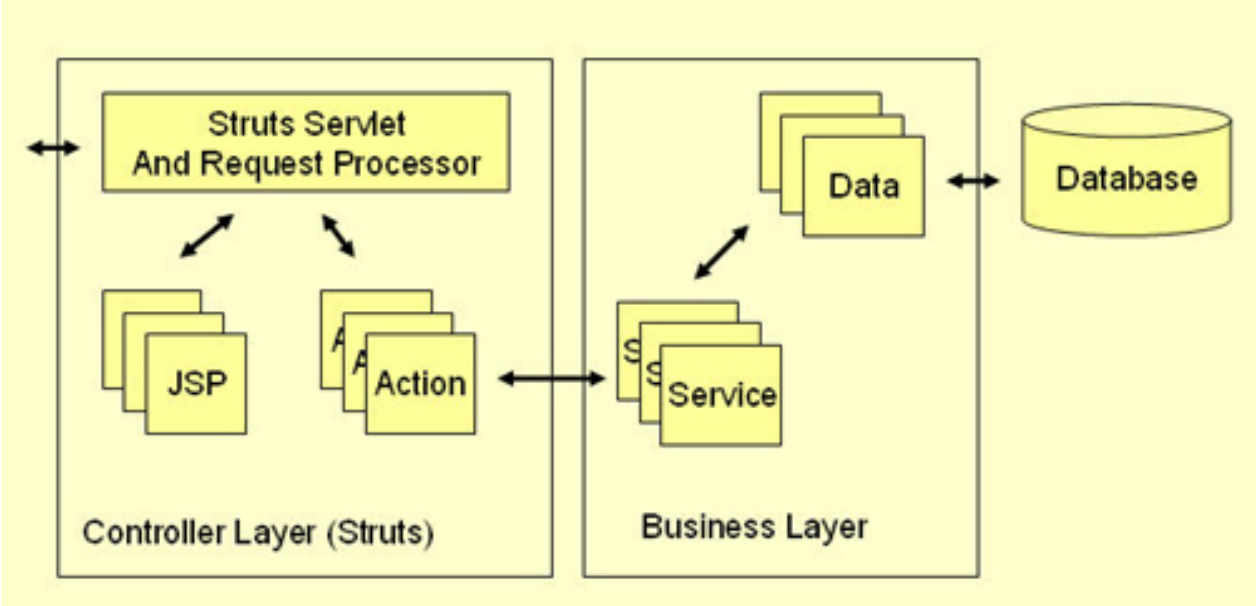
JSP MVC – Model View Controller – Model 2 – Architecture Design Pattern



2. Advanced Java Server Page – JSP Tech Topics

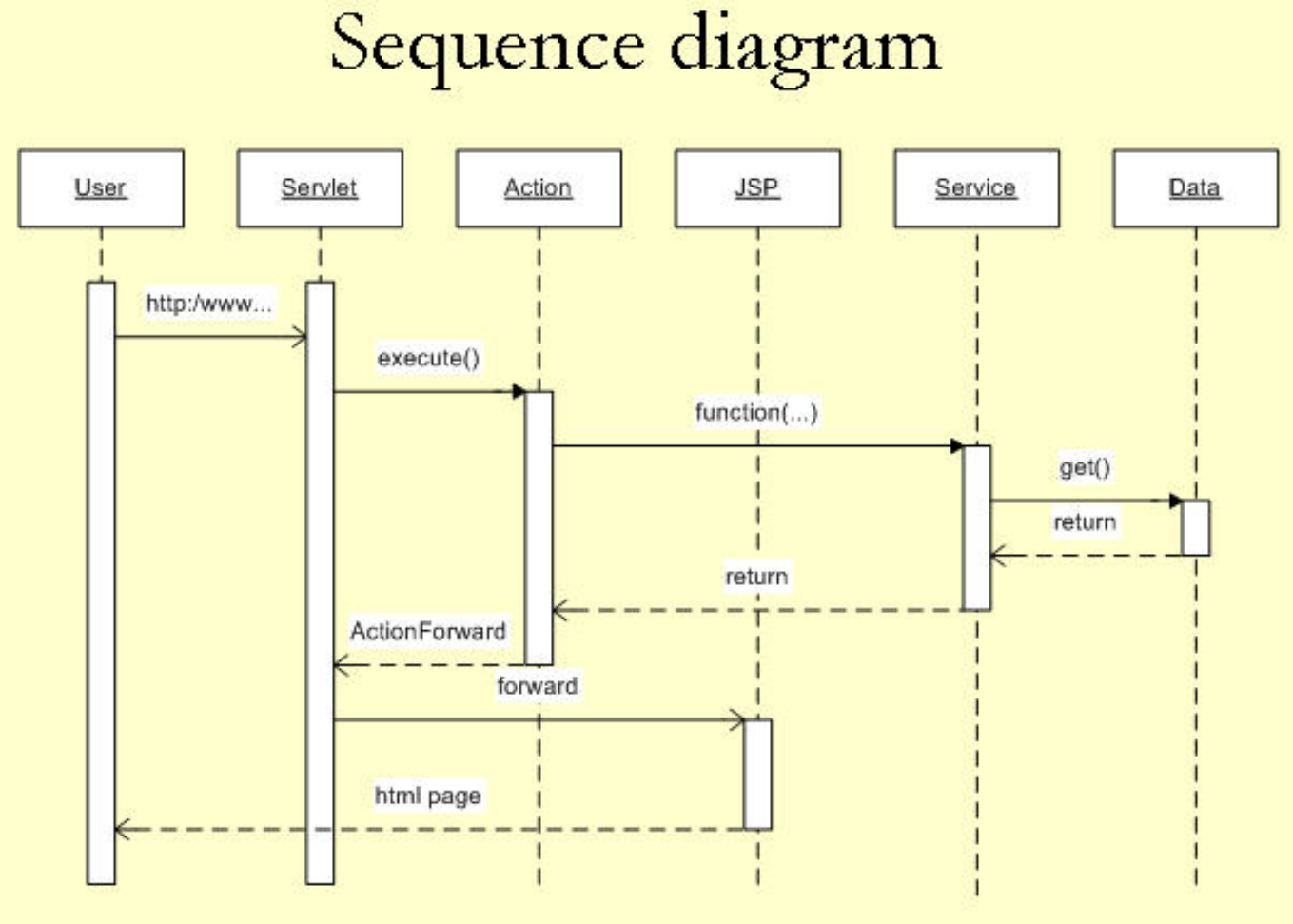
JSP MVC – Model View Controller – Model 2 – Architecture Design Pattern





2. Advanced Java Server Page – JSP Tech Topics

JSP MVC – Model View Controller – Model 2 – Architecture Design Pattern



2. Advanced Java Server Page – JSP Tech Topics

JSP MVC – Model View Controller – Model 2 – Architecture Design Pattern

Java MVC – Struts Intro

1. User clicks on a link in an HTML page.

2. Servlet controller receives the request, looks up mapping information in struts-config.xml, and routes to an action.

3. Action makes a call to a Model layer service.

4. Service makes a call to the Data layer (database) and the requested data is returned.

5. Service returns to the action.

6. Action forwards to a View resource (JSP page)

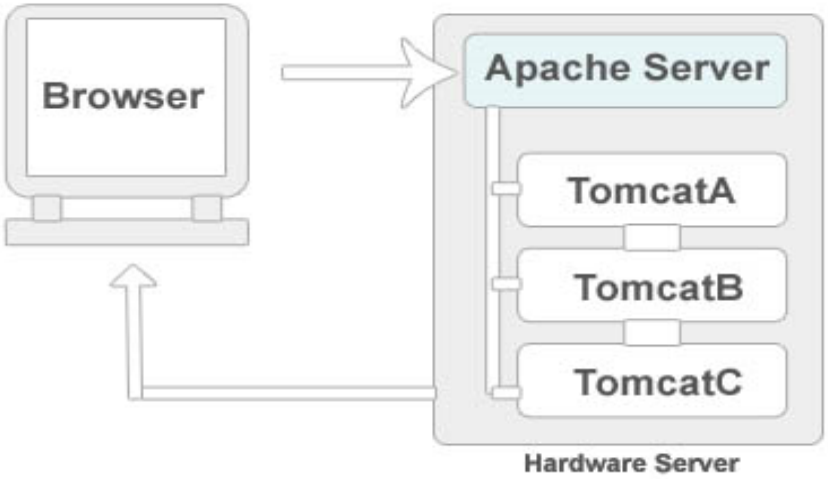
7. Servlet looks up the mapping for the requested resource and forwards to the appropriate JSP page.

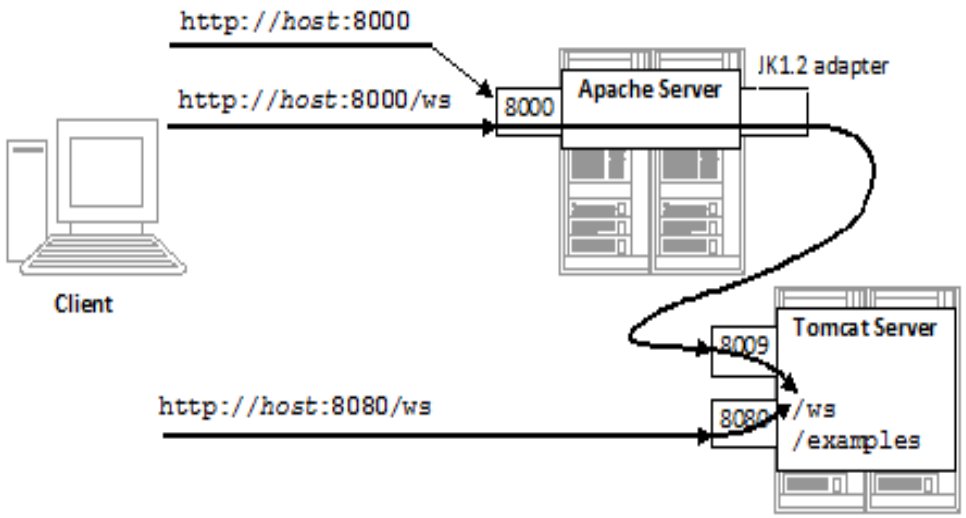
8. JSP file is invoked and sent to the browser as HTML.
9. User is presented with a new HTML page in a web browser.

2. Advanced Java Server Page – JSP Tech Topics

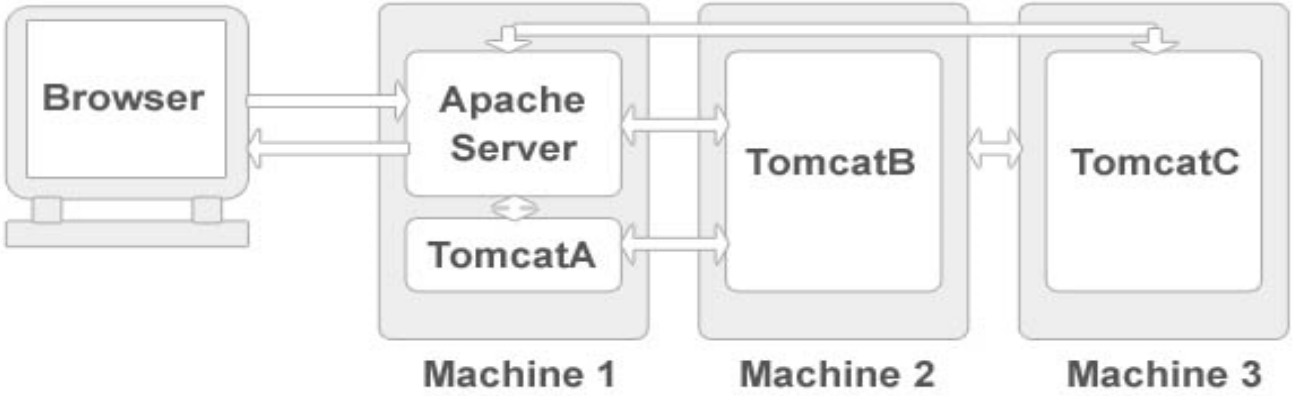
Web Server Cluster

Horizontal Clustering



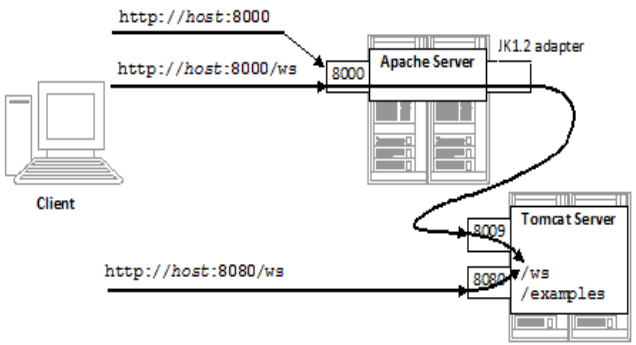


Vertical Clustering



2. Advanced Java Server Page – JSP Tech Topics

Web Server Cluster




```
<?xml version="1.0" encoding="ISO-8859-1"?>

<web-app xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee htt
  version="2.5">
  <distributable />

</web-app>
```

1. <Server port="8005" shutdown="SHUTDOWN">

2. <Connector port="8080" protocol="HTTP/1.1"
   connectionTimeout="20000"
   redirectPort="8443" />

3. <Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />

4. Add jvmRoute
<Engine name="Catalina" defaultHost="localhost" jvmRoute="tomcatC">

5. Uncommented clustering tag
<Cluster className="org.apache.catalina.ha.tcp.SimpleTcpCluster"/>

2. Advanced Java Server Page – JSP Tech Topics

Web Server Cluster

DEMO – httpd.conf

for horizontal tomcat clustering

```
workers.tomcat_home=/tomcatA
workers.java_home=$JAVA_HOME
ps=/
worker.list=tomcatA,tomcatB,tomcatC,loadbalancer


worker.tomcatA.port=8009
worker.tomcatA.host=192.168.1.1
worker.tomcatA.type=ajp13
worker.tomcatA.lbfactor=1


worker.tomcatB.port=8009
worker.tomcatB.host=192.168.1.2
worker.tomcatB.type=ajp13
worker.tomcatB.lbfactor=1

worker.tomcatC.port=8009
worker.tomcatC.host=192.168.1.3
worker.tomcatC.type=ajp13
worker.tomcatC.lbfactor=1

worker.loadbalancer.type=lb
worker.loadbalancer.balanced_workers=tomcatA,tomcatB,tomcatC
worker.loadbalancer.sticky_session=1
```

```
LoadModule jk_module modules/mod_jk-apache-2.2.4.so
JkWorkersFile "C:\cluster\Apache\conf\workers.properties"
JkLogFile "logs/mod_jk.log"
JkLogLevel error
JkMount /cluster loadbalancer
JkMount /cluster/* loadbalancer
```

lbfactor properties define for load balancing factor, restrict number of request to send particular tomcat instance
e.g
worker.tomcatC.lbfactor=100
increase and decrease request to this tomcatC instance

Java Server Page Programming
for easy sharing

Section Conclusions

Java Servlet & JSP Programming uses HTTP knowledge.

Any JSP file is translated into a Java Servlet by the Java web app server

Java Servlets migrates the boilerplate configuration code from web.xml to Java source code though annotations

Most important predefined objects in JSP are: request, response, out, session, application, config, pageContext, page

Most important actions in JSP are: jsp:include; jsp:useBean, (jsp:setProperty, jsp:getProperty); jsp:forward, jsp:plugin

JSP "directives" are different that "actions" + tag-libs/filters are useful for a modular and reliable solution

MVC – Model View Controller is a design pattern implemented with success by various frameworks in Java

For HA – High Availability and Scalability, the one may use Web Servers Clusters or Cloud Solutions - such as *Google App Engine* – Cloud PaaS Platform for Java, Ruby, Python or GO

Cloud Computing Technology Intro – IaaS, PaaS, SaaS

Communicate & Exchange Ideas

Questions & Answers!

But wait…

There's More!

Thanks!

DAD – Distributed Application Development
End of Lecture 5

node.js Recap, REST API and JavaScript/ECMAScript/Java Microservices

Lecture 09

presentation

DAD – Distributed Applications Development

Cristian Toma

D.I.C.E/D.E.I.C – Department of Economic Informatics & Cybernetics
www.dice.ase.ro

Cristian Toma – Business Card

Agenda for Lecture 09

Java Script: CallBack Functions and Async Programming, Buffers, Promise, Event-Emitter, node.js modules

ECMAScript/node.js Overview

About node.js …

- Created by Ryan Dahl in 2009
- Development && maintenance sponsored by Joyent
- License: MIT
- Latest LTS Version (Y2020): 12.16.1 (includes npm 6.13.4)
- The syntax is compliant with Java Script / ECMAScript 2015+
- Based on Google V8 Engine
- +100 000 packages
- it has interpreter on x86, ARM, ESP8266, etc. controllers, on various OS-es: MacOS, Linux, Windows, Linux Embedded, etc.

https://nodejs.org | https://nodejs.org/en/download/

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

Other projects as complementary to node.js …

- Vert.x => Polygot programming
- Akka => Scala and Java/Kotlin
- Tornado => Python
- Libevent => C
- EventMachine => Ruby

Node.js is not…

- Another Web framework
- For beginners
- Supporting Multi-thread

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

1. node.js Intro / RECAP

Development of Server-side for the Web & others …

Traditional desktop applications have a user interface wired up with background logic

- user interfaces are based on Windows Forms, Jswing, WPF, Gtk, Qt, etc. and dependant on operating system

• On the web user interfaces are standardized

- HTML for markup
- CSS for style
- JavaScript for dynamic content

• In the past proprietary technologies existed on the web, e.g. Adobe Flash, ActiveX: They are slowly dying …

Server Side technologies include: **Java Servlet/JSP**, PHP, ASP .NET, Rails, Django, and: **node.js**

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

1. node.js Intro / RECAP

Why node.js?

- Non Blocking I/O
- V8 Java-Script Engine
- Single Thread with Event Loop
- 40,000+ modules
- 1 Language for Frontend and Backend
- Active community

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

1. node.js Intro / RECAP

What is node.js?

« A platform built on Chrome's JavaScript runtime for easily building fast, scalable network applications. » http://nodejs.org/

- Asynchronous I/O framework
- Core in C++ on top of v8
- Rest of it, in JavaScript / ECMAScript
- Swiss army knife for network Related stuffs
- Can handle thousands of Concurrent connections with Minimal overhead (CPU/Memory) on a single process
- It's NOT a web framework, and it's also NOT a language
- It is a development platform basing on interpreting JavaScript / ECMAScript code in native C/C++

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

1. node.js Intro / RECAP

Why JavaScript/ECMAScript?

- Friendly call-backs
- Ubiquitous
- No I/o Primitives
- One language to RULE them all

JavaScript is well known for client-side scripts running inside the browser

node.js is JavaScript running on the server-side

SSJS -> Server-Side JavaScript

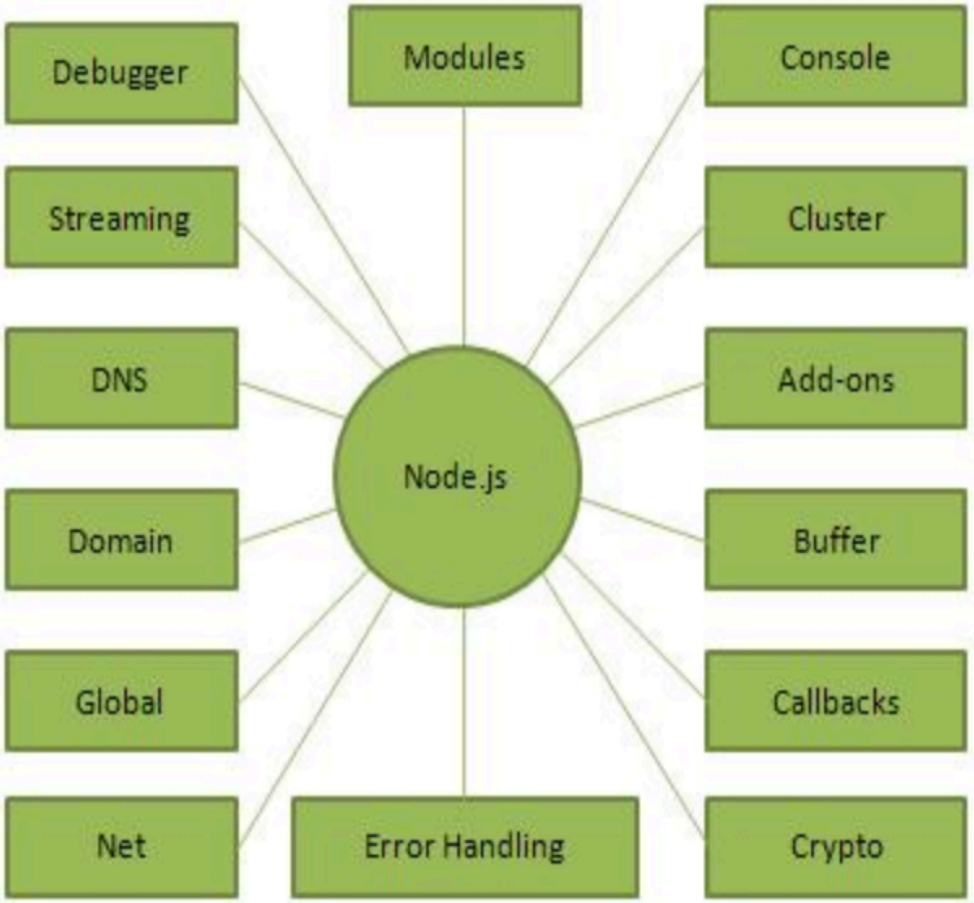
Use a language you know

Use the same language for client side and server side

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

Concepts

The following diagram depicts some important parts of Node.js which we will discuss in detail in the subsequent chapters.

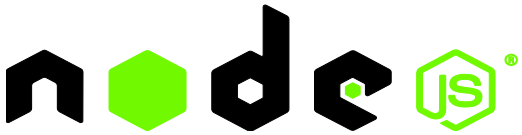


JavaScript Engines……

Every browser has **its own VM**

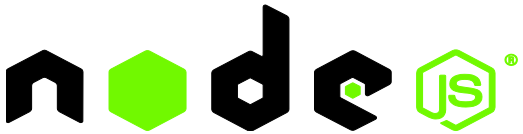
Firefox? **Spidermonkey**
Internet Explorer? **Chakra**
Chrome? **V8**
Safari? **JavaScriptCore**
Opera? **Carakan**

Also **Rhino**, stand alone



- In simple words Node.js is **'server-side JavaScript'**.
- In *not-so-simple* words Node.js is a high-performance **network applications framework**, well optimized for high concurrent environments.
- It's a **command line** tool.
- In 'Node.js' , '**.js**' doesn't mean that its solely written JavaScript. It is 40% JS and 60% C++.
- From the official site:

'Node's goal is to provide an easy way to build scalable network programs' - (from nodejs.org!)



https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

Introduction: Advanced (& Confusing)

- Node.js uses an **event-driven**, **non-blocking I/O** model, which makes it lightweight. (from nodejs.org!)
- It makes use of **event-loops** via JavaScript's **callback** functionality to implement the non-blocking I/O.
- Programs for Node.js are written in JavaScript but not in the same JavaScript we are use to. There is no DOM implementation provided by Node.js, i.e. you **can not** do this:

```
var element = document.getElementById("elementId");
```

- Everything inside Node.js runs in a **single-thread**.

https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

EXAMPLE-1: GETTING STARTED & HELLO WORLD

- Install/build Node.js.
  - (Yes! Windows installer is available!)
- Open your favorite editor and start typing JavaScript.
- When you are done, open cmd/terminal and type this:

  `'node YOUR_FILE.js'`

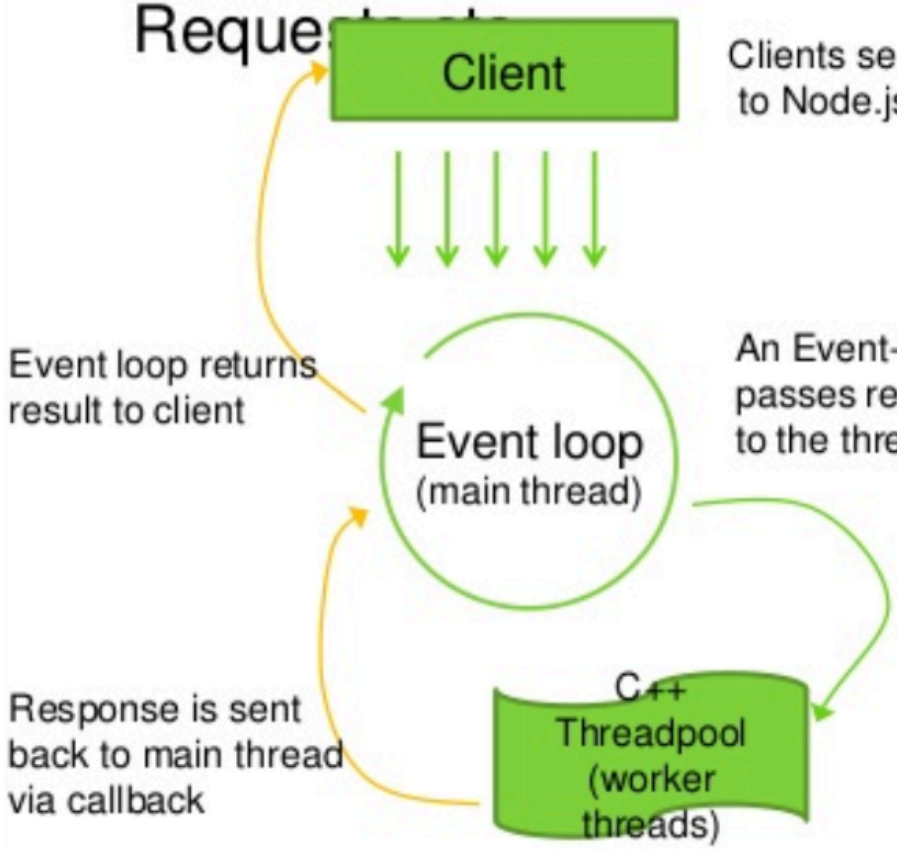
- Here is a simple example, which prints '*hello world*'

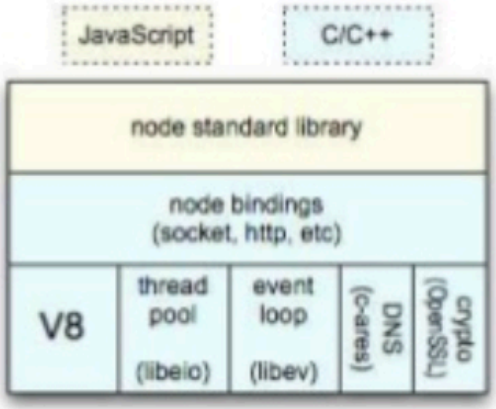
```
var sys = require("sys");
setTimeout(function(){
sys.puts("world");},3000);
sys.puts("hello");
//it prints 'hello' first and waits for 3 seconds and then
  prints 'world'
```

https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

SOME THEORY: EVENT-LOOPS

- Event-loops are the core of event-driven programming, almost all the UI programs use event-loops to track the user event, for example: Clicks, Ajax Reque





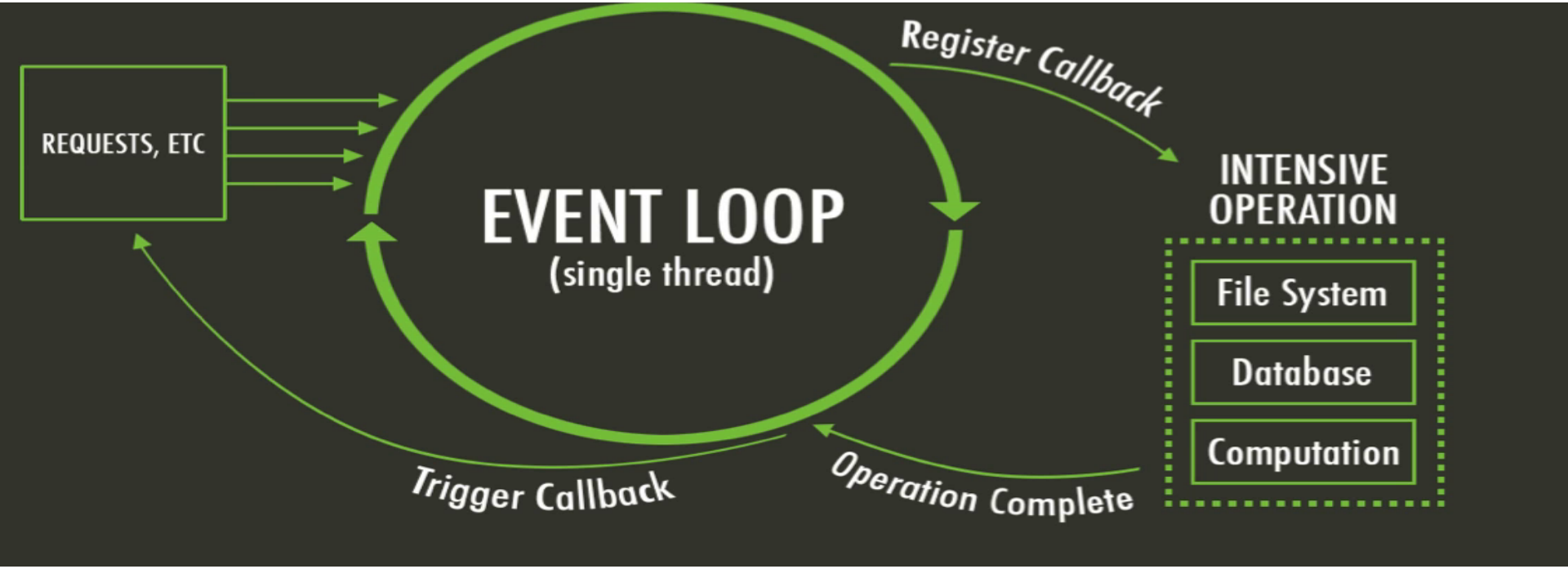
https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

The idea behind node.js....

- Perform asynchronous processing on single thread instead of classical multithread processing, minimize overhead & latency, maximize scalability
- Scale horizontally instead of vertically
- Ideal for applications that serve a lot of requests but dont use/need lots of computational power per request
- Not so ideal for heavy calculations, e.g. massive parallel computing
- Also: Less problems with concurrency

https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

Node.js Event Loop



There are a couple of implications of this apparently very simple and basic model

- Avoid synchronous code at all costs because it blocks the event loop
- Which means: callbacks, callbacks, and more callbacks

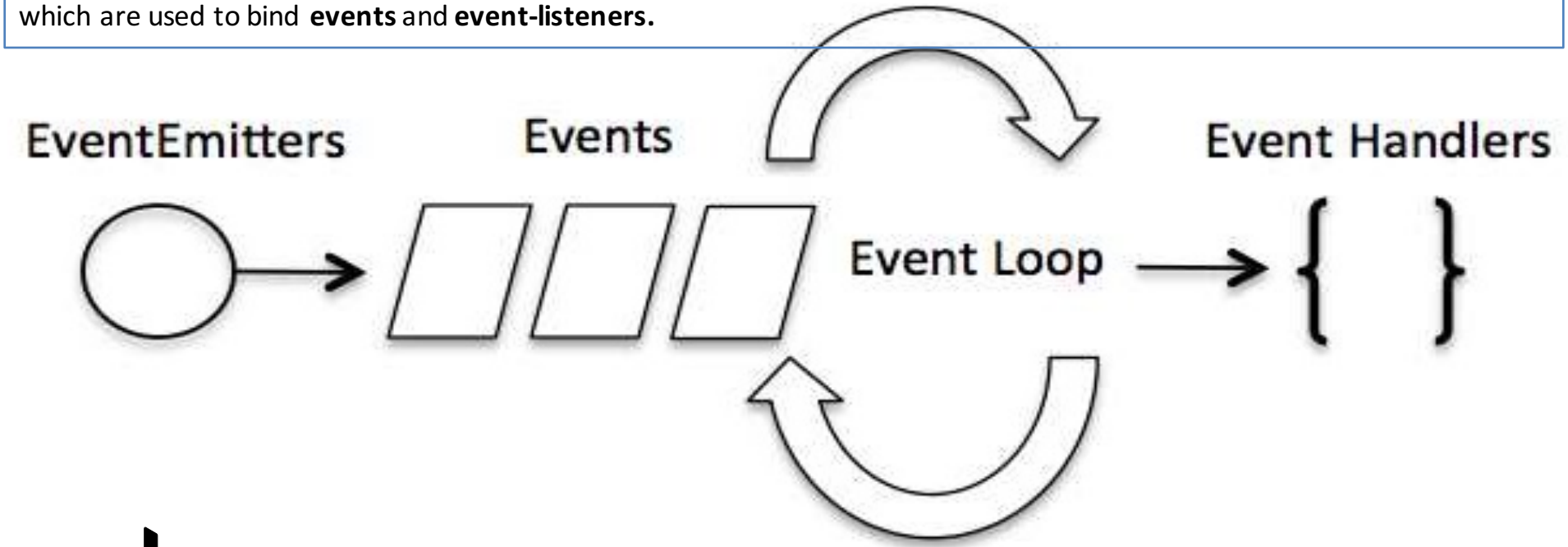
https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

Node.js is a single-threaded application, but it can support concurrency via the concept of **event** and **callbacks**. Every API of Node.js is asynchronous and being single-threaded, they use **async function calls** to maintain concurrency. Node uses observer pattern. Node thread keeps an event loop and whenever a task gets completed, it fires the corresponding event which signals the event-listener function to execute.

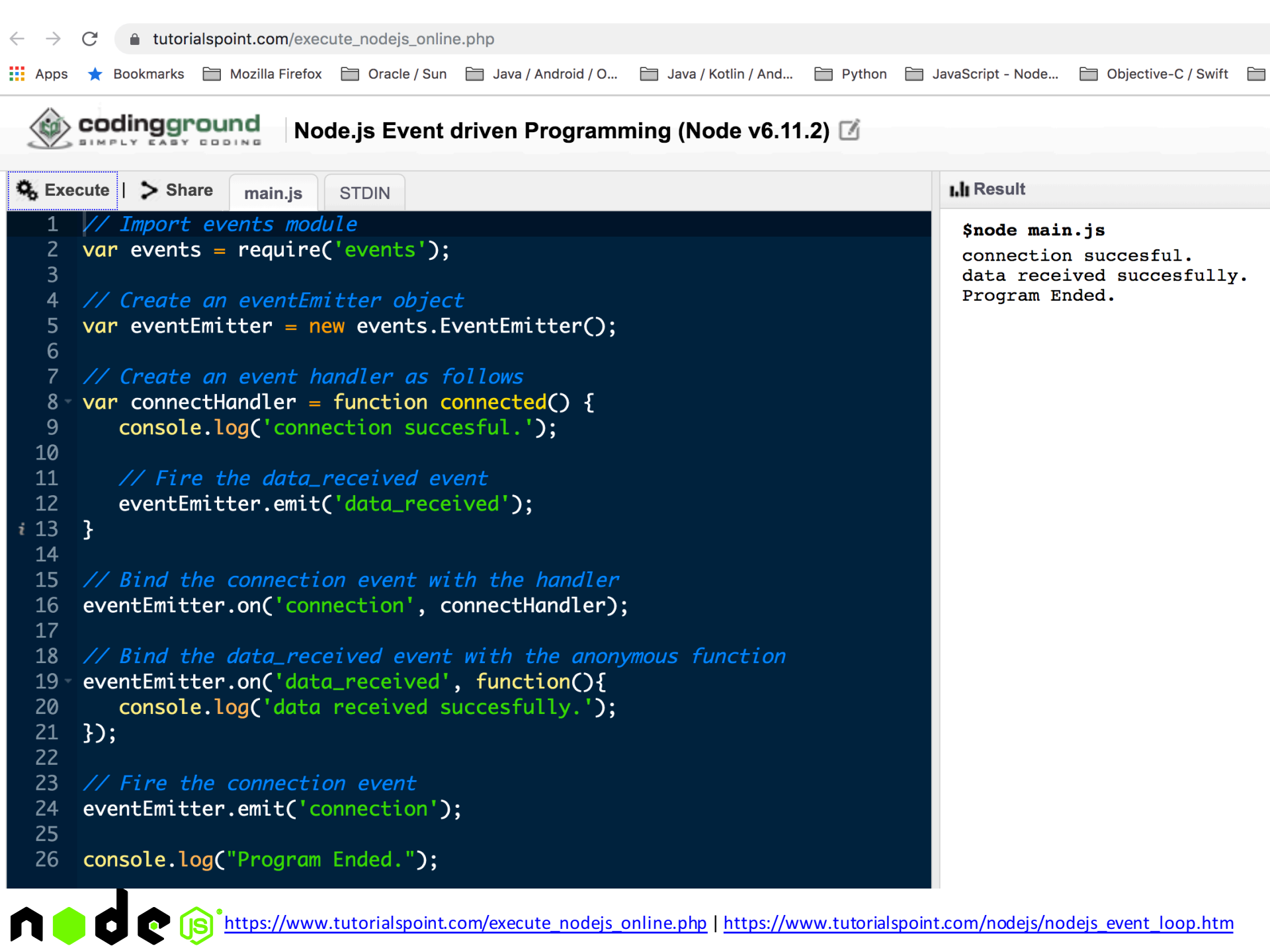
Event-Driven Programming

Node.js uses events heavily and it is also one of the reasons why Node.js is pretty fast compared to other similar technologies. As soon as Node starts its server, it simply initiates its variables, declares functions and then simply waits for the event to occur.

In an event-driven application, there is generally a main loop that listens for events, and then triggers a callback function when one of those events is detected. Although events look quite similar to callbacks, the difference lies in the fact that callback functions are called when an asynchronous function returns its result, whereas event handling works on the observer pattern. The functions that listen to events act as **Observers**. Whenever an event gets fired, its listener function starts executing. Node.js has multiple in-built events available through events module and **EventEmitter** class which are used to bind **events** and **event-listeners.**



https://www.tutorialspoint.com/execute_nodejs_online.php | https://www.tutorialspoint.com/nodejs/nodejs_event_loop.htm




```
1  // Import events module
2  var events = require('events');
3
4  // Create an eventEmitter object
5  var eventEmitter = new events.EventEmitter();
6
7  // Create an event handler as follows
8  var connectHandler = function connected() {
9     console.log('connection succesful.');
10
11    // Fire the data_received event
12    eventEmitter.emit('data_received');
13 }
14
15 // Bind the connection event with the handler
16 eventEmitter.on('connection', connectHandler);
17
18 // Bind the data_received event with the anonymous function
19 eventEmitter.on('data_received', function(){
20    console.log('data received succesfully.');
21 });
22
23 // Fire the connection event
24 eventEmitter.emit('connection');
25
26 console.log("Program Ended.");
```

Blocking vs Non-Blocking……

Example :: Read data from file and show data

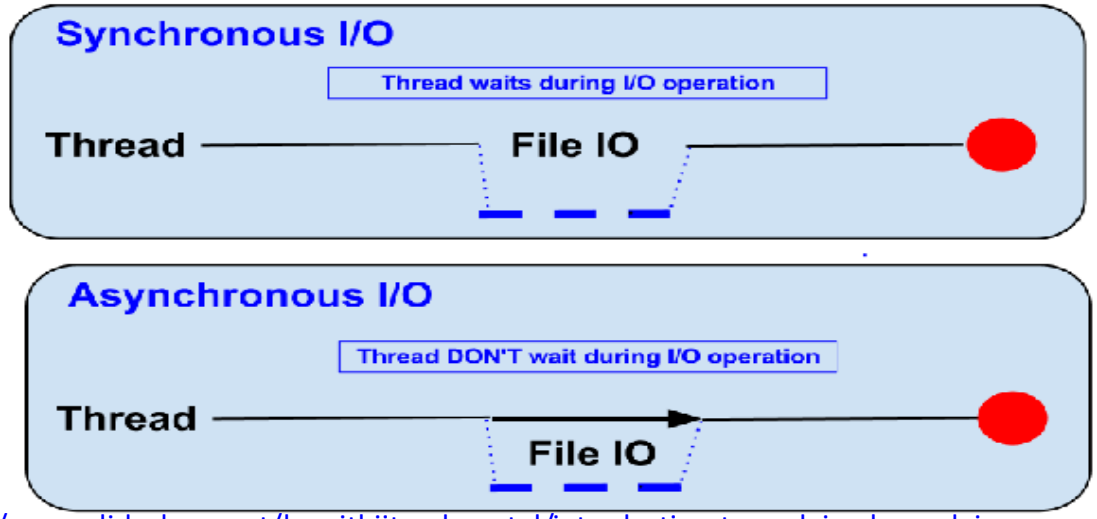
SOME THEORY: NON-BLOCKING I/O

- Traditional I/O

```
var result = db.query("select x from table_Y");
doSomethingWith(result); //wait for result!
doSomethingWithOutResult(); //execution is blocked!
```

- Non-traditional, Non-blocking I/O

```
db.query("select x from table_Y",function (result){
  doSomethingWith(result);
});
doSomethingWithOutResult();
  delay!
```



https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

Node.js for….

- Web application
- Web-socket server
- Ad server
- Proxy server
- Streaming server
- Fast file upload client
- Any Real-time data apps
- Anything with high I/O

Where to Use Node.js?

• Following are the areas where Node.js is proving itself as a perfect technology partner.

• I/O bound Applications

• Data Streaming Applications

• Data Intensive Real-time Applications (DIRT)

• JSON APIs based Applications

• Single Page Applications

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

In NodeJS

Standard JavaScript with

- Buffer
- C/C++ Addons
- Child Processes
- Cluster
- Console
- Crypto
- Debugger
- DNS
- Domain
- Events
- File System
- Globals
- HTTP
- HTTPS
- Modules
- Net
- OS
- Path
- Process
- Punycode
- Query Strings
- Readline
- REPL
- Stream
- String Decoder
- Timers
- TLS/SSL
- TTY
- UDP/Datagram
- URL
- Utilities
- VM
- ZLIB

... but without DOM manipulation

https://www.slideshare.net/bewithjitendrapatel/introduction-to-nodejs-aka-nodejs

When to not use Node.js

- When you are doing heavy and CPU intensive calculations on server side, because event-loops are CPU hungry.
- Node.js API is still in beta, it keeps on changing a lot from one revision to another and there is a very little backward compatibility. Most of the packages are also unstable. Therefore is not yet production ready.
- Node.js is a no match for enterprise level application frameworks like Spring(java), Django(python), Symfony(php) etc. Applications written on such platforms are meant to be highly user interactive and involve complex business logic.

https://www.slideshare.net/vikasing/introduction-to-nodejs-11730771

- Read the free O'reilly Book *'Up and Running with Node.js'* @ http://ofps.oreilly.com/titles/9781449398583/
- Visit www.nodejs.org for Info/News about Node.js
- Watch Node.js tutorials @ http://nodetuts.com/
- For Info on MongoDB: http://www.mongodb.org/display/DOCS/Home
- For anything else **Google!**

Section Conclusion

Fact: **node.js is used in DAD**

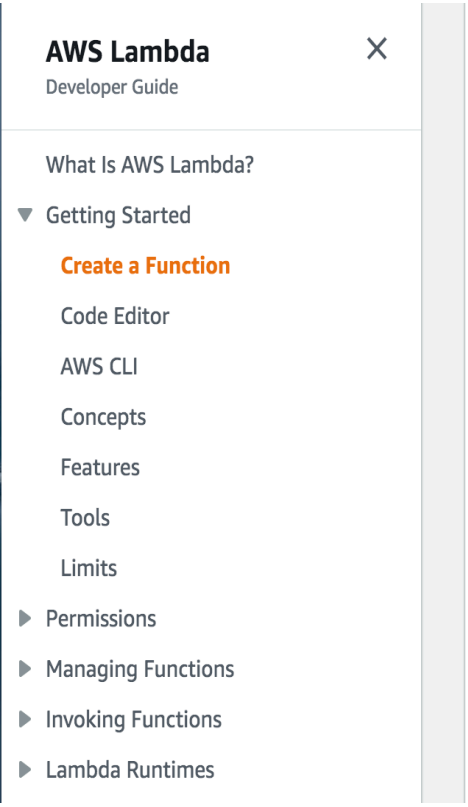
In few **samples** it is simple to remember: ECMAScript / JavaScript RECAP and node.js best practice for DAD / REST Services development and FaaS – Function as a Service implementation

FaaS: Function as a Service – Amazon AWS Lambda example

Server-less Development – FaaS: AWS Lambda

2. AWS Lambda – FaaS Cloud

https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html



Create a Lambda Function with the Console

PDF | Kindle | RSS

In this Getting Started exercise you create a Lambda function using the AWS Lambda console. Next, you manually invoke the Lambda function using sample event data. AWS Lambda executes the Lambda function and returns results. You then verify execution results, including the logs that your Lambda function created and various CloudWatch metrics.

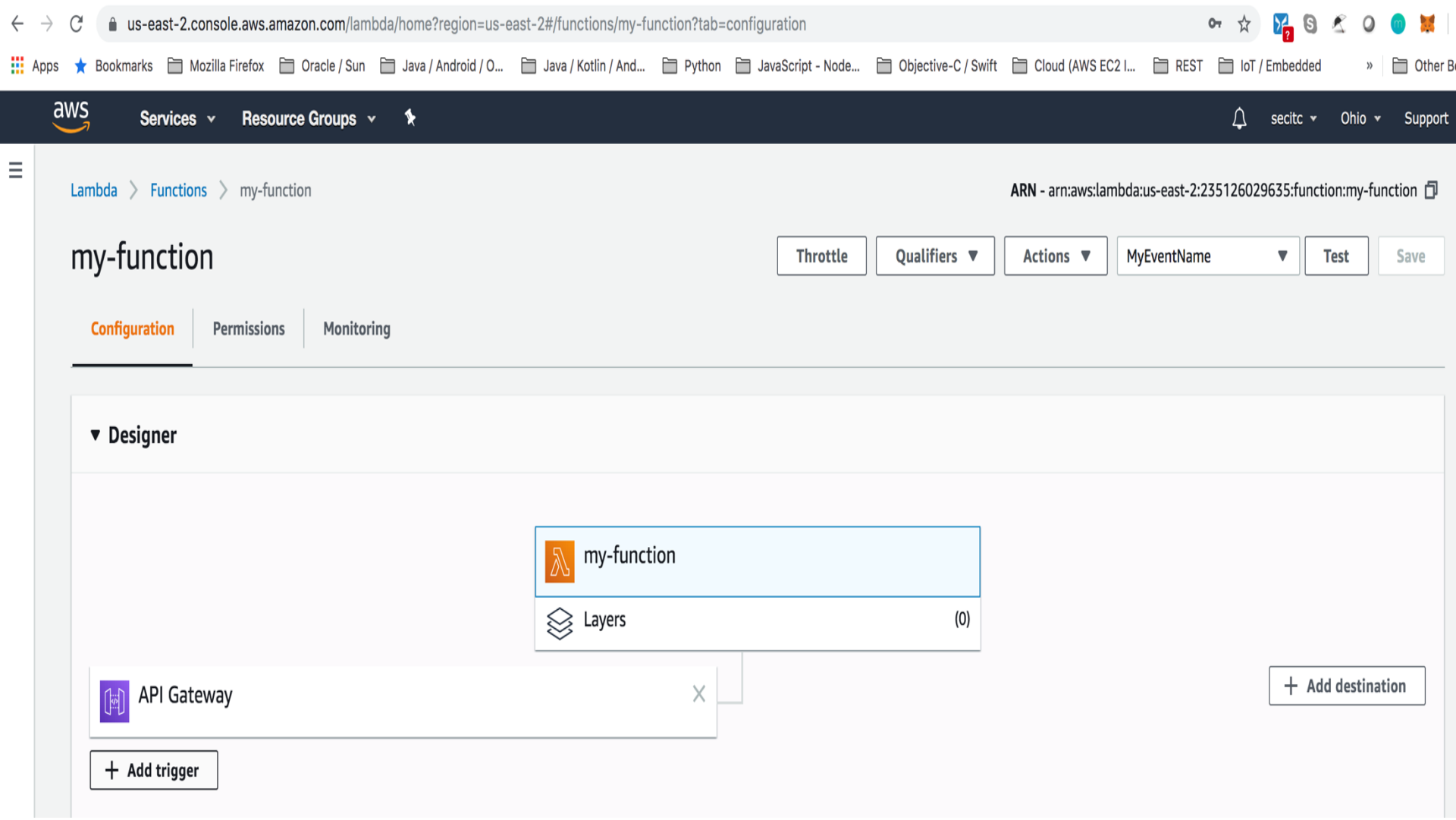
To create a Lambda function

1. Open the AWS Lambda console.
2. Choose **Create a function**.
3. For **Function name**, enter `my-function`.
4. Choose **Create function**.

Lambda creates a Node.js function and an execution role that grants the function permission to upload logs. Lambda assumes the execution role when you invoke your function, and uses it to create credentials for the AWS SDK and to read data from event sources.

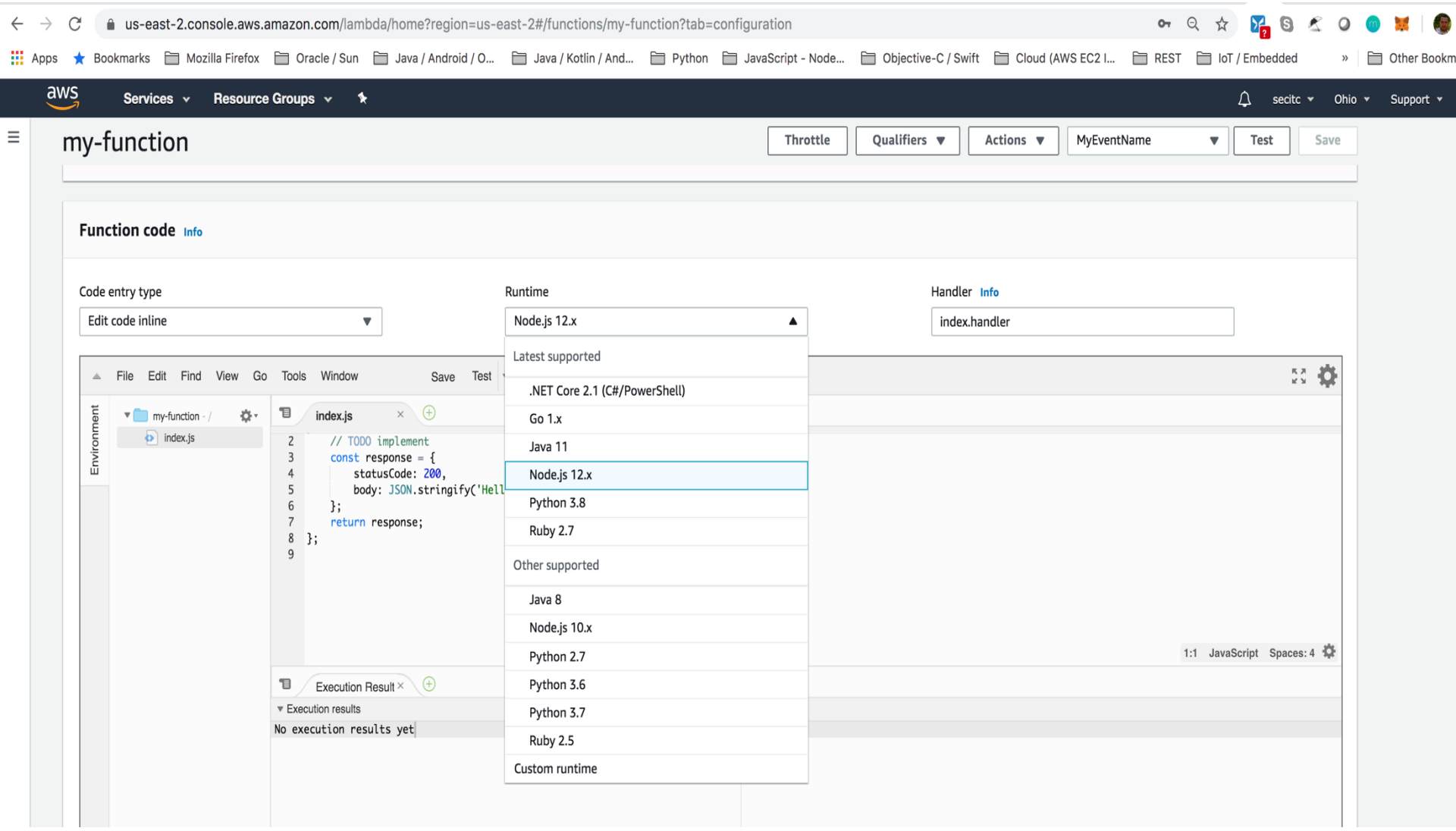
https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

2. AWS Lambda – FaaS Cloud



https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

2. AWS Lambda – FaaS Cloud

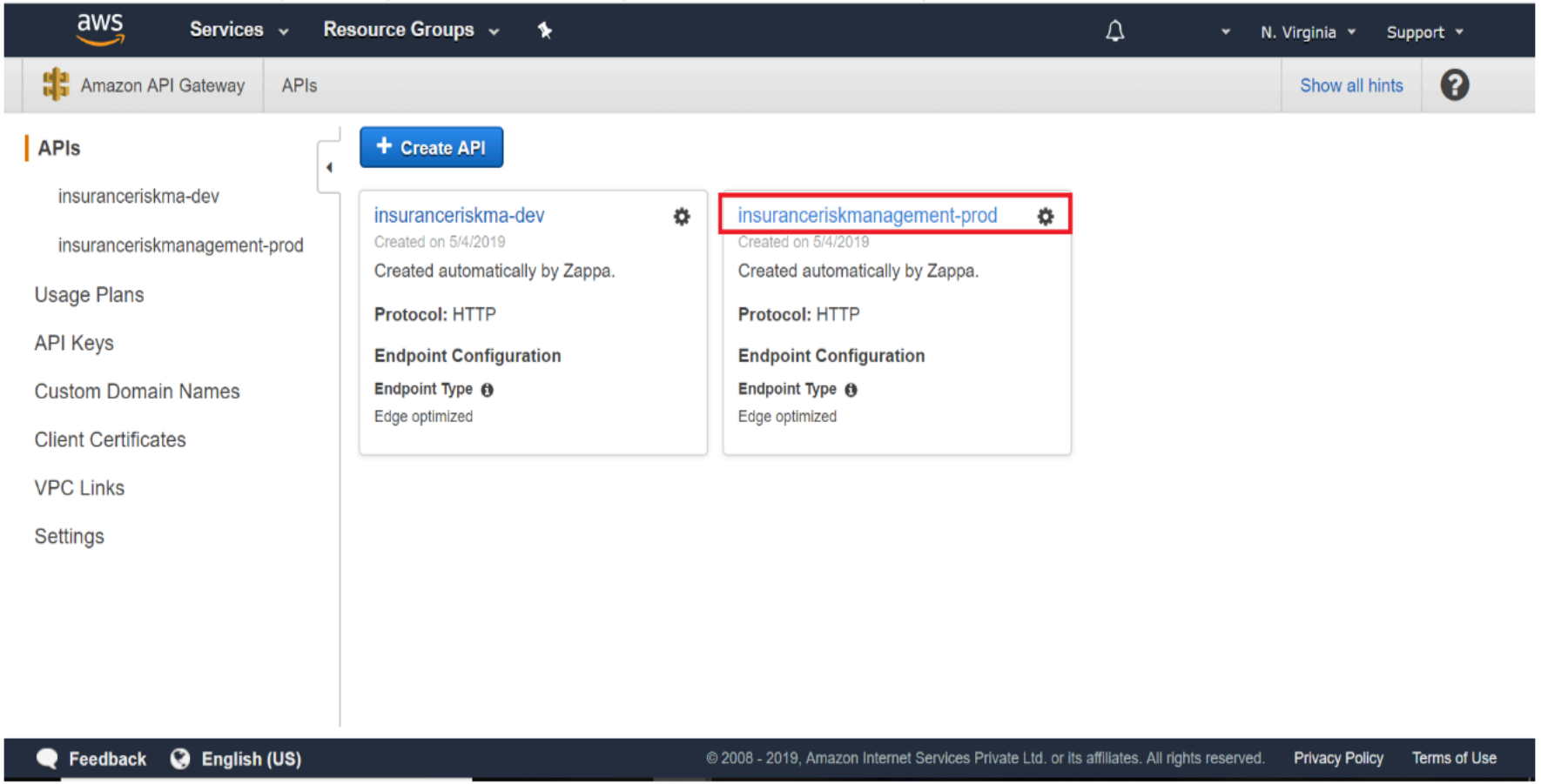


https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

2. AWS Lambda – FaaS Cloud

1. go to https://console.aws.amazon.com/apigateway

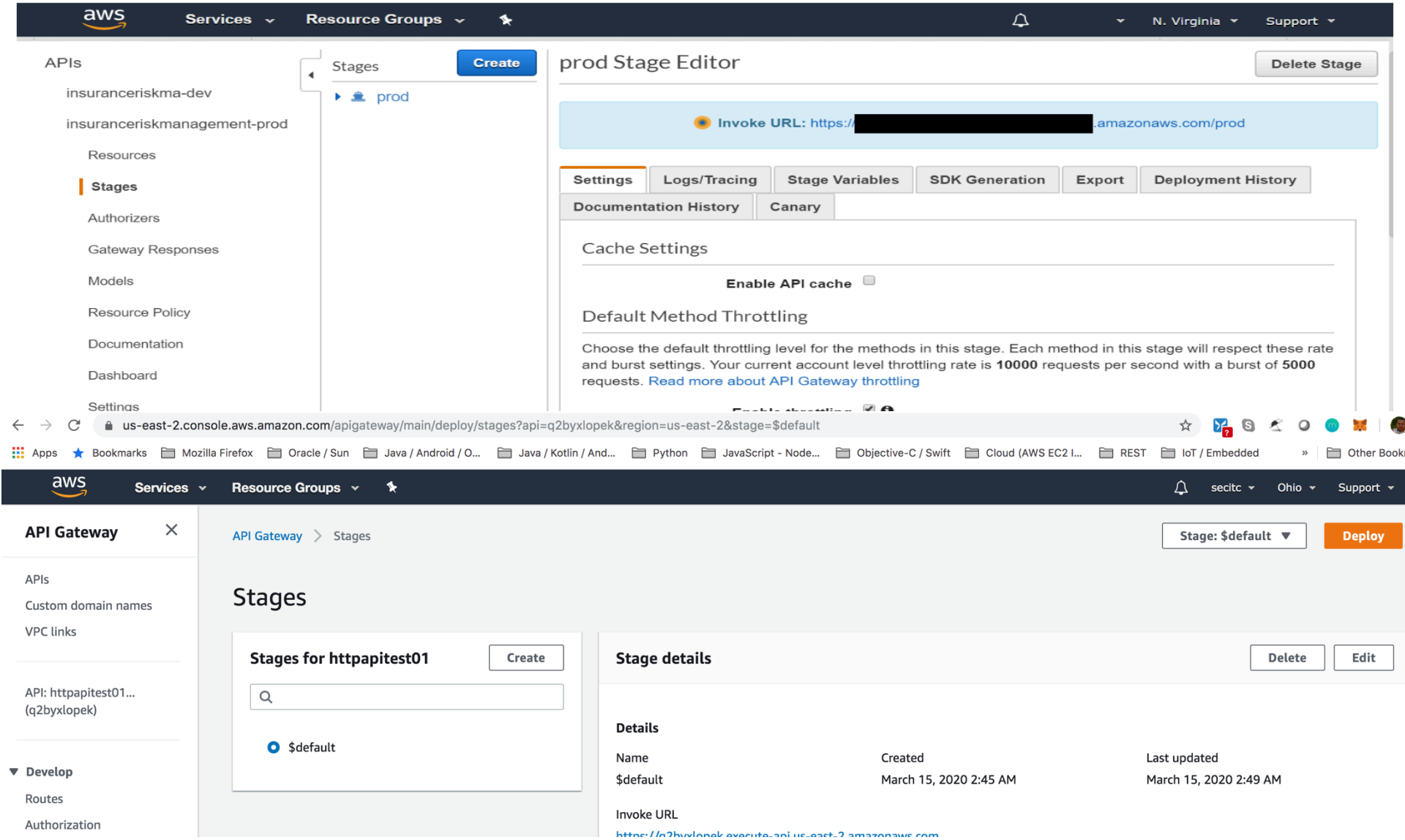
2. select **api link** (which you have deployed on aws lambda).



https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

2. AWS Lambda – FaaS Cloud

3. select **stages** in left side panel and see the **invoke url**.



https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

2. AWS Lambda – FaaS Cloud

Adding a http listener can be done by going to your lambda function, selecting the 'triggers' tab and 'add trigger', finally selecting API Gateway - but as others mentioned this does create a public facing url.

https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html | https://docs.aws.amazon.com/lambda/latest/dg/getting-started-create-function.html

Section Conclusions

JMS – Java Message Service is JEE implementation for MOM – Message Oriented Middleware

The chosen Java Web & Application Server with JMS implementation for testing is Apache TomEE

Apache Kafka demo

JMS DEMO
for easy sharing

Distributed Application Development

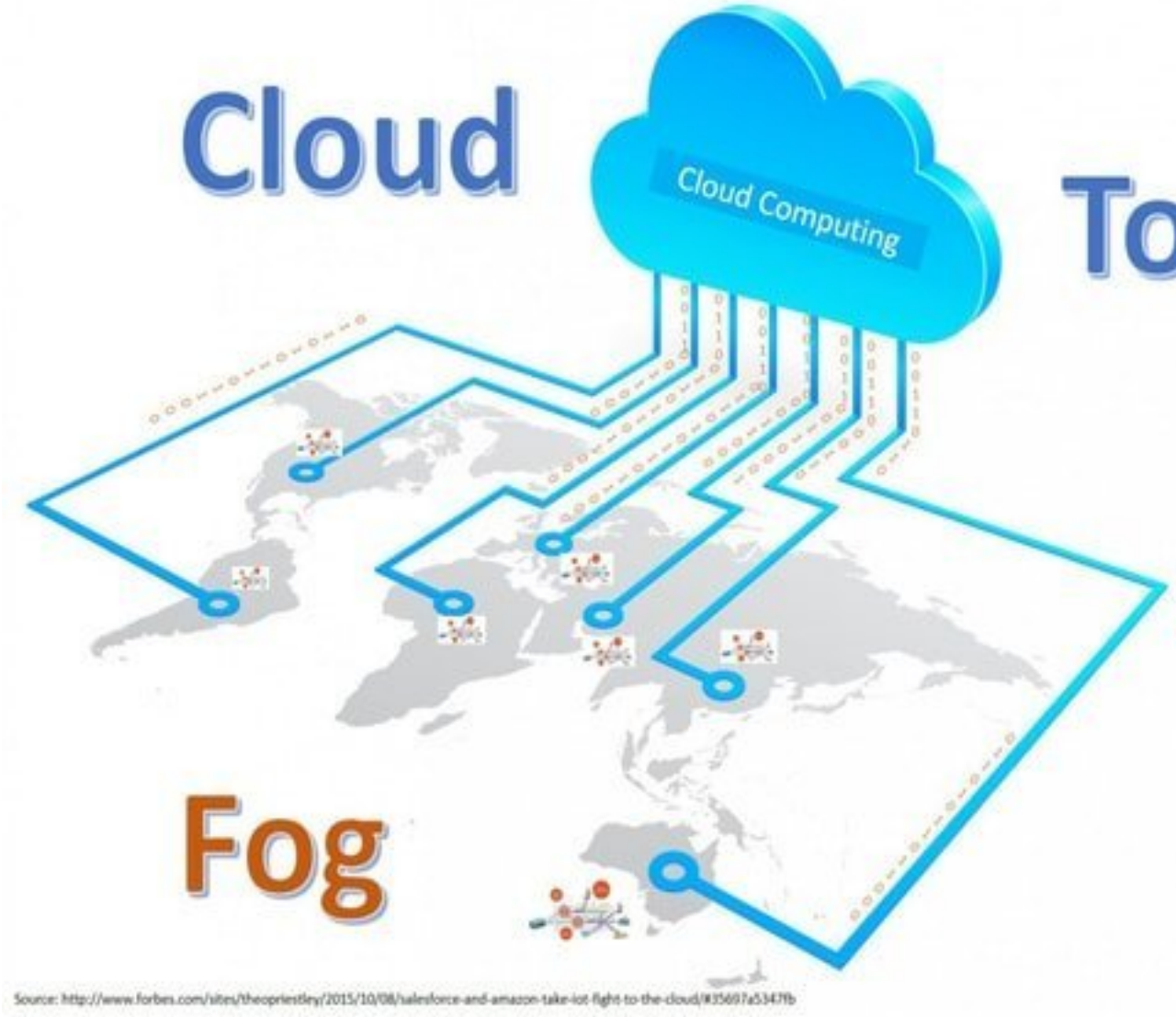
Communicate & Exchange Ideas

Questions & Answers!

But wait…

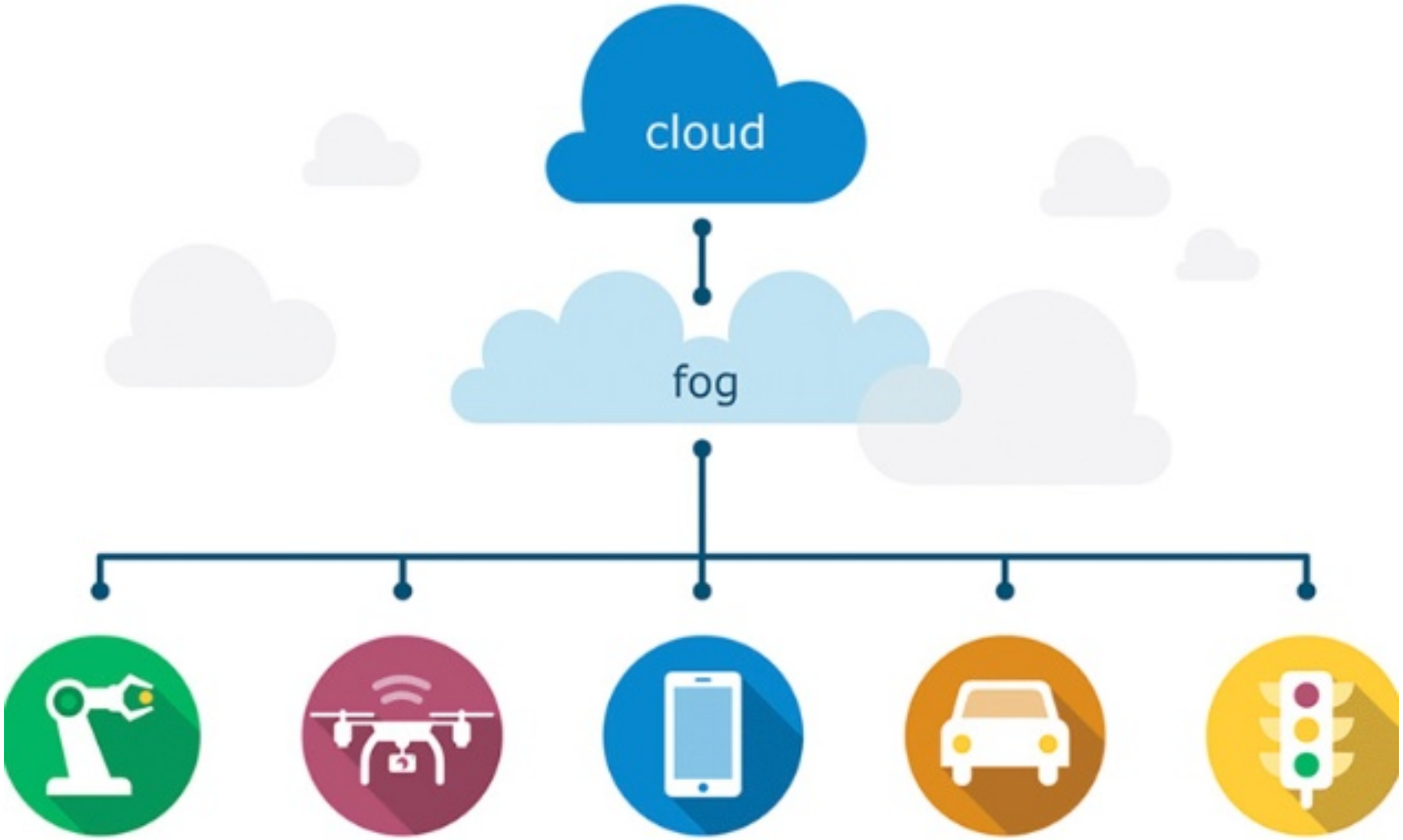
There's More!

Fog Computing vs. FaaS



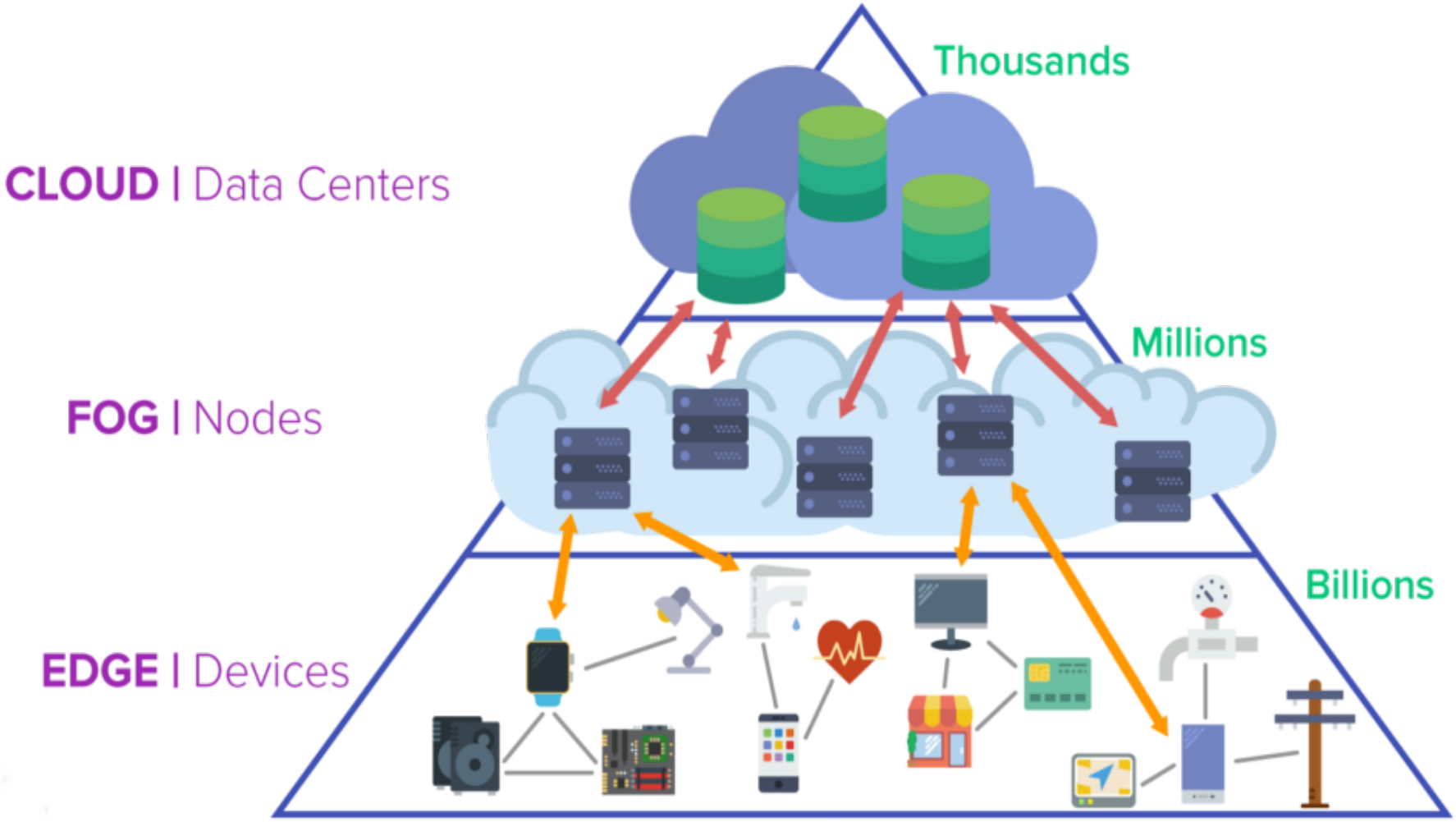
https://erpinnews.com/fog-computing-vs-edge-computing | https://leanbi.ch/en/blog/iot-and-predictive-analytics-fog-and-edge-computing-for-industries-versus-cloud-19-1-2018/

Fog Computing



https://erpinnews.com/fog-computing-vs-edge-computing | https://leanbi.ch/en/blog/iot-and-predictive-analytics-fog-and-edge-computing-for-industries-versus-cloud-19-1-2018/

Fog Computing



https://erpinnews.com/fog-computing-vs-edge-computing | https://leanbi.ch/en/blog/iot-and-predictive-analytics-fog-and-edge-computing-for-industries-versus-cloud-19-1-2018/

Thanks!

DAD – Distributed Application Development
End of Lecture 09